



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria
Corso di Laurea Triennale in Ingegneria Informatica

Appunti

Calcolo Numerico

Professori:
Prof. Stefano Massei

Autore:
Enea Passardi

Anno Accademico 2024/2025 (2026)

INDICE

Unimap	3
Introduzione	7
 I Teoria degli errori	 8
1 Rappresentazione dei numeri	9
1 Rappresentazione interna di un calcolatore	10
2 Numeri sotto-normalizzati	13
3 Operazioni macchina	13
 2 Approssimazioni di funzioni	14
1 Errore inerente	15
2 Errore algoritmico	16
3 Problemi dell'approssimazione numerica	17
3.1 Problema diretto	17
3.2 Problema indiretto	18
4 Approssimazioni di funzioni non razionali	18
 II Metodi numerici per sistemi lineari	 20
3 Autovalori e Autovettori	21
1 Matrici simili	22
2 Matrici Hermitiane	23
3 Norme	24
4 Dischi di Gershgorin	26
 4 Sistemi lineari	30
1 Matrici a blocchi	30
2 Metodi diretti	33
2.1 Fattorizzazione LU	34
3 Pivoting	35
4 Condizionamento di un sistema lineare	36
5 Metodi iterativi	37
5.1 Esempi di metodi iterativi	39

5	Sistemi rettangolari	42
1	Problema ai minimi quadrati	42
2	Fattorizzazione QR	45
2.1	Ricavare la fattorizzazione QR	45
III	Interpolazione	48
6	Interpolazione polinomiale	49
1	Base di Lagrange	50
2	Base di Newton	51
3	Interpolazione osculatoria di Hermite	52
7	Approssimazione a tratti e funzioni spline	54
1	Interpolazione a tratti	55
2	Interpolazione ai minimi quadrati	57
IV	Integrazione numerica	58
8	Formule di quadratura e grado di precisione	59
1	Errore nelle formule di quadratura	61
2	Formule di Newton-Cotes	63
3	Integrazione su intervalli	64
4	Formule Gaussiane	65
5	Formule di quadratura che coinvolgono derivate	66
V	Equazioni non lineari	67
9	Definizioni e Metodi iterativi	68
1	Metodo di bisezione	70
2	Metodo delle secanti	70
3	Metodi iterativi ad un punto	71
4	Metodo di newton	74
10	Sistemi non lineari	77
1	Metodo di Newton-Raphson	78
VI	Problemi agli autovalori	80
11	Metodo delle potenze	81
1	Metodo delle potenze	81
2	Metodo delle potenze inverse	84
3	Metodo delle potenze inverse con shift	84
12	Metodo QR	85
1	Calcolo degli autovettori di una matrice triangolare	85
2	Come si definisce la successione $\{A_n\}$	86
VII	Appendice	88
A	Ricavare la matrice di permutazione per una matrice riducibile	89

Unimap

1. **Lun 24/02/2025 08:30-10:30 (2:0 h)** lezione: Introduzione ai numeri in virgola mobile, teorema di rappresentazione, proprietà di mantissa ed esponente. Distribuzione dei numeri floating point sulla retta reale, errore assoluto e relativo nella rappresentazione di numeri reali: arrotondamento per troncamento e round-to-nearest. Implementazione dei numeri floating point sul calcolatore, numeri sottonormalizzati (subnormal numbers). (STEFANO MASSEI)
2. **Ven 28/02/2025 10:30-13:30 (3:0 h)** lezione: Operazioni aritmetiche fra numeri floating point, perdita di proprietà classiche come associatività e distributività. Errore inerente ed errore algoritmico nel calcolo di una funzione. Coefficienti di amplificazione per gli errori assoluti e relativi delle operazioni aritmetiche di base. Formula per stimare l'errore inerente. Metodo del grafo per la stima dell'errore algoritmico. Matlab/Octave: esempi di perdita precisione nei calcoli in aritmetica finita dovuti ad arrotondamento, overflow e cancellazione numerica. Esempi di problema diretto e problema inverso riguardanti l'errore assoluto. (STEFANO MASSEI)
3. **Lun 03/03/2025 08:30-10:30 (2:0 h)** lezione: Analisi dell'errore nella valutazione di funzioni razionali, esempi di problema diretto e problema inverso riguardanti l'errore assoluto. Valutazione di funzioni non razionali, errore analitico. Limitare l'errore analitico mediante il resto dell'espansione di Taylor, approssimazione della funzione esponenziale. (STEFANO MASSEI)
4. **Ven 07/03/2025 10:30-13:30 (3:0 h)** lezione: Richiami di algebra lineare: matrici, vettori, costo di operazioni matriciali e vettoriali, proprietà del prodotto di matrici. Definizione di vettori linearmente indipendenti, rango di una matrice, sottomatrici, minori e determinante. Teorema di Binet-Cauchy. Inversa di una matrice, matrici strutturate: Hermitiane, anti-Hermitiane, unitarie, ortogonali, normali, simmetriche ed anti-simmetriche. Matrici di permutazione. Sistemi lineari, teorema di Rouchè-Capelli per determinare l'esistenza e la dimensione dello spazio delle soluzioni. Esercitazione Matlab/Octave: esempi di varie funzionalità del software per definire e accedere a strutture dati vettoriali e matriciali; misurazione e verifica delle complessità computazionali di somma e prodotto tra matrici. (STEFANO MASSEI)
5. **Lun 10/03/2025 08:30-10:30 (2:0 h)** lezione: Richiami sulla definizione di autovalori e autovettori: polinomio caratteristico, equazione caratteristica. Trasformazioni per similitudine. molteplicità algebrica e molteplicità geometrica degli autovalori. Diagonalizzabilità di una matrice. Relazione tra traccia e determinante di una matrice con gli autovalori. Autovalori di una matrice hermitiana. Autovalori ed autovettori di potenze e polinomi di matrice. (STEFANO MASSEI)
6. **Ven 14/03/2025 10:30-13:30 (3:0 h)** lezione: Norme matriciali indotte. Norme vettoriali e norme matriciali compatibili tra loro. Relazione tra raggio spettrale e norma matriciale: teorema di Hirsch. Cerchi di Gershgorin, primo Teorema di Gershgorin con dimostrazione. Conseguenza del primo teorema di Gershgorin per matrici a predominanza diagonale forte. Raffinamento della stima di Gershgorin utilizzando la matrice trasposta. Esercitazione Matlab/Octave: implementazione di routine per disegnare i cerchi di Gershgorin. (STEFANO MASSEI)
7. **Lun 17/03/2025 08:30-10:30 (2:0 h)** lezione: II° e III° Teorema di Gershgorin. Conseguenze dei teoremi di Gershgorin nel caso di cerchi disgiunti e per matrici a predominanza diagonale debole. Matrici a blocchi, determinante di matrici triangolari con blocchi diagonali quadrati, operazioni aritmetiche con matrici blocchi. Definizione di matrice riducibile, risoluzione di sistemi lineari con matrice dei coefficienti riducibile. (STEFANO MASSEI)
8. **Ven 21/03/2025 10:30-13:30 (3:0 h)** lezione: Teorema con dimostrazione: matrice riducibile se e solo se il grafo associato non è fortemente connesso. Metodo di sostituzione all'indietro a blocchi. Metodi diretti e metodi iterativi per risolvere sistemi lineari; esempio: il metodo

- di Cramer. Metodo dell'eliminazione di Gauss per risolvere sistemi lineari. Equivalenza dell'algoritmo con il calcolo della fattorizzazione LU. Complessità computazionali del metodo di sostituzione all'indietro, metodo di Gauss e metodo di Cramer. Esercitazione Matlab/Octave: Matrici di permutazione e riducibilità, implementazione dei metodi di sostituzione all'indietro per sistemi triangolari, implementazione del metodo di eliminazione di Gauss. (STEFANO MASSEI)
9. **Lun 24/03/2025 08:30-10:30 (2:0 h)** lezione: Metodo di eliminazione di Gauss con strategia di pivoting parziale, calcolo della fattorizzazione LU con pivoting per colonne. Uso del metodo di Gauss per il calcolo del determinante di una matrice. Condizionamento di un sistema lineare: relazione tra errore relativo sulla matrice e sul termine noto con l'errore relativo sulla soluzione. Numero di condizionamento. Relazione tra il residuo associato ad una approssimazione della soluzione e la sua distanza dalla soluzione esatta. (STEFANO MASSEI)
 10. **Ven 28/03/2025 10:30-13:30 (3:0 h)** lezione: Metodi iterativi per sistemi lineari, descrizione dell'idea e delle situazioni in cui possono essere convenienti. Relazioni tra errore e residuo associati ad un'iterazione. Condizione di convergenza di un metodo iterativo e criteri di arresto. Metodi di Jacobi e Gauss Seidel. Esercitazione Matlab/Octave: eliminazione di Gauss per la soluzioni di sistemi con più di un termine noto e per il calcolo del determinante; implementazione dei metodi di Jacobi e Gauss Seidel. (STEFANO MASSEI)
 11. **Lun 31/03/2025 08:30-10:30 (2:0 h)** lezione: Metodi di Jacobi e Gauss Seidel: convergenza per matrici a predominanza diagonale, con dimostrazione. Esempi di calcolo del raggio spettrale della matrice di iterazione, per Jacobi e Gauss Seidel. Metodo di Gauss con pivoting per sistemi rettangolari. Sistemi sovradeterminati, risolvere un sistema lineare nel senso dei minimi quadrati. Metodo delle equazioni normali. Teorema di esistenza e unicità della soluzione del problema ai minimi quadrati con dimostrazione. (STEFANO MASSEI)
 12. **Ven 04/04/2025 10:30-13:30 (3:0 h)** lezione: Fattorizzazione QR: definizione e teorema di esistenza. Calcolo della fattorizzazione QR mediante matrici di Householder e costo dell'algoritmo. Metodo QR per risolvere problemi lineari ai minimi quadrati. Analisi del costo e confronto con il metodo delle equazioni normali. Esempio di risoluzione problema ai minimi quadrati con fattorizzazione QR. Esercitazione Matlab/Octave: implementazione del calcolo della fattorizzazione QR tramite trasformazioni di Householder e risoluzione di problemi ai minimi quadrati. (STEFANO MASSEI)
 13. **Lun 07/04/2025 08:30-10:30 (2:0 h)** lezione: Problema dell'approssimazione e dell'interpolazione di funzioni. Dimostrazione di esistenza ed unicità del polinomio interpolante con grado al più k , dati $k + 1$ nodi di interpolazione. Rappresentazione in basi diverse del polinomio di interpolazione. Rappresentazione nella base di Lagrange. (STEFANO MASSEI)
 14. **Ven 11/04/2025 10:30-13:30 (3:0 h)** lezione: Rappresentazione nella base di Newton: introduzione delle differenze divise e delle loro proprietà. Formula dell'errore di approssimazione e tabelle per il calcolo delle differenze divise. Esempi di calcolo del polinomio interpolante e di minimizzazione del suo grado al variare di parametri. Interpolazione osculatoria di Hermite: definizione del problema, risultato di esistenza ed unicità del polinomio interpolante di Hermite. Accenno alla derivazione dei polinomi della base di Hermite ed espressione completa del polinomio di interpolazione. Esempio di calcolo del polinomio di Hermite nel caso di due nodi. Esercitazione Matlab/Octave: malcondizionamento della matrice di Vandermonde, interpolazione di Lagrange, fenomeno di Runge. (STEFANO MASSEI)
 15. **Lun 14/04/2025 08:30-10:30 (2:0 h)** lezione: Fenomeno di Runge. Interpolazione polinomiale a tratti e funzioni spline. Spline cubiche con condizioni al bordo naturali, periodiche e vincolate. Teorema di esistenza ed unicità della spline cubica naturale su nodi equispaziati, con idea della dimostrazione. Approssimazione di funzioni nel senso dei minimi quadrati, definizione del problema di ottimizzazione e discussione su differenze ed analogie con i problemi di interpolazione. Caratterizzazione dei punti di minimo della somma degli scarti quadratici come soluzione di un sistema lineare sovradeterminato nel senso dei minimi quadrati.

Esempio di approssimazioni ai minimi quadrati con una retta ed una parabola. (STEFANO MASSEI)

16. **Lun 28/04/2025 08:30-10:30 (2:0 h)** lezione: Integrazione numerica: definizione di formula di quadratura e grado di precisione. Derivazione della formula dei trapezi e della formula di Simpson come formule con grado di precisione massimo con 2 e 3 nodi equispaziati nell'intervallo. Formule Gaussiane, ovvero scelta ottimale, rispetto al grado di precisione, di nodi e pesi per una formula di quadratura. Calcolo esplicito nel caso di due nodi per l'intervallo $[-1, 1]$. Teorema di Peano per l'espressione dell'errore. (STEFANO MASSEI)
17. **Ven 02/05/2025 10:30-13:30 (3:0 h)** lezione: Teorema di Peano per l'espressione dell'errore: calcolo dell'espressione dell'errore per la formula dei trapezi e per la formula di Simpson. Formule di quadratura interpolatorie: espressione di nodi e pesi. Formule interpolatorie con nodi equispaziati: formule di Newton-Cotes. Formula dei trapezi e di Simpson come formule di Newton-Cotes. Proprietà di invarianza del segno del nucleo di Peano per le formule di Newton-Cotes ed espressione del resto. Formule di quadratura Gaussiane: definizione come formule di quadratura interpolatorie sugli zeri dei polinomi ortogonali. Esempi: nodi di Chebyshev e Legendre. (STEFANO MASSEI)
18. **Lun 05/05/2025 08:30-10:30 (2:0 h)** lezione: Formule di Newton-Cotes generalizzate: derivazione ed espressione del resto per la formula dei trapezi e di Cavalieri-Simpson. Formule di quadratura che coinvolgono valutazione delle derivate: esercizi. Esercitazione Matlab/Octave: implementazione delle formule generalizzate dei trapezi e di Cavalieri-Simpson. Verifica dell'andamento asintotico dell'errore all'aumentare dei sottointervalli. Confronto delle due formule in base al numero di valutazioni utilizzate per raggiungere una determinata accuratezza. (STEFANO MASSEI)
19. **Ven 09/05/2025 10:30-13:30 (3:0 h)** lezione: Equazioni non lineari, stima di intervalli dove giacciono le radici con tecniche di analisi. Definizione di ordine di convergenza di una successione convergente. Metodo di bisezione: algoritmo, criterio di arresto, numero di iterazioni, ordine e fattore di convergenza, costo computazionale di ogni iterazione. Metodo delle secanti: algoritmo e ordine di convergenza superlineare. Metodi stazionari a un punto. Teorema di convergenza locale con dimostrazione. Teorema sull'ordine di convergenza per iterazioni funzionali derivabili, con dimostrazione. Convergenza monotona ed alternante in base al segno della derivata. (STEFANO MASSEI)
20. **Lun 12/05/2025 08:30-10:30 (2:0 h)** lezione: Esercizi sulla determinazione della convergenza locale, monotonia e ordine di convergenza per iterazioni di punto fisso dipendenti da parametri. Metodo di Newton, interpretazione grafica. Definizione di radice semplice e di radice multipla con caratterizzazione, per funzioni regolari, in termini di numero di derivate che si annullano. Metodo di Newton, teorema di convergenza locale. Esempi di applicazione del metodo di Newton per il calcolo del reciproco. Analisi dell'ordine di convergenza nel caso di radici multiple; modifica del metodo che permette di ristabilire la convergenza superlineare. Esercitazione Matlab/Octave: implementazione e confronto di iterazioni di punto fisso. (STEFANO MASSEI)
21. **Ven 16/05/2025 10:30-13:30 (3:0 h)** lezione: Metodo di Newton: garanzie di convergenza nel caso di segno costante di derivate prima e seconda. Metodi per sistemi di equazioni non lineari. Iterazione di punto fisso vettoriale. Matrice Jacobiana associata ad una funzione da $\mathbb{R}^m$ in $\mathbb{R}^m$. Convergenza locale in caso di raggio spettrale della matrice Jacobiana minore di 1. Metodo di Newton-Raphson. Teorema di convergenza locale per il metodo di Newton-Raphson. Esempi di verifica delle ipotesi del teorema di convergenza per sistemi non lineari in due variabili. Varianti del metodo di Newton-Raphson: metodo di Newton semplificato e metodo di Jacobi-Newton. Vantaggi dal punto di vista computazionale. (STEFANO MASSEI)
22. **Gio 22/05/2025 08:30-10:30 (2:0 h)** lezione: Metodi per il calcolo degli autovalori: metodo delle potenze. Descrizione dell'algoritmo e dimostrazione della convergenza nel caso di matrici hermitiane. Discussione delle assunzioni da fare per garantire la convergenza nel

caso di matrici non hermitiane diagonalizzabili. Calcolo di autovalori non dominanti: metodo delle potenze inverse. (STEFANO MASSEI)

23. **Ven 23/05/2025 10:30-13:30 (3:0 h)** lezione: Metodo QR per il calcolo degli autovettori e autovalori di una matrice: idea dell'algoritmo come successione di matrici simili con convergenza ad una matrice triangolare a blocchi. Relazione fra gli autovettori della matrice limite e quelli della matrice di partenza. Calcolo degli autovettori di una matrice triangolare con elementi diagonali distinti. Esercizi di riepilogo. (STEFANO MASSEI)

Introduzione

Il corso di **Calcolo numerico**, talvolta chiamato anche **Metodi Numerici**, è un corso che ha come obiettivo principale quello di descrivere e analizzare dei **metodi di approssimazione** che permettano, in modo algoritmico, di andare a fare delle approssimazioni - di diverse tipologie - in modo **algoritmico** e **iterativo**. Supponiamo di voler calcolare l'area sottesa alla curva $f(x) = \sin(x^2)$ nell'intervallo (a, b)

$$\int_a^b \sin(x^2) dx$$

Questo integrale non ammette una primitiva esprimibile in termini di funzioni elementari, rendendo impossibile la sua valutazione esatta per via analitica. Tuttavia, possiamo ricorrere a dei metodi di **approssimazione** iterativi che permettono di ottenere una buona valutazione per l'integrale. Lo stesso ragionamento lo possiamo fare anche per le equazioni differenziali. Supponiamo di avere un'equazione differenziale

$$y^{(4)} + e^x y'' + \sin(y) y' + x^3 y = \cos(x)$$

Risolvere un'equazione di questo tipo risulta quasi del tutto impossibile, al suo interno si trovano termini non lineari e termini esponenziali. Possiamo però, anche in questo caso, ricorrere a determinati algoritmi iterativi dai quali otteniamo delle buone approssimazioni.

Nel corso ci focalizzeremo quindi sullo **studio teorico e pratico** (tramite l'uso di **Matlab**) di questi algoritmi numerici. Il programma del corso prevede:

- Teoria degli errori.
- Algoritmi per la risoluzione di sistemi lineari.
- Algoritmi per l'approssimazione di integrali.
- Algoritmi per l'approssimazione di equazioni differenziali.
- Interpolazione di funzioni.
- Metodi numerici applicati agli autovalori.

Il calcolo numerico permette di simulare fenomeni complessi – come il comportamento strutturale di un ponte, la diffusione di un inquinante o la traiettoria di una navicella spaziale – senza dover effettuare esperimenti reali, spesso costosi o rischiosi. L'**analisi numerica**, intesa come lo studio dei metodi per risolvere numericamente problemi matematici, è oggi una disciplina autonoma, essenziale per chiunque lavori in ambito tecnico-scientifico. Essa si concentra, tra l'altro, sulla **stabilità numerica degli algoritmi**, sul **condizionamento dei problemi**, sulla **complessità computazionale**, e sulla **discretizzazione dei modelli continui**. In sintesi, il corso non si limiterà a fornire strumenti per “fare i conti”, ma offrirà anche **una visione critica e metodologica** dei metodi numerici, evidenziandone potenzialità e limiti, in un contesto in cui l'uso del calcolatore è imprescindibile.

Note dell'autore

Queste dispense sono frutto di diverse rielaborazioni di appunti di miei colleghi e delle lezioni tenute dal Prof. Massei nell'anno 2024/2025 e nell'anno 2025/2026. Non posso tuttavia assicurare la correttezza di tutto ciò che è presente all'interno del materiale, si consiglia pertanto di seguire il corso e di considerare questo materiale come supporto allo studio.

Licenza

Quest'opera è distribuita con licenza Creative Commons “Attribuzione – Non commerciale – Condividi allo stesso modo 3.0 Italia”.



Parte I

Teoria degli errori

CAPITOLO 1

RAPPRESENTAZIONE DEI NUMERI

Uno dei primi argomenti che affrontiamo durante il corso è quello della **teoria degli errori**. Il *calcolatore*, essendo una macchina *finita*, può rappresentare solo un numero finito di cifre. Di conseguenza, molti numeri reali – come ad esempio π oppure $\sqrt{2}$ – non possono essere rappresentati in modo esatto, ma soltanto approssimati. Lo stesso problema si presenta anche con alcuni numeri razionali, ad esempio $\frac{1}{3}$, che in base decimale ha uno sviluppo periodico infinito. Un'ulteriore difficoltà è legata alla **base di rappresentazione** utilizzata dal calcolatore. In generale, dato un intero $\beta > 1$ (la base), ogni numero reale $x \neq 0$ può essere rappresentato come:

$$x = \operatorname{sgn}(x) \cdot \beta^e \cdot \sum_{i=1}^{+\infty} \alpha_i \beta^{-i}, \quad \operatorname{sgn}(x) = \begin{cases} -1 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

dove $e \in \mathbb{Z}$ viene detto **esponente** e ogni α_i è un intero, detto **cifra della rappresentazione**, tale che $0 \leq \alpha_i \leq \beta - 1$, $\forall i = 1, \dots, +\infty$. Questo tipo di rappresentazione si chiama **rappresentazione in virgola mobile**, e permette di descrivere qualunque numero reale, in qualunque base. Ad esempio, il numero π in base 10 si può scrivere come:

$$0.1_{10} = 0.0001100110011\dots_2$$

che deve essere troncata in memoria, introducendo inevitabilmente un **errore di rappresentazione**.

Andando quindi a sfruttare questa rappresentazione, detta **rappresentazione in virgola mobile**, possiamo rappresentare un **qualsiasi numero in qualsiasi base**. Prendiamo come esempio il numero π , sfruttando la notazione per i numeri a virgola mobile abbiamo che

$$1 \cdot 10^1 \cdot (3 \cdot 10^{-1} + 1 \cdot 10^{-2} + 4 \cdot 10^{-3} + \dots) = 3.14\dots$$

Possiamo applicare lo stesso procedimento a qualunque $x \in \mathbb{R}$ trovando, sempre, almeno una rappresentazione in virgola mobile. Il problema principale dell'utilizzo di questa notazione è che, senza alcuna ipotesi aggiuntiva, la rappresentazione non è di fatto unica, il che può generare non pochi problemi, specie per quanto riguarda le operazioni aritmetiche tra queste rappresentazioni. Possiamo introdurre, **senza dimostrazione**, un teorema che ci permette di ricavare una **condizione di unicità**:

Theorem 0.1 (Teorema di Rappresentazione). Data una base intera $\beta > 1$ e un qualunque numero reale x diverso da 0, esiste un'unica rappresentazione in base β

$$x = \operatorname{sgn}(x) \cdot \beta^e \cdot \sum_{i=1}^{+\infty} \alpha_i \beta^{-i}$$

tale per cui $\alpha_1 \neq 0$ e non vi sia un intero k per cui si abbia $\alpha_j = \beta - 1$, $\forall j > k$.

Per capire l'importanza della condizione, consideriamo un controesempio. Supponiamo di scrivere:

$$x = 1 \cdot 10^2 \cdot (0 \cdot 10^{-1} + 3 \cdot 10^{-2} + 1 \cdot 10^{-3} + 4 \cdot 10^{-4} + \dots) = 03.14\dots = 3.14\dots$$

Abbiamo così ottenuto due rappresentazioni diverse dello stesso numero π , il che contraddice l'unicità. Dunque, le condizioni date nel teorema sono necessarie.

Cerchiamo ora di analizzare la struttura della rappresentazione. Fino a questo momento abbiamo introdotto solo tre elementi che la compongono: la *funzione segno*, i *coefficienti della rappresentazione* e la *base*. Introduciamo ora l'ultimo elemento: fissato $\beta > 1$, $0 \leq \alpha_i \leq \beta - 1$, $\forall i = 1, 2, \dots$ e $e \in \mathbb{Z}$, della rappresentazione normalizzata, vale che la serie

$$m_a = \sum_{i=1}^{+\infty} \alpha_i \beta^{-i}$$

è detta **mantissa** della rappresentazione x in base β . La mantissa rappresenta la "parte frazionaria normalizzata" della rappresentazione. Per essa vale la seguente proprietà fondamentale:

Lemma 0.2. Sia m_a la mantissa di una rappresentazione di un numero reale $x \neq 0$. Allora vale che:

$$\frac{1}{\beta} \leq \sum_{i=1}^{+\infty} \alpha_i \beta^{-i} < 1$$

Dimostrazione. Focalizziamoci sui membri della disuguaglianza singolarmente.

- Dimostriamo la disuguaglianza di sinistra

$$\sum_{i=1}^{+\infty} \alpha_i \beta^{-i} \geq \alpha_1 \beta^{-1} \geq \frac{1}{\beta}$$

- Dimostriamo la disuguaglianza di destra:

$$\begin{aligned} \sum_{i=1}^{+\infty} \alpha_i \beta^{-i} &< \sum_{i=1}^{+\infty} (\beta - 1) \beta^{-i} = (\beta - 1) \sum_{i=1}^{+\infty} \beta^{-i} \\ &= (\beta - 1) \left(\frac{1}{1 - \beta^{-1}} - 1 \right) = (\beta - 1) \cdot \frac{\beta^{-1}}{1 - \beta^{-1}} \\ &= (\beta - 1) \cdot \frac{1}{\beta - 1} = 1 \end{aligned}$$

□

1 Rappresentazione interna di un calcolatore

Come anche detto all'inizio del capitolo, il problema principale relativa alla rappresentazione in virgola mobile e, più in generale, alla rappresentazione dell'informazione interna a un calcolatore è la necessità di avere **un numero finito di cifre** che renda possibile memorizzare questi dati all'interno della memoria interna di un qualsiasi calcolatore.

$$x \approx \text{sgn}(x) \cdot \beta^e \cdot \sum_{i=1}^m \alpha_i \beta^{-i}$$

Possiamo quindi andare a definire un insieme vero e proprio, costituito da tutti e solo i numeri che sono **rappresentabili internamente a un calcolatore**. Questo insieme è chiamato **insieme dei numeri macchina**, ed è così formalmente definito:

Definition 1.1. (Numeri macchina) Siano β, L, m, U numeri interi tali per cui $\beta > 1$, $L \geq 1$, m e $U > 0$. Si definisce **insieme dei numeri macchina**, con **rappresentazione normalizzata** l'insieme

$$F(\beta, L, m, U) = \left\{ x \in \mathbb{R} : x = \text{sgn}(x) \cdot \beta^e \cdot \sum_{i=1}^m \alpha_i \beta^{-i}, \right. \\ \left. \alpha_1 \neq 0, 0 \leq \alpha_i \leq \beta - 1, -L \leq e \leq U \right\} \cup \{0\}$$

Dove β rappresenta la base, m il numero di cifre su cui sviluppiamo la mantissa, L e U sono invece, rispettivamente, **limite inferiore** e **limite superiore** per l'esponente.

Il problema che ci poniamo è come passare da **numeri non macchina**, quindi con una mantissa con una lunghezza arbitraria e non sempre finita, a numeri **macchina**, quindi con una mantissa di lunghezza definita. La risposta a questa domanda prende il nome di **rounding**, un procedimento con il quale possiamo passare da **numeri non macchina** a **numeri macchina** sfruttando una **funzione di rounding**. Quindi, in generale, ci viene dato un numero $x \in \mathbb{R} \notin F$ con un numero infinito di cifre al quale, sfruttando la funzione $\text{ROUND}(x)$, dobbiamo associare un corrispettivo numero macchina; formalmente possiamo definire la funzione di rounding come:

$$\text{ROUND}(x) : \mathbb{R} \rightarrow F$$

In generale, abbiamo diverse possibili **funzioni di rounding**, tuttavia le più utilizzate, o comunque, le più famose sono:

- **Troncamento:** questa funzione arrotonda x al numero macchina precedente

$$\text{ROUND}(x) = T_R(x) = \lfloor x \rfloor = \text{sgn}(x) \cdot \beta^e \cdot \left(\sum_{i=1}^m \alpha_i \beta^{-i} \right)$$

- **Arrotondamento per eccesso:** questa funzione approssima x al numero macchina successivo

$$\text{ROUND}(x) = T_U(x) = \lceil x \rceil = \text{sgn}(x) \cdot \beta^e \cdot \left(\sum_{i=1}^m \alpha_i \beta^{-i} + \beta^{-m} \right)$$

- **Round to Nearest:** Questa funzione guarda la prima cifra del numero e in base ad essa decide se arrotondare per eccesso o se troncare

$$\text{ROUND}(x) = \text{RTN}(x) = \begin{cases} \lfloor x \rfloor & \alpha_{m+1} \geq \frac{\beta}{2} \\ \lceil x \rceil & \alpha_{m+1} < \frac{\beta}{2} \end{cases}$$

Nelle rappresentazioni con numeri macchina troviamo anche due casi limite: **underflow** e **overflow**. Il primo caso si verifica quando il valore assoluto del numero che vogliamo arrotondare risulta maggiore di U , mentre il secondo caso si verifica quando il valore assoluto del numero che vogliamo arrotondare risulta minore di L . In generale, possiamo risolvere questi casi limite nel seguente modo

- Nel caso di **overflow** ($|x| > \beta^U (1 - \beta^{-m})$) possiamo approssimare x al massimo numero rappresentabile

$$\beta^U (\beta - 1) \cdot \left(\sum_{i=1}^m \beta^{-i} \right) = \beta^U (\beta - 1) \left(\frac{1 - \beta^{-m-1}}{1 - \beta} - 1 \right) = \\ \beta^U (\beta - 1) \left(\frac{1 - \beta^{-m}}{\beta - 1} \right) = \boxed{\beta^U (1 - \beta^{-m})}$$

- Nel caso di **underflow** ($|x| < \beta^{L-1}$) possiamo approssimare x al minimo numero rappresentabile

$$\boxed{\beta^L \beta^{-1} = \beta^{L-1}}$$

Uno dei problemi principali dell'arrotondamento è l'errore che viene generato nell'operazione. In particolare, possiamo definire delle relazioni algebriche che legano le varie funzioni di **rounding** all'errore assoluto che generano:

$$|U_p(x) - x| = |T_r(x) - x| \leq \beta^{e-m}$$

$$|R_n(x) - x| \leq \frac{\beta^{e-m}}{2} \wedge |x - R_n(x)| = \min_{z \in F} |z - x|$$

Osserviamo quindi che le funzioni **arrotondamento per difetto** e **arrotondamento per eccesso** hanno l'errore assoluto massimo più alto, mentre la funzione di **round-to-nearest**, nonostante combini le due funzioni di rounding, ha un errore che è la metà. Le due condizioni sulla funzione di **round-to-nearest** permettono quindi di imporre due condizioni:

- La prima condizione afferma che l'errore che il sistema commette nell'arrotondare un numero non possa superare una certa soglia, pari al valore presente nell'espressione.
- La seconda condizione afferma che tra tutti i numeri che il computer può generare venga scelto quello più vicino in assoluto a x .

Esiste uno standard, noto come **IEEE standard** (o *Institute of Electrical and Electronics Engineers standard*), che definisce alcune rappresentazioni standard per i **numeri in virgola mobile**. Prenderemo come esempio le due rappresentazioni più utilizzate per questa tipologia di numeri

	β	m	l	U	Memoria	e
Precisione singola	2	24	-125	128	4 Bytes	8
Precisione doppia	2	53	-1021	1024	8 Bytes	11

Tabella 1.1: Rappresentazione in virgola mobile. La base è 2 per entrambe le rappresentazioni, mentre invece l'esponente cambia, nella prima rappresentazione abbiamo infatti 24 bit (circa 7 cifre decimali), mentre nella seconda rappresentazione abbiamo 53 bit, cioè circa 15/16 cifre decimali

Insieme all'errore assoluto di arrotondamento, è importante introdurre un'ulteriore grandezza: l'**errore relativo dell'arrotondamento**, definito come

$$\frac{x - R_N(x)}{x} = \varepsilon_x \implies |\varepsilon_x| \leq \frac{1}{2} \beta^{1-m} = \frac{1}{2} u$$

Uno dei concetti principali all'interno dell'analisi numerica è quello di precisione macchina. Possiamo definire - in termini semplici - la **precisione macchina** come *il limite oltre il quale il calcolatore non è più in grado di distinguere due numeri molto vicini tra loro*. In termini puramente generali, preso un numero positivo $x > 0$, possiamo sicuramente dire che $1 + x > x$; tuttavia, all'interno di un calcolatore la memoria è fondamentalmente **finita** e, quindi, i numeri **decimali** vengono arrotondati. In questo contesto, la **precisione macchina** rappresenta il più piccolo numero positivo per cui il computer è in grado di riconoscere che:

$$1 + \varepsilon > 1$$

Attraverso la **precisione macchina** siamo anche in grado di definire un'altra relazione fondamentale. Ipotizziamo infatti di prendere un certo numero x e di applicarci una funzione di arrotondamento come Floor; sappiamo che questa funzione arrotonda il nostro numero all'intero successivo, restituendo in uscita:

$$\text{Floor}(x) = x \cdot (1 + \delta)$$

Dove δ è proprio l'errore introdotto, che sappiamo essere minore o uguale proprio della precisione macchina. Il valore assunto dalla precisione macchina, talvolta indicato anche come u , è diverso tra le diverse rappresentazioni floating point:

Precisione singola	$U \approx 10^{-8}$
Precisione doppia	$U \approx 10^{-16}$

2 Numeri sotto-normalizzati

Fino ad ora abbiamo studiato tutto ciò che riguarda il limite può distinguere vicino ad 1. Tuttavia, dobbiamo cercare di capire cosa succede quando andiamo a considerare il **limite estremo** vicino allo **zero**. Normalmente, i numeri in virgola mobili vengono salvati sempre assumendo che il numero inizi con un uno davanti alla virgola, nello specifico:

$$(-1)^{\text{segno}} \cdot 1.\text{mantissa} \cdot 2^{\text{esponente}}$$

Il più piccolo numero rappresentabile con questa notazione corrisponde al numero con mantissa 1 e esponente pari al minimo esponente possibile (che dipende dalla precisione della rappresentazione scelta); tutto ciò che si trova tra questo numero **piccolissimo** e lo zero viene automaticamente arrotondato come zero, che nella pratica è un problema **enorme**. La risoluzione a questo problema è quella di introdurre un'eccezione alla regola che abbiamo appena introdotta: i **numeri sotto-normalizzati**. L'idea è che al momento in cui l'esponente scende sotto al suo valore minimo assoluto, il calcolatore smette di usare l'uno implicito per rappresentare il numero, ma usa uno zero implicito. Un numero sotto-normalizzato è quindi un numero scritto nella forma:

$$(-1)^{\text{segno}} \cdot 0.\text{mantissa} \cdot 2^{\text{esponente}}$$

L'introduzione di questa classe di numeri permette quindi di eliminare il problema del **Flush-To-Zero**, che potrebbe portare a delle problematiche nell'esecuzione di diversi algoritmi, specie quelli dove si lavora con precisioni e tolleranze molto piccoli. Siccome all'interno della classe dei numeri sotto-normalizzati ($x \in [0, \beta^{L-1}]$) si ha un esponente fisso e lo zero implicito, allora l'unico modo in cui è possibile formare numeri diversi è quello di variare la **mantissa**. I numeri ottenuti sono quindi tutti distanti esattamente allo stesso modo l'uno dall'altro.

3 Operazioni macchina

Le operazioni aritmetiche di base sono delle operazioni che, presi due numeri x, y in $\mathbb{R}$ restituiscono, eccetto casi particolari, numeri $z \in \mathbb{R}$. Però, non possiamo garantire con certezza che presi due numeri in rappresentazione a virgola mobile con la mantissa su m cifre, il risultato sia anch'esso sullo stesso numero di cifre della mantissa

$$\text{Operazioni} : x, y \in F(m, \beta, L, U) \rightarrow z \in F(m, \beta, L, U)$$

Dobbiamo quindi introdurre una nuova classe di operazioni aritmetiche, dette **operazioni macchina** e contrassegnate nel seguente modo

$$\text{Operazioni macchina} = \{\ominus, \oplus, \otimes, \odot\}$$

Questo insieme di operazioni ha la particolarità di, preso due numeri su m cifre di mantissa, restituire un risultato anch'esso su m cifre di mantissa; rispettando in questo modo la rappresentabilità del numero. Queste operazioni non sono altro che degli arrotondamenti delle corrispettive operazioni aritmetiche esatte. Presa una qualunque operazione $\odot \in \text{Operazioni macchina}$

$$x \odot y = \text{Rn}(x \cdot y)$$

Il problema dell'utilizzo di questa classe di operazioni è che, in alcuni casi, si rendono false le proprietà delle operazioni aritmetiche esatte; in generale

$$\begin{aligned} x \oplus (y \oplus w) &\neq (x \oplus y) \oplus w \\ x \otimes (y \otimes w) &\neq (x \otimes y) \otimes w \\ x \otimes (y \oplus w) &\neq (x \otimes y) + x \otimes w \end{aligned}$$

CAPITOLO 2

APPROSSIMAZIONI DI FUNZIONI

Le funzioni effettivamente calcolabili attraverso un calcolatore sono solo le **funzioni razionali**, il cui valore è ottenibile mediante l'utilizzo di un **numero finito di operazioni aritmetiche**. Tutte le funzioni non razionali, come ad esempio $\sin(x)$, $\cos(x)$ o e^x possono essere approssimate mediante un numero finito di operazioni aritmetiche. Nel calcolo di una generica funzione $f : \mathbb{R}^n \rightarrow \mathbb{R}$, il valore effettivamente calcolato in un corrispondenza di un vettore $x_0 \in \mathbb{R}^n$ può essere affetto da errori indotti da diversi fenomeni

- Errore generato dalla rappresentazione del vettore x_0 come **numero macchina**; in altre parole, l'errore generatore dal suo troncamento (**errore inerente**).
- Errore generato dal fatto che le operazioni sono effettuate in **aritmetica finita** (**errore logaritmico**).
- Errore generato, se la funzione $f(x)$ non è già **razionale**, dalla razionalizzazione della funzione $f(x)$

Definiamo come $\bar{x}$ il numero macchina corrispondente al troncamento alla m -esima cifra di x_0 e, definiamo come $\tilde{f}(x)$ la funzione **razionalizzata** (quindi quella effettivamente calcolata). Un esempio di linearizzazione corrisponde allo **sviluppo in serie di Taylor** fino al secondo ordine; che nel caso della funzione $f(x) = e^x$ coincide con

$$f(x) = e^x \implies \tilde{f}(x) = 1 + x + \frac{x^2}{2!}$$

mentre il troncamento, che deve essere **eventualmente** applicato sul punto p_0 , può essere fatto sfruttando la funzione $\tilde{x}_0 = \text{Rn}(x_0)$. Introduciamo ora una definizione

Definition 0.1. (Errore assoluto totale) Dati $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tilde{f}(x) \approx f(x)$ e $\tilde{x}_0 = \text{Rn}(x_0)$, si definisce **errore assoluto totale** la differenza tra la funzione razionalizzata calcolata nel punto $\tilde{x}_0$ e la funzione $f(x)$ calcolata nel punto x_0

$$\delta_f = \tilde{f}(\tilde{x}_0) - f(x_0)$$

Per il momento supponiamo che le funzioni siano tutte **razionali**, ovvero, che siano tutte scrivibili tramite operazioni **aritmetiche di base**:

$$\text{Operazioni} = \{+, \cdot, -, \div\}$$

L'idea che proviamo a sviluppare è la seguente: dividiamo l'errore totale in due componenti, una componente relativa all'errore generato dalla razionalizzazione della funzione e una componente relativa all'approssimazione del punto $\tilde{x}_0$:

$$\delta_f = \tilde{f}(\tilde{x}) - f(x_0) = \tilde{f}(\tilde{x}) - f(x_0) - f(\tilde{x}) + f(\tilde{x}) = \tilde{f}(\tilde{x}) - f(\tilde{x}) + f(\tilde{x}) - f(x_0)$$

abbiamo quindi ottenuto un'equazione composta da due membri: la prima parte ($\tilde{f}(\tilde{x}) - f(\tilde{x})$) è detto **errore logaritmico** e viene indicato con δ_a ; mentre la seconda parte ($f(\tilde{x}) - f(x_0)$) viene detto **errore inerente** e viene indicato con δ_s . Introduciamo adesso un'ulteriore definizione

Definition 0.2. (Errore relativo totale) Dati $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tilde{f}(x) \approx f(x)$ e $\tilde{x}_0 = \text{Rn}(x_0)$, si definisce **relativo** la differenza tra la funzione razionalizzata calcolata nel punto $\tilde{x}_0$ e la funzione $f(x)$ calcolata nel punto x_0 , il tutto diviso per la funzione $f(x)$ calcolata nel punto x_0

$$\varepsilon_e = \frac{\delta_f}{f(x_0)} = \frac{\delta_a + \delta_d}{f(x_0)} = \frac{\delta_a}{f(x_0)} + \frac{\delta_d}{f(x_0)}$$

Come nel caso precedente abbiamo diviso l'errore relativo in due componenti, dove $\delta_d/f(x_0)$ viene detto **errore inerente relativo**. Tuttavia, non possiamo invece dire che il primo membro, $\delta_a/f(x_0)$, corrisponde all'errore algoritmico relativo. Dobbiamo fare dei passaggi aggiuntivi per arrivare alla formulazione finale di tale errore:

$$\begin{aligned} \frac{\delta_a}{f(x_0)} &= \frac{\delta_a}{f(x_0)} \cdot \frac{f(\tilde{x})}{f(\tilde{x})} = \frac{\delta_a}{f(\tilde{x})} \cdot \frac{f(\tilde{x})}{f(x_0)} = \frac{\tilde{f}(\tilde{x}) - f(\tilde{x})}{f(\tilde{x})} \cdot \frac{f(\tilde{x})}{f(x_0)} = \\ &= \frac{\tilde{f}(\tilde{x}) - f(\tilde{x})}{f(\tilde{x})} \cdot \left[\frac{f(\tilde{x})}{f(x_0)} \right] = \frac{\tilde{f}(\tilde{x}) - f(\tilde{x})}{f(\tilde{x})} \cdot \left[\frac{f(\tilde{x}) - f(x_0) + f(x_0)}{f(x_0)} \right] = \\ \varepsilon_a \cdot \left[1 + \frac{f(\tilde{x}) - f(x_0)}{f(x_0)} \right] &= \varepsilon_a(1 + \varepsilon_d) \end{aligned}$$

Quindi, possiamo riscrivere l'**errore relativo totale** come

$$\varepsilon_f = \varepsilon_a(1 + \varepsilon_d) + \varepsilon_s = \varepsilon_a + \varepsilon_d + \varepsilon_a\varepsilon_d$$

Possiamo semplificare l'espressione andando a svolgere una approssimazione della formula rimuovendo il corrispettivo termine $\varepsilon_a\varepsilon_d$. La giustificazione dell'approssimazione è data dal fatto che $\varepsilon_a\varepsilon_d \ll \min(\varepsilon_a, \varepsilon_d)$ e quindi essendo molto piccolo e molto vicino allo zero possiamo eliminarlo senza rendere l'approssimazione sconsigliata.

Uno degli obiettivi che di dobbiamo porre è quello di trovare dei vincoli che permettano di limitare il modulo dell'errore assoluto totale, poiché un valore di errore troppo alto potrebbe portarci ad ottenere risultati sbagliati. Dobbiamo quindi imporre delle disequazioni:

$$|\delta_a| < \tau_1, \quad |\delta_d| < \tau_2 \implies |\delta_f| < \tau_1 + \tau_2$$

1 Errore inerente

Riprendiamo la formula dell'errore assoluto inerente, che ricordiamo essere l'errore relativo all'approssimazione del punto x_0 in $\tilde{x}_0$, dove $\tilde{x}_0 = \text{Rn}(x_0)$, corrispondente a $\delta_d = f(\tilde{x}_0) - f(x_0)$. Supponiamo adesso che la funzione f sia continua e derivabile in un dominio $D \subseteq \mathbb{R}^n$, così da rendere possibile fare lo sviluppo di **taylor** (al secondo ordine) della funzione $f(\tilde{x}_0)$ nel punto x_0 : $f(\tilde{x}_0) \approx f(x_0) + \nabla f(x_0)^T(x_1 - x_0)$. Sostituendo nell'espressione dell'errore relativo inerente risulta che:

$$\cancel{f(x_0)} + \nabla f(x_0)^T(x_1 - x_0) - \cancel{f(x_0)} = \nabla f(x_0)^T(x_1 - x_0) = \sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0)(x_1^{(i)} - x_0^{(i)}) =$$

$$\sum_{i=1}^m A_i \cdot \delta_i, \quad \delta_i = x_1^{(i)} - x_0^{(i)}, \quad A_i = \frac{\partial f}{\partial x_i}(x_0)$$

dove δ_i rappresenta l' i -esima componente dell'errore inerente e il coefficiente che lo moltiplica viene detto **coefficiente di amplificazione**. L'espressione che abbiamo ottenuto ci permette di ottenere una formula dell'errore dipendente da coefficienti specifici, facilmente calcolabili, che amplificano l'errore inerente componente per componente. Possiamo anche definire un limite superiore per questi moltiplicatori

$$|\delta_d| \leq \sum_{i=1}^m A_i \cdot |\delta_i|, \quad A_i = \max_{p \in D} \left| \frac{\partial f}{\partial x_i}(p) \right|$$

Possiamo ottenere una formula analoga anche per quanto riguarda l'errore relativo inerente.

$$\begin{aligned}\varepsilon_d &= \frac{\sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0) \cdot (x_1^{(i)} - x_0^{(i)})}{f(x_0)} = \sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{(x_1^{(i)} - x_0^{(i)})}{f(x_0)} = \\ &= \sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{(x_1^{(i)} - x_0^{(i)})}{f(x_0)} \cdot \frac{x_0^{(i)}}{x_0^{(i)}} = \sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{(x_1^{(i)} - x_0^{(i)})}{x_0^{(i)}} \cdot \frac{x_0^{(i)}}{f(x_0)} = \\ &= \sum_{i=1}^m \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{(x_1^{(i)} - x_0^{(i)})}{x_0^{(i)}} \cdot \frac{x_0^{(i)}}{f(x_0)} = \sum_{i=1}^m \frac{(x_1^{(i)} - x_0^{(i)})}{x_0^{(i)}} \cdot \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{x_0^{(i)}}{f(x_0)} = \\ &= \left[\sum_{i=1}^m \varepsilon_i \rho_i \right], \quad \varepsilon_i = \frac{(x_1^{(i)} - x_0^{(i)})}{x_0^{(i)}}, \quad \rho_i = \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{x_0^{(i)}}{f(x_0)}\end{aligned}$$

Possiamo anche definire un limite superiore per i coefficienti di amplificazione

$$|\varepsilon_d| \leq \sum_{i=1}^m \rho_i \cdot |\varepsilon_i|, \quad \rho_i = \max_{p \in D} \left| \frac{\partial f}{\partial x_i}(x_0) \cdot \frac{x_0^{(i)}}{f(x_0)} \right|$$

Se risulta $|\rho_i| \rightarrow 1$, $\forall \rho = 1, \dots, m$ allora diciamo **bencondizionato**, in quanto ε_s è nell'ordine di grandezza del minimo tra i $|\varepsilon_i|$. Tutti i casi in cui i valori di $|\rho|$ sono molto grandi sono detti **malcondizionati**.

Tutte le operazioni macchina che abbiamo introdotto nella sezione precedente generano degli errori inerenti relativi al troncamento del risultato:

OP	δ_s	ε_s
$x + y$	$\delta_x + \delta_y$	$\frac{x}{x+y}\varepsilon_x + \frac{y}{x+y}\varepsilon_y$
$x - y$	$\delta_x - \delta_y$	$\frac{x}{x-y}\varepsilon_x - \frac{y}{x-y}\varepsilon_y$
$x \cdot y$	$y\delta_x + x\delta_y$	$\varepsilon_x + \varepsilon_y$
x/y	$\frac{\delta_x}{y} - \frac{x}{y^2}\delta_y$	$\varepsilon_x - \varepsilon_y$

Tabella 2.1: Errori assoluti e relativi nelle operazioni aritmetiche di macchina

2 Errore algoritmico

Abbiamo, all'inizio del capitolo, parlato dell'errore inerente, quindi l'errore dovuto al troncamento del numero nel passaggio a numero macchina con mantissa su m cifre. Andiamo ad analizzare adesso l'errore algoritmico. Dalle definizioni sappiamo che

$$\delta_a = \tilde{f}(\tilde{x}) - f(\tilde{x}), \quad \varepsilon_a = \frac{\tilde{f}(\tilde{x}) - f(\tilde{x})}{f(\tilde{x})}$$

Supponiamo, per adesso, che l'errore inerente sia nullo. Per calcolare questo errore è sufficiente seguire l'errore dell'algoritmo passo per passo, man mano che si va avanti nella sua esecuzione. Supponiamo di voler calcolare l'errore algoritmico della seguente funzione

$$f(x, y, z, w) = x \left(\frac{y}{z} - w \right)$$

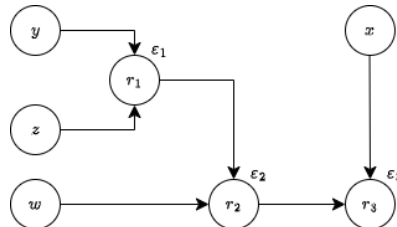
Seguendo il ragionamento che abbiamo fatto, possiamo scrivere la nostra corrispettiva funzione $\tilde{f}(x)$

$$f(x, y, z, w) = x \otimes \left(y \oplus z \ominus w \right)$$

che può essere scritta sotto forma di algoritmo, passo per passo, nel seguente modo:

$$\begin{aligned} r_1 &= y \oplus z \\ r_2 &= r_1 \ominus w \\ r_3 &= x \otimes r_2 \end{aligned}$$

Per semplificare il calcolo dell'errore algoritmico è possibile anche sfruttare una struttura a grafo per la rappresentazione delle diverse fasi del calcolo



3 Problemi dell'approssimazione numerica

Esistono due problemi tipici del mondo dell'approssimazione numerica che affronteremo durante questo corso: **problema diretto** e **problema indiretto**. Il primo problema, cioè quello diretto, si articola nel seguente modo: data $f(x)$ funzione **razionale** e $f_a(x)$ algoritmo, data inoltre una stima degli errori δ_{r_i} , stimare δ_f per $x_0 \in D \subseteq \mathbb{R}^n$. Il secondo problema, cioè quello indiretto, si articola in modo opposto: data $f(x)$ funzione **razionale** e dato un limite superiore per l'errore algoritmico totale τ , determinare i parametri della rappresentazione e/o l'algoritmo tale per cui $|\delta_f| \leq \tau$.

3.1 Problema diretto

Prendiamo un esempio numerico, data $f(x_1, x_2) = x_1/x_2$, $\tilde{f}(x) = x_1 \oplus x_2$ e $D = [1, 3] \times [4, 3]$ si supponga di voler stimare δ_f per $x_0 \in D$. Il primo passo è quello di calcolare l'errore inerente sulla singola componente, cioè quelli derivati dall'arrotondamento di x_1 e x_2 . Sappiamo che $x_j \in [1, 10]$, quindi abbiamo esponente pari ad 1; inoltre, sappiamo anche che la mantissa ha $m = 3$ cifre e, quindi, ipotizzando di aver scelto $R_n(x)$ come funzione di arrotondamento si ottiene

$$|\delta_i| = \frac{1}{2}\beta^{e-m} = \frac{1}{2}10^{-2}$$

Applicando la formula esplicita possiamo calcolare l'errore inerente

$$\delta_d = \frac{\partial f}{\partial x_1}(p_0)\delta_{x_1} + \frac{\partial f}{\partial x_2}(p_0)\delta_{x_2} = \delta_x \left[\frac{\partial f}{\partial x_1}(p_0) + \frac{\partial f}{\partial x_2}(p_0) \right], \quad p_0 \in [1, 3] \times [4, 3] = D$$

A questo punto possiamo calcolare i due termini derivati e fare le opportune maggiorazioni, ottenendo infine:

$$|\delta_d| \leq \frac{1}{2}10^{-2} \left[\max_D \left| \frac{\partial f}{\partial x_1}(p_0) \right| + \max_D \left| \frac{\partial f}{\partial x_2}(p_0) \right| \right] = \frac{1}{2}10^{-2} \left[\frac{1}{4} + \frac{3}{16} \right]$$

Una volta calcolato l'errore inerente totale è il turno del calcolo dell'errore algoritmico, che possiamo stimare utilizzando la tecnica del grafo. Osserviamo infatti che c'è solamente un'operazione da fare e, quindi, l'errore sarebbe:

$$\delta_a = \delta_1 + \frac{1}{x_2}\delta_{x_1} + \frac{x_1}{x_2^2}\delta_{x_2}$$

Siccome partiamo dall'assunto che l'errore inerente sulle due cifre sia nullo (ovviamente non è nullo, è solo un'assunzione che facciamo per semplificare i calcoli, d'altronde l'errore inerente l'abbiamo

già calcolato) e quindi possiamo rimuovere i termini relativi a δ_{x_i} . Risulta quindi che l'unico errore è solo quello relativo all'approssimazione della divisione, che anche in questo caso viene fatta con $R_n(x)$. Per calcolare l'esponente è sufficiente notare che l'insieme delle soluzioni possibili della frazione, relativamente all'insieme D , è costituito dai soli valori $[0.1, 1)$ e, quindi, $e = 0$. Possiamo quindi concludere che

$$|\delta_a| \leq \frac{1}{2}10^{-3}$$

A questo punto è possibile sommare le due maggiorazioni ottenute per ricavare il valore dell'errore assoluto:

$$|\delta_f| \leq |\delta_a| + |\delta_d| \leq |\delta_a| + \frac{1}{2}10^{-2} \left(\frac{1}{4} + \frac{3}{16} \right) \leq \max_D \left| \frac{x_1}{x_2} \right| \frac{1}{2}10^{-2} + \frac{1}{2}10^{-2} \left(\frac{1}{4} + \frac{3}{16} \right) = \frac{23}{32}10^{-2}$$

3.2 Problema indiretto

Prendiamo come esempio la funzione $f(x) = x_1/x_2$, $D = [1, 2] \times [-2, -1] := D$, l'insieme dei numeri macchina $F(10, m, -99, +99)$ e la funzione $Rn(x)$ per arrotondare. Determinare un **algoritmo** e un numero di cifre m per la mantissa tali per cui $|\delta_f| < 10^{-2}$. Prendiamo come algoritmo $f_a(x) = x_1 \oplus x_2$, che da come errore algoritmico $|\delta_a|$ pari al solo errore di arrotondamento della divisione, che nel caso del **round to nearest**, è pari a $\beta^{e-m}/2$. In questo caso e identifica l'esponente nella rappresentazione floating point della divisione; distinguiamo quindi i due casi possibili:

- Nel caso peggiore il rapporto tra x_1 e x_2 è compreso tra $[1, 10)$, quindi $e = 1$.
- Nel caso migliore il rapporto tra x_1 e x_2 è compreso tra $[0.1, 1]$, quindi $e = 0$.

Ovviamente per stimare correttamente l'errore prendiamo l'esponente relativo al caso peggiore, cioè il caso in cui l'esponente vale 1; quindi l'errore algoritmico sarà pari a:

$$|\delta_a| \leq \frac{1}{2}\beta^{1-m}$$

Per trovare l'errore inerente dobbiamo ricavare l'errore inerente. Il primo passo è ricavare l'errore inerente sulle singole componenti e siccome x_1 e x_2 sono compresi, in modulo, nell'intervallo $[1, 2]$, allora il loro esponente è pari ad 1; di conseguenza, l'errore inerente per componente avrà la stessa limitazione dell'errore algoritmico. L'errore inerente è quindi dato da

$$|\delta_d| \leq \frac{1}{2}10^{1-m} \left(\max_{x \in D} \left| \frac{1}{x_1} \right| + \max_{x \in D} \left| \frac{x_1}{x_2^2} \right| \right) = \frac{3}{2}10^{1-m}$$

Possiamo quindi scrivere la nostra disuguaglianza finale per l'errore totale

$$|\delta_f| \leq \frac{3}{2}10^{1-m} + \frac{1}{2}10^{1-m} \leq 10^{-2}$$

Attraverso delle prove è facilmente dimostrabile che per $m = 4$ la disuguaglianza è sicuramente rispettata.

4 Approssimazioni di funzioni non razionali

Fino ad ora abbiamo supposto di approssimare soltanto funzioni razionali, cioè funzioni $f(x)$ tali per cui f è un polinomio o un rapporto di polinomi nella forma

$$f(x) = \frac{Q_n(x)}{R_m(x)}$$

Tuttavia la stragrande maggioranza delle funzioni con cui ci si trova a lavorare nella realtà sono ben diverse; in generale, infatti, la maggior parte delle funzioni con cui si lavora sono **non razionali**, cioè composte da **almeno** un termine non razionale. Ad esempio, la funzione

$$f(x) = e^{\tan(x)} \sinh(x)$$

è un esempio di funzione irrazionale, poiché contiene tre diversi termini non razionali. L'idea alla base dell'approssimazione di queste funzioni è quella di aggiungere un ulteriore passo prima della definizione dell'algoritmo; in altri termini, aggiungiamo uno step intermedio che permetta di ottenere i seguenti passaggi:

$$\text{Funzione irrazionale} \xrightarrow{\text{Razionalizzazione}} \text{Funzione razionale} \rightarrow \text{Algoritmo}$$

L'introduzione di una fase di **razionalizzazione** porta però un problema; infatti razionalizzando una funzione si esegue una vera e propria approssimazione, convertendo i termini irrazionali in termini razionali, generando quindi un **errore**. Questo **errore** viene detto **errore analitico** e viene definito nel seguente modo

Definition 4.1. (Errore analitico) Data una funzione $f(x) : \mathbb{R} \rightarrow \mathbb{R}$ **non razionale** e data la funzione $\tilde{f}(x) : \mathbb{R} \rightarrow \mathbb{R}$ **razionale** tale per cui

$$\tilde{f}(x) = \text{Razionalizzazione}(f(x))$$

Si definisce come errore **analitico** assoluto e relativo, rispettivamente, la differenza tra la funzione $f(x)$ e la funzione $\tilde{f}(x)$ nel punto x_0 e la differenza tra la funzione $f(x)$ e la funzione $\tilde{f}(x)$ nel punto x_0 diviso $f(x)$ valutato in x_0 :

$$\delta_{\text{AN}} = f(x_0) - \tilde{f}(x_0) \quad \varepsilon_{\text{AN}} = \frac{f(x_0) - \tilde{f}(x_0)}{f(x_0)}$$

Introducendo l'errore analitico è necessario prestare attenzione un paio di accortezza; la prima accortezza è quella di calcolare l'errore inerente sulla funzione originale, in quanto vi è una dipendenza rispetto alle derivate, mentre, la seconda accortezza è quella di calcolare l'errore algoritmico sull'algoritmo relativo alla funzione razionalizzata $\tilde{f}_a(x)$. La **stima** dell'**errore analitico** può essere facilmente fatta sfruttando un metodo di approssimazione già visto durante il corso di analisi I, cioè lo sviluppo di Taylor. In particolare, supponiamo di avere una funzione $f(x) : \mathbb{R} \rightarrow \mathbb{R}$ con $f \in C^{k+1}(\mathbb{R})$, un punto $x_0 \in \mathbb{R}$, allora possiamo definire lo sviluppo di Taylor con resto di Peano come

$$f(x) = f(x_0) + f^{(1)}(x_0)(x - x_0) + \frac{f^{(2)}(x_0)}{2!}(x - x_0)^2 + \dots + \frac{f^{(k+1)}(x_0)}{(k+1)}(x - x_0)^{k+1}$$

In particolare, se portiamo a sinistra il termine relativo all'espansione di Taylor per un grado k -esimo, che denotiamo con $T_k(x)$, si ottiene:

$$f(x) - T_k(x) = \frac{f^{(k+1)}(x_0)}{(k+1)}(x - x_0)^{k+1}$$

Quindi, in modo molto facile è possibile ottenere una stima dell'errore analitico derivato dall'arrotondamento di una funzione con sviluppo di Taylor.

Parte II

Metodi numerici per sistemi lineari

CAPITOLO 3

AUTOVALORI E AUTOVETTORI

Un elemento fondamentale dello studio delle matrici riguarda lo studio degli autovalori e degli autovettori di una matrice. L'importanza di questi due costrutti matematici è data dall'utilità che essi trovano in alcuni algoritmi di approssimazione: ad esempio, un algoritmo che sfruttava un **problema agli autovalori** era l'algoritmo utilizzato da google per il ranking delle pagine durante la ricerca. Definiamo adesso il significato di autovalore:

Definition 0.1. (Autovalore) Data una matrice $A \in \mathbb{C}^{n \times n}$ si dice **autovalore** di A ogni numero $\lambda \in \mathbb{C}$ tale per cui, il sistema lineare

$$Av = \lambda v, \quad v \in \mathbb{C}$$

ammette soluzioni $v \neq 0$. Inoltre, ogni v è detto **autovettore destro** relativo all'autovalore λ . Inoltre, si definisce come **autovettore sinistro** di λ il vettore $w \in \mathbb{C}$ tale per cui

$$v^H A = \lambda w^H$$

Il problema che ci dobbiamo porre adesso è capire come dimostrare l'esistenza degli autovalori e dei rispettivi autovettori. In linea puramente teorica infatti, non sappiamo se essi esistano per ogni matrice. Possiamo tuttavia dimostrarlo facilmente nel seguente modo; se (λ, x) è un autocoppia per la matrice A , allora vale che $\lambda x = Ax$, quindi che

$$(A - \lambda I)x = 0$$

Per definizione stessa di autovettore, x deve essere diverso dall'autovettore di modulo zero e, pertanto, il sistema deve ammettere delle soluzioni che siano non banali; in termini puramente algebrici deve quindi valere che:

$$\det(A - \lambda I) = 0$$

Se scriviamo questa espressione in forma algebrica, notiamo immediatamente che corrisponde ad un particolare polinomio in λ , chiamato **polinomio caratteristico**

$$p(\lambda) = (-1)^n \lambda^n + c_{n-1} \lambda^{n-1} + \dots + c_0$$

Dal teorema fondamentale dell'algebra, sappiamo che un polinomio complesso, ammette sempre n soluzioni complesse contante con molteplicità. Quindi, possiamo concludere che nel campo dei numeri complessi, l'esistenza degli autovalori è sempre garantita (stessa cosa però non si può dire per matrici a coefficienti reali). Dimostriamo ora l'esistenza degli autovettori; il teorema della dimensione afferma che, data una qualsiasi matrice quadrata M di ordine n , la somma tra la dimensione del suo nucleo e il suo rango è esattamente pari ad n

$$\dim(\ker(A - \lambda I)) + \text{Rk}(A - \lambda I) = n$$

Siccome però, il determinante della matrice è sicuramente minore di n , allora la dimensione del nucleo della matrice è sicuramente maggiore o uguale a 1. Osservando però la definizione di nucleo, ci si rende immediatamente conto che per la matrice $A - \lambda_1 I$, il nucleo corrisponde proprio a quei vettori x tali per cui

$$(A - \lambda_1 I)x = 0$$

Che corrisponde proprio alla definizione di autovettore che abbiamo dato. Possiamo quindi concludere che l'insieme degli autovettori associati ad un singolo autovalore formano un **autospazio** di dimensione proprio corrispondente a quella del nucleo della matrice. La dimensione dell'autospazio originato da un autovalore prende il nome di **molteplicità geometrica**. Ogni autovalore è però caratterizzato anche da un altro numero, detto **molteplicità algebrica**; in particolare:

Definition 0.2. Si definiscono **molteplicità algebrica** e **molteplicità geometrica**, rispettivamente, come:

- Si definisce come **molteplicità algebrica** di un autovalore λ il numero di volte in cui λ compare come radice del **polinomio caratteristico**.
- Si definisce come **molteplicità geometrica** di un autovalore λ la dimensione dello spazio delle soluzioni relativo a λ .

Una proprietà molto interessante degli autovalori afferma che, presa una generica matrice $A \in \mathbb{C}^{n \times n}$, gli autovalori di A e gli autovalori di A^T coincidono. Inoltre, ad ogni matrice per cui esistono autovalori, è possibile associare una definizione che risulta molto utile nei diversi metodi che vedremo:

Definition 0.3. Si definisce come **raggio spettrale** il massimo valore, in modulo, tra tutti gli autovalori della matrice

$$\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|$$

Una matrice tale per cui il suo raggio spettrale della matrice è minore di 1 è detta **matrice convergente**. Fino ad ora abbiamo solo detto come sia possibile, conoscendo un autovalore, determinare un autovettore della matrice; tuttavia, esiste una semplice formula che permette anche di implementare l'operazione inversa, cioè determinare l'autovalore della matrice, partendo proprio dall'autovettore.

Definition 0.4. (Quoziente di Rayleigh) Sia $A \in \mathbb{C}^{n \times n}$ una matrice a coefficienti complessi (vale anche nel caso reale), e ipotizziamo che (λ, x) sia una generica **autocoppia** per A ; si definisce come **quoziente di rayleigh** il seguente rapporto:

$$\lambda = \frac{x^T A x}{x^H x}$$

1 Matrici simili

Un fatto che è interessante da analizzare è il caso di **matrici simili**. Data una matrice $A \in \mathbb{C}^{n \times n}$ ed una matrice S non singolare, si dice **trasformata per similitudine** della matrice A , la matrice B tale che

$$B = S^{-1} A S$$

e B si dice **matrice simile di A** . Un risultato molto importante che riguarda proprio gli autovalori di matrici simili asserisce che

Theorem 1.1. Due matrici simili A e B hanno gli stessi autovalori. Inoltre, per ogni autovalore λ_i , se x è **autovettore** di A , allora $S^{-1}x$ è autovettore di B .

Dimostrazione. Partiamo dal dimostrare che due matrici simili hanno medesimi autovettori

$$\begin{aligned} \det(B - \lambda I) &= \det(S^{-1} B S - \lambda S^{-1} S) = \\ \det(S^{-1} (A - \lambda I) S) &= \\ \det(S^{-1}) \det(A - \lambda I) \det(S) &= \det(A - \lambda I) \end{aligned}$$

La seconda parte della dimostrazione la si ricava nel seguente modo

$$Ax = \lambda x \implies (S^{-1}AS)S^{-1}x = \lambda S^{-1}x$$

□

2 Matrici Hermitiane

Le matrici hermitiane sono una classe fondamentale di matrici per le quali vale la proprietà $A = A^H$. Da questa singola uguaglianza derivano diverse proprietà strutturali notevoli: gli elementi sulla diagonale principale sono tutti reali, la traccia (essendo la somma degli autovalori) è un numero reale e, infine, i centri dei cerchi di Gershgorin sono tutti localizzati sull'asse reale.

Un primo risultato fondamentale riguarda la natura dello spettro di queste matrici. Non solo gli autovettori associati ad autovalori distinti sono ortogonali rispetto al prodotto scalare canonico, ma gli autovalori stessi sono garantiti essere reali, anche in presenza di entrate complesse nella matrice.

Theorem 2.1. Tutti gli autovalori di una matrice **hermitiana** sono reali.

Dimostrazione. Sia λ un autovalore di A associato all'autovettore x . Dalla definizione di autovalore si ha $Ax = \lambda x$. Moltiplicando a sinistra da entrambe le parti per x^H si ottiene

$$x^H Ax = \lambda x^H x$$

dividendo da entrambe le parti per $x^H x$ (che rappresenta la norma quadra del vettore, e quindi un reale positivo) si ottiene

$$\lambda = \frac{x^H Ax}{x^H x}$$

Il numeratore del membro di destra (noto come quoziente di Rayleigh) è reale poiché generato da una matrice hermitiana. Essendo reale anche il denominatore, segue la tesi. □

Il calcolo e l'analisi degli autovalori possono essere enormemente semplificati nel caso di matrici **diagonalizzabili**.

Definition 2.1. Una matrice A si dice diagonalizzabile se e solo se esiste una matrice X non singolare tale per cui

$$D = X^{-1}AX, \quad D \text{ in forma diagonale}$$

Una matrice in forma diagonale non è altro che una matrice con tutti elementi nulli tranne quelli sulla diagonale. Quando una matrice è diagonalizzabile e riusciamo a trovare X , la conseguente matrice diagonale appena trovata avrà la forma

$$D = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix}, \quad \lambda_i \text{ autovalori di } A$$

Per le matrici hermitiane, la diagonalizzabilità non è solo un'ipotesi fortunata, ma una certezza strutturale sancita da uno dei risultati più importanti dell'algebra lineare.

Theorem 2.2 (Teorema spettrale per matrici hermitiane). Sia $A \in \mathbb{C}^{n \times n}$ una matrice hermitiana ($A = A^H$). Allora:

- Tutti gli autovalori di A sono reali.
- La matrice A è diagonalizzabile mediante una matrice unitaria.
- Esiste quindi una matrice unitaria $U \in \mathbb{C}^{n \times n}$ (per cui vale $U^H U = I$) composta da autovettori ortonormali di A tale che

$$U^H A U = \Lambda$$

dove Λ è una matrice diagonale reale contenente gli autovalori di A .

Dal teorema spettrale derivano proprietà numeriche di estrema utilità. In particolare, è garantito che la norma due di una qualsiasi matrice hermitiana coincida esattamente con il suo raggio spettrale ($\rho(A)$). Grazie a questa uguaglianza, possiamo dimostrare che il numero di condizionamento in norma due di una matrice hermitiana corrisponde esattamente al rapporto tra il suo autovalore di modulo massimo e quello di modulo minimo:

$$\mu_2(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \rho(A)\rho(A^{-1}) = \max_i |\lambda_i| \cdot \max_j \frac{1}{|\lambda_j|} = \frac{\max |\lambda_i|}{\min |\lambda_j|}$$

3 Norme

Nella fisica classica di tutti i giorni utilizziamo frequentemente grandezze scalari, come ad esempio la **distanza**. Questi concetti trovano un parallelo nel mondo algebrico dei vettori: se infatti un particolare vettore ha un'attinenza con una realtà fisica, è indispensabile introdurre una metrica che permetta di quantificare la sua "distanza" rispetto all'origine $x = 0$. Formalmente, possiamo definire una norma come segue:

Definition 3.1 (Norma vettoriale). Una funzione $\|\cdot\| : \mathbb{C}^n \rightarrow \mathbb{R}_{\geq 0}$ si dice **norma** se rispetta le seguenti proprietà:

- **Annullamento:** Solo il vettore nullo ha norma zero.

$$\|x\| = 0 \iff x = 0$$

- **Omogeneità assoluta:** Se si scala il vettore rispetto a uno scalare $\lambda \in \mathbb{C}$, vale che:

$$\|\lambda x\| = |\lambda| \cdot \|x\|$$

- **Subadditività:** La norma rispetta la disuguaglianza triangolare rispetto alla somma:

$$\|x + y\| \leq \|x\| + \|y\|$$

Esistono particolari tipologie di norme, ampiamente utilizzate nei corsi di algebra lineare e analisi numerica. Un esempio classico è la norma $\|\cdot\|_2$ (norma euclidea), che corrisponde alla radice quadrata della somma dei quadrati dei moduli di ogni componente. Questa norma è un caso specifico della più generale **norma** p , che insieme alla norma ∞ costituisce la base degli spazi normati fondamentali:

$$\underbrace{\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}}_{\text{norma } p} \quad \underbrace{\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|}_{\text{norma infinito}}$$

Una caratteristica cruciale delle norme è la loro capacità di alterare la topologia e la geometria dello spazio vettoriale: **la scelta della norma fa variare le proprietà geometriche (come la forma della "sfera unitaria")**. Tuttavia, in spazi a dimensione finita, pur rappresentando forme geometriche diverse, tutte le norme rispettano un principio di **equivalenza**.

Theorem 3.1 (Teorema delle norme equivalenti). Date due qualsiasi norme vettoriali $\|\cdot\|_a$ e $\|\cdot\|_b$ su uno spazio a dimensione finita $\mathbb{C}^n$, esistono sempre due costanti reali positive α e β tali per cui:

$$\alpha\|x\|_a \leq \|x\|_b \leq \beta\|x\|_a, \quad \forall x \in \mathbb{C}^n$$

Essendo $\mathbb{C}^n$ di dimensione finita, tutte le norme sono topologicamente equivalenti. Nel caso specifico delle norme 1, 2 e ∞ , valgono le seguenti disuguaglianze notevoli (che legano le costanti α e β alla dimensione n dello spazio):

$$\begin{aligned} \|x\|_\infty &\leq \|x\|_1 \leq n\|x\|_\infty \\ \|x\|_\infty &\leq \|x\|_2 \leq \sqrt{n}\|x\|_\infty \\ \|x\|_2 &\leq \|x\|_1 \leq \sqrt{n}\|x\|_2 \end{aligned}$$

A livello di costo computazionale, sia la norma 1 che la norma ∞ richiedono un numero di operazioni dell'ordine di $O(n)$. Possiamo ora generalizzare ulteriormente il concetto estendendolo alle matrici.

Definition 3.2 (Norma matriciale). Una funzione $\|\cdot\|_m : \mathbb{C}^{n \times n} \rightarrow \mathbb{R}_{\geq 0}$ si dice **norma matriciale** se verifica le seguenti proprietà:

- $\|A\|_m = 0 \iff A = 0$ (la matrice nulla è l'unica con norma zero).
- $\|\alpha A\|_m = |\alpha| \cdot \|A\|_m$, per ogni scalare $\alpha \in \mathbb{C}$.
- $\|A + B\|_m \leq \|A\|_m + \|B\|_m$ (Subadditività).

Esiste inoltre una proprietà fondamentale (submoltiplicatività) che spesso le norme matriciali devono soddisfare affinché siano coerenti con il prodotto tra matrici:

$$\|A \cdot B\|_m \leq \|A\|_m \cdot \|B\|_m$$

Una norma matriciale si definisce **indotta** (o **naturale**) se deriva direttamente dall'azione della matrice sui vettori di uno spazio dotato di norma vettoriale $\|\cdot\|_v$:

$$\|A\|_m = \sup_{x \neq 0} \frac{\|Ax\|_v}{\|x\|_v} = \max_{\|x\|_v=1} \|Ax\|_v$$

In parole semplici, considerando la sfera unitaria dei vettori ($\|x\|_v = 1$), la norma indotta di A rappresenta la massima "dilatazione" che la matrice riesce a imprimere a un vettore. Le **norme matriciali indotte** dalle rispettive norme vettoriali 1, 2 e ∞ possiedono formulazioni pratiche molto dirette:

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}| \quad (\text{Massimo somma colonne})$$

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}| \quad (\text{Massimo somma righe})$$

$$\|A\|_2 = \sqrt{\rho(A^H A)} \quad (\text{Norma spettrale})$$

Il terzo caso, la norma 2, è computazionalmente il più costoso poiché richiede il calcolo del raggio spettrale ρ (l'autovalore di modulo massimo) della matrice definita positiva $A^H A$. Tuttavia, non tutte le norme matriciali sono indotte. Un esempio celebre è la **norma di Frobenius**:

$$\|A\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2}$$

Si può dimostrare facilmente per assurdo che questa norma non è indotta da alcuna norma vettoriale. Sappiamo infatti che, per definizione, la norma indotta della matrice Identità I deve necessariamente valere 1:

$$\|I\|_m = \max_{\|x\|_v=1} \frac{\|Ix\|_v}{\|x\|_v} = 1$$

Calcolando però la norma di Frobenius della matrice Identità di dimensione $n \times n$ (che ha n volte il numero 1 sulla diagonale e zero altrove), otteniamo $\|I\|_F = \sqrt{n}$. Poiché $\sqrt{n} \neq 1$ per $n > 1$, la norma di Frobenius non può essere una norma indotta. Esiste infine una caratteristica cruciale che mette in relazione le norme vettoriali e matriciali, definita **compatibilità**.

Definition 3.3 (Norme Compatibili). Una norma matriciale $\|\cdot\|_m$ si dice compatibile con una norma vettoriale $\|\cdot\|_v$ se vale la seguente disuguaglianza:

$$\|Ax\|_v \leq \|A\|_m \cdot \|x\|_v \quad \forall x \in \mathbb{C}^n$$

Per costruzione, ogni norma matriciale indotta è sempre automaticamente compatibile con la norma vettoriale che l'ha generata. Questa proprietà di compatibilità crea un ponte diretto tra il mondo delle norme e quello degli **autovalori**. Considerando l'equazione agli autovalori $Ax = \lambda x$ e applicando la disuguaglianza di compatibilità, si ottiene:

$$\|Ax\|_v = \|\lambda x\|_v = |\lambda| \cdot \|x\|_v \leq \|A\|_m \cdot \|x\|_v$$

Essendo l'autovettore x non nullo, possiamo dividere ambo i membri per $\|x\|_v \neq 0$, arrivando a un risultato teorico di estrema importanza:

$$|\lambda| \leq \|A\|_m$$

Questa relazione garantisce che il modulo di qualsiasi autovalore di una matrice è sempre limitato (maggiorato) dal valore di una qualunque norma matriciale compatibile.

4 Dischi di Gershgorin

La localizzazione degli autovalori di una matrice è un processo alquanto complesso e computazionalmente costoso (è necessario andare a calcolare il determinante di una differenza tra matrici). L'obiettivo che ci poniamo è quindi quello di trovare dei metodi che permettano di fare una stima approssimata sulla posizione degli autovalori di una generica matrice $A \in \mathbb{C}^{n \times n}$. Fino ad ora abbiamo visto solo un risultato utile a tale scopo, tuttavia l'assunzione $\|\cdot\|_m$ compatibile può essere rimossa, ossia, il risultato vale per ogni norma matriciale: questo risultato è noto come teorema di **Hirsch**:

Theorem 4.1. Data $A \in \mathbb{C}^{n \times n}$, allora $\forall \|\cdot\|$ vale

$$\rho(A) = \max |\lambda_i| \leq \|A\|, \quad \lambda_i \text{ Autovettori di } A$$

Dimostrazione. Si supponga (λ, x) aut Coppia per A . Si ipotizzi di costruire una matrice B tale per cui

$$B = \begin{bmatrix} x_1 & 0 & 0 & \dots & 0 & 0 \\ x_2 & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ x_{n-1} & 0 & 0 & \dots & 0 & 0 \\ x_n & 0 & 0 & \dots & 0 & 0 \end{bmatrix}$$

quindi con il vettore x sulla prima colonna e 0 sulle restanti. Quindi, il prodotto $A \cdot B$, visto per colonne, è equivalente a

$$A \cdot B = [Ax \quad A \cdot 0 \quad \dots \quad A \cdot 0]$$

Dalla definizione di autovalore di una matrice, secondo la quale $\lambda x = Ax$, possiamo scrivere colonne, è equivalente a

$$[\lambda x \quad \lambda \cdot 0 \quad \dots \quad \lambda \cdot 0]$$

si può portare λ in evidenza

$$\lambda [x \quad 0 \quad \dots \quad 0] = \lambda B = A \cdot B$$

dalla catena di uguaglianze si ricava

$$\|\lambda B\| = \|A \cdot B\| \leq \|A\| \cdot \|B\|$$

dividendo tutto per la **norma** di B si ottiene

$$|\lambda| \leq \|A\|$$

□

Il problema di questo teorema è la sua enorme approssimatezza, infatti ci viene garantito che i cerchi si trovino in una determinata porzione del piano complesso che può essere anche molto grande. Introduciamo quindi altri tre teoremi, noti come **teoremi di Gershgorin**. Affinché sia possibile introdurre tali teoremi è prima necessario descrivere la struttura matematica su cui essi si basano: i **dischi di Gershgorin**.

Definition 4.1. (dischi di Gershgorin) Data una matrice $A \in \mathbb{C}^{n \times n}$ definiamo il **disco di Gershgorin** i -esimo come

$$\mathcal{F}_i(A) = \{z \in \mathbb{C} : |z - a_{ii}| \leq \rho_i\}, \quad \rho_i = \sum_{j=1, j \neq i}^n |a_{ij}|$$

detto anche **disco di Gershgorin associato alla riga i -esima** della matrice A .

Una volta definito il significato di **Disco di Gershgorin** si possono introdurre i tre teoremi fondamentali, i quali permettono fondamentalmente di **localizzare gli autovalori** della matrice in modo semplice e veloce. Il primo di questi teoremi corrisponde a:

Theorem 4.2. (Primo Teorema di Gershgorin) Sia $A \in \mathbb{C}^{n \times n}$. Se $\lambda \in \mathbb{C}$ è un autovalore di A , allora esso appartiene all'unione di tutti i dischi di Gershgorin relativi alla matrice A :

$$\lambda \in \bigcup_{i=1}^n \mathcal{F}_i(A)$$

Dimostrazione. Sia (λ, x) un'autocoppia per A e indichiamo con k l'indice della componente di massimo modulo nel vettore x :

$$|x_k| = \max_{1 \leq i \leq n} |x_i|$$

Poiché x è un autovettore, per definizione $x \neq 0$, il che ci garantisce che $|x_k| > 0$ e ci permette di dividere per questa quantità senza perdere di significato. Dalla definizione di autovalore si ha $Ax = \lambda x$. Scrivendo questa relazione per componenti, l'equazione k -esima del sistema lineare risulta:

$$\sum_{j=1}^n a_{kj} x_j = \lambda x_k$$

Estraiamo dalla sommatoria il termine sulla diagonale (relativo a $j = k$) e portiamolo a destra dell'uguaglianza:

$$\sum_{\substack{j=1 \\ j \neq k}}^n a_{kj} x_j = \lambda x_k - a_{kk} x_k = (\lambda - a_{kk}) x_k$$

Applichiamo l'operatore modulo da entrambe le parti. Sfruttando la disuguaglianza triangolare al membro sinistro, otteniamo:

$$|\lambda - a_{kk}| |x_k| = \left| \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj} x_j \right| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| |x_j|$$

Dividiamo ora entrambi i membri per $|x_k|$:

$$|\lambda - a_{kk}| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| \frac{|x_j|}{|x_k|}$$

Poiché x_k è per costruzione la componente di modulo massimo del vettore x , sappiamo che $|x_j| \leq |x_k|$ per ogni indice j . Di conseguenza, il rapporto $\frac{|x_j|}{|x_k|} \leq 1$. Sostituendo questa maggiorazione si ottiene:

$$|\lambda - a_{kk}| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| \cdot 1 = r_k$$

Questa espressione geometrica significa esattamente che $\lambda \in \mathcal{F}_k(A)$, ovvero che l'autovalore dista dal centro a_{kk} meno del raggio r_k , cadendo così all'interno del k -esimo disco di Gershgorin. Essendo contenuto in uno specifico disco, λ apparterrà inevitabilmente anche all'unione di tutti i dischi della matrice. Questo conclude la dimostrazione. $\square$

Il teorema appena enunciato è però estremamente debole: viene garantito solo che l'autovalore λ della matrice A appartiene all'unione di tutti i cerchi di **Gershgorin** della matrice A . Inoltre, abbiamo dimostrato che $\lambda \in F$ supponendo di conoscere l'autovettore (la conoscenza di x_k richiede che sia noto l'autovettore a priori), ma nella situazione usuale non conosciamo e non sappiamo niente a riguardo degli autovalori (e quindi degli autovettori). Enunciamo quindi il **II teorema di Gershgorin**

Theorem 4.3. (II teorema di Gershgorin) Se M_1 è unione di s cerchi di Gershgorin ed M_2 è unione di $m-s$ cerchi di Gershgorin e, se vale $M_1 \cap M_2 = \emptyset$, allora M_1 contiene esattamente s autovalori e M_2 contiene gli altri $m - s$ autovalori

Questo risultato è sicuramente più forte del primo, anche se non ci garantisce che ad ogni cerchio sia necessariamente associato un autovalore: potrebbero esistere dei cerchi che non contengono nessun autovalore. Però, se ho un cerchio isolato dai restanti, posso dire con assoluta certezza che al suo interno sia compreso un autovalore. Questo teorema porta con sé due corollari

- Se la matrice A ha n cerchi disgiunti, allora ha n autovalori distinti e, di conseguenza, **molteplicità algebrica** e **molteplicità geometrica** sono entrambi pari a uno. Quindi, dalle considerazioni fatte nei capitoli precedenti, la **matrice è diagonalizzabile**.
- Se la matrice A , a coefficienti reali, ha un cerchio isolato, allora quel cerchio conterrà un solo autovalore reale.

Anche il primo teorema porta con sé un corollario, che però è applicabile solo nel caso in cui la matrice sia a **predominanza diagonale forte**

Definition 4.2. Una matrice si dice a **predominanza diagonale forte** se e solo se la somma dei moduli degli elementi fuori dalla diagonale è minore dell'elemento sulla diagonale **per ogni riga**

$$a_{ii} > \sum_{i=1, i \neq j}^n a_{ij}$$

Le **matrici a predominanza diagonale forte** sicuramente invertibili. Lo si può disegnando i cerchi di Gershgorin: siccome l'elemento sulla diagonale è maggiore del raggio del cerchio, nessun cerchio comprenderà mai l'origine del piano complesso, ergo $0 \notin \mathcal{F}_i(A)$, $\forall i = 1, \dots, m$ e quindi $\det(A) \neq 0$.

Un modo per restringere l'area in cui si localizzano gli autovalori consiste nel considerare anche i cerchi della matrice A^T . Infatti, siccome A e A^T hanno medesimi autovalori, allora quest'ultimi saranno sicuramente compresi nell'intersezione dei cerchi delle due matrici.

$$\bigcup \mathcal{F}_i(A) \cap \bigcup \mathcal{F}(A^T)$$

Il terzo risultato permette di escludere che 0 sia un autovalore della matrice, a patto che la matrice sia **irriducibile**. Per introdurre il concetto di matrice irriducibile è necessario introdurre il concetto di grafo di una **matrice**

Definition 4.3. (Grafo di una matrice) Data una matrice $a \in \mathbb{C}^{n \times n}$ si definisce **grafo associato alla matrice A** $G(A) = (V, E)$, il grafo tale per cui $V = \{1, \dots, n\}$ e $E = \{(i, j) : a_{ij} \neq 0\}$. Se l'elemento $a_{ij} \neq 0$ possiamo dire che il nodo **i** è **connesso al nodo j**

Definition 4.4. (Grafo fortemente connesso) Un grafo si dice **fortemente connesso** se, per ogni scelta di vertici $i, j \in N$ esiste un **cammino orientato** che connette **i** e **j**

Definition 4.5. (Matrice irriducibile) Una matrice $A \in \mathbb{C}^{n \times n}$ si dice **irriducibile** se il **grafo** associato alla matrice è **fortemente connesso**

Il concetto di matrice **irriducibile** è fortemente legato all'enunciato del terzo teorema di **Gershgorin**, ed è inoltre un concetto molto utile anche nell'ambito della convergenza dei sistemi lineari:

Theorem 4.4. Data una matrice $A \in \mathbb{C}^{n \times n}$ irriducibile, supponendo λ autovalore di A , se vale che λ appartiene al bordo dell'unione di tutti i cerchi:

$$\lambda \in \partial \left(\bigcup_{i=1}^n \mathcal{F}_i(A) \right)$$

Allora λ appartiene alla frontiera dell'intersezione di tutti i cerchi, cioè a tutti i punti in cui i bordi dei cerchi si intersecano tra di loro:

$$\lambda \in \bigcap_{i=1}^n \partial \left(\mathcal{F}_i(A) \right)$$

In termini semplici: se un **autovalore** appartiene al bordo di un cerchio di **Gershgorin**, allora deve appartenere a tutti.

Il terzo teorema di Gershgorin può essere anche utilizzato per dimostrare che le **matrici a predominanza diagonali deboli** che sono anche **irriducibili** sono sicuramente **non singolari**, cioè hanno determinante diverso da 0. Definiamo il concetto di matrice a predominanza diagonale debole:

Definition 4.6. (matrici a predominanza diagonale debole) Una matrice si dice **a predominanza diagonale forte** se e solo se la somma dei moduli degli elementi fuori dalla diagonale è minore dell'elemento sulla diagonale **per ogni riga**

$$a_{ii} \geq \sum_{\substack{i=1 \\ i \neq j}}^n a_{ij}$$

ed esiste almeno un k tale per cui

$$a_{kk} > \sum_{\substack{i=1 \\ i \neq j}}^n a_{ij}$$

Per definizione stessa di matrice a predominanza diagonale debole, esisterà un certo numero di cerchi per cui $\lambda = 0$ si trova in corrispondenza della frontiera stessa del cerchio, mentre esisterà un certo numero di cerchi per cui $\lambda = 0$ non si trova in corrispondenza del bordo di un cerchio. Siccome esiste almeno un elemento per cui $\lambda = 0$ appartiene al bordo del cerchio, allora $\lambda = 0$ apparerà necessariamente all'unione di tutti i cerchi di **Gershgorin**. Dal terzo teorema di Gershgorin sappiamo tuttavia che, affinché $\lambda = 0$ sia autovalore della matrice, è necessario che appartenga anche all'intersezione delle frontiere di tutti i cerchi di Gershgorin. Siccome però, nel caso di matrici a predominanza diagonale deboli, esiste almeno un particolare indice per cui il raggio del cerchio non interseca $\lambda = 0$, è possibile concludere che quest'ultimo non è autovalore della matrice. Pertanto, possiamo asserire con certezza che nel caso di matrici a predominanza diagonale debole e irriducibili, l'invertibilità è sempre garantita.

CAPITOLO 4

SISTEMI LINEARI

Uno dei problemi che affrontiamo durante questo corso riguarda l'approssimazione e la risoluzione di **sistemi lineari**. In generale, possiamo definire un sistema lineare come una serie di equazioni nella forma

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n} = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n} = b_2 \\ \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn} = b_n \end{cases}$$

Possiamo scrivere il sistema in una forma più compatta, costituita da una **matrice dei coefficienti**, un **vettore delle incognite** e un **vettore dei termini noti** nella forma:

$$Ax = b, \quad A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Esistono due classi di metodi che permettono di risolvere queste tipologie di sistemi. La prima classe di metodi, quella dei **metodi esatti**, è costituita da tutti quei metodi che riescono, in un numero finito di passi, ad arrivare ad una soluzione $\bar{x}$ **esatta**. La seconda classe di metodi, quella dei **metodi iterativi**, è costituita da tutti quei metodi che generano una successione $\{x_k\}_{k \in \mathbb{N}}$ tale per cui

$$\lim_{k \rightarrow +\infty} x_k = x$$

Tra i metodi esatti troviamo il **metodo dell'eliminazione di Gauss** e il **metodo di Cramer**. Invece, tra i metodi iterativi troviamo la **fattorizzazione QR**.

1 Matrici a blocchi

Può essere talvolta conveniente definire una matrice in termini di sottomatrici al posto di esplicitarne le entrate scalari. Questa tipologia di matrici prende il nome di **matrici a blocchi**:

Definition 1.1. Si definisce **matrice a blocchi** una qualsiasi matrice $A \in \mathbb{C}^{n \times n}$ tale per cui

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad A_{ij} \in \mathbb{C}^{m_i \times n_i}, \quad \sum_{i=1}^s m_i = m, \quad \sum_{i=1}^p n_i = n$$

Le ultime due condizioni permettono di imporre dei vincoli sulla scelta dei sottoblocchi, in particolare, blocchi allineati per righe devono avere lo stesso numero di righe e blocchi allineati

per colonne devono avere lo stesso numero di colonne. Imponendo queste due condizioni rendiamo, di fatto, sempre possibile scrivere una matrice con i suoi sottoblocchi (poiché di dimensioni compatibili). Nel caso in cui

$$\begin{cases} m = n \\ s = t \\ m_i = n_i \quad \forall i = 1, \dots, t \end{cases}$$

Allora possiamo definire una matrice come **triangolare a blocchi** se e solo se è congrua alla seguente struttura

$$\begin{bmatrix} A_{11} & 0 \\ A_{12} & A_{22} \end{bmatrix} \quad \text{oppure} \quad \begin{bmatrix} A_{11} & A_{21} \\ 0 & A_{22} \end{bmatrix}$$

Se invece i soli elementi sulla diagonale sono diversi da 0, allora la matrice si dice **diagonale a blocchi**. La possibilità di definire della matrici che siano diagonali a blocchi ci consente di sbloccare delle proprietà estremamente utili

- Gli autovalori della matrice A non sono altro che l'unione degli autovalori delle matrici A_{ii} sulla diagonale.
- Il determinante della matrice A non è altro che il prodotto dei determinanti delle matrici A_{ii} sulla diagonale.
- L'inversa di A non è altro che una matrice triangolare in cui gli elementi sulla diagonale sono invertiti.
- Le operazioni tra matrici scritte a blocchi con blocchi diagonali quadrati, si possono scrivere in modo analogo alle operazioni scalari.

Passando dalle matrici a blocchi siamo in grado di definire anche il concetto di **matrice riducibile**, duale al concetto di **matrice irriducibile**:

Definition 1.2. Data una matrice $A \in \mathbb{C}^{n \times n}$, si dice che A è **riducibile** se e solo se esiste una matrice di permutazione $\Pi \in \mathbb{R}^{n \times n}$ tale per cui la matrice $\Pi A \Pi^T$ è triangolare a blocchi.

$$\Pi A \Pi^T = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ 0 & A_{22} & A_{23} & \dots & A_{2n} \\ 0 & 0 & A_{33} & \dots & A_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & A_{nn} \end{bmatrix}$$

Questa definizione rimane valida anche per matrici di permutazione che portano la matrice in forma triangolare a blocchi inferiore.

La prima osservazione che occorre fare è che la matrice di permutazione non è unica, possono esistere diverse matrici di permutazione che portano il sistema nella forma desiderata. Tuttavia, abbiamo definito le matrici **irriducibili** a partire dal grafo della matrice, mentre abbiamo definito le matrici **riducibili** a partire dal fatto che sia possibile trovare o meno una matrice di permutazione che porta la matrice in una forma desiderata. Introducendo un teorema, che dimostreremo, è possibile unificare le due definizioni andando a ricondurre anche il caso delle matrici riducibili a definizioni sul grafo della matrice

Theorem 1.1. Una matrice $A \in \mathbb{C}^{n \times n}$ è **riducibile** se e solo se il suo grafo orientato associato $G(A)$ non è **fortemente connesso**.

$$A \text{ riducibile} \iff G(A) \text{ non fortemente connesso}$$

Dimostrazione. Data una matrice di permutazione Π , si può dimostrare che i grafi $G(\Pi A \Pi^T)$ e $G(A)$ sono isomorfi (cambia solo l'etichettatura dei vertici). Quindi, se $G(A)$ è fortemente connesso, anche $G(\Pi A \Pi^T)$ lo è, e viceversa. Dimostriamo le due implicazioni:

($\Rightarrow$) Se la matrice A è riducibile, esiste una matrice di permutazione Π tale per cui:

$$B = \Pi A \Pi^T = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix}$$

dove A_{22} è una sottomatrice quadrata di $(n - k)$ righe, mentre A_{11} è quadrata di k righe (e A_{12} ha dimensioni coerenti). Vista la presenza del blocco nullo in basso a sinistra, non esiste alcun arco che colleghi i vertici dell'insieme $\{k + 1, \dots, n\}$ ai vertici dell'insieme $\{1, \dots, k\}$. Di conseguenza, partendo da un vertice del secondo gruppo, non è possibile raggiungere alcun vertice del primo. Il grafo $G(B)$ non è quindi fortemente connesso e, per l'isomorfismo, non lo è neanche $G(A)$.

($\Leftarrow$) Se il grafo $G(A)$ non è fortemente connesso, allora esiste almeno una coppia di vertici (j, h) tale per cui partendo dal vertice j non è possibile raggiungere il vertice h . Dividiamo quindi l'insieme degli indici nei due seguenti sottoinsiemi:

$$\mathcal{P} = \{x \in \{1, \dots, n\} : x \text{ è raggiungibile da } j\}$$

$$\mathcal{Q} = \{x \in \{1, \dots, n\} : x \text{ non è raggiungibile da } j\}$$

Questi insiemi non sono vuoti poiché $j \in \mathcal{P}$ (un vertice raggiunge sempre se stesso) e $h \in \mathcal{Q}$. Consideriamo ora una matrice di permutazione che porti in testa alla matrice gli indici di $\mathcal{Q}$ e in coda gli indici di $\mathcal{P}$, ottenendo una partizione a blocchi:

$$\begin{bmatrix} A_{\mathcal{Q}\mathcal{Q}} & A_{\mathcal{Q}\mathcal{P}} \\ A_{\mathcal{P}\mathcal{Q}} & A_{\mathcal{P}\mathcal{P}} \end{bmatrix}$$

Se esistesse un arco da un vertice $p \in \mathcal{P}$ a un vertice $q \in \mathcal{Q}$, significherebbe che q è raggiungibile da p . Tuttavia, poiché p è a sua volta raggiungibile da j , allora anche q sarebbe raggiungibile da j , il che contraddice l'appartenenza $q \in \mathcal{Q}$.

Di conseguenza, non può esistere alcun arco da $\mathcal{P}$ a $\mathcal{Q}$, il che impone che il blocco $A_{\mathcal{P}\mathcal{Q}}$ sia identicamente nullo. Abbiamo quindi trovato una permutazione che porta la matrice in forma triangolare a blocchi:

$$\begin{bmatrix} A_{\mathcal{Q}\mathcal{Q}} & A_{\mathcal{Q}\mathcal{P}} \\ 0 & A_{\mathcal{P}\mathcal{P}} \end{bmatrix}$$

Pertanto, la matrice A è **riducibile**. □

In appendice è possibile anche trovare un procedimento algebrico che, sfruttando questa dimostrazione, permette di ricavare la matrice di permutazione per la matrice riducibile. Le matrici a blocchi possono semplificare il calcolo della soluzione di un sistema lineare. Dato il sistema $Ax = b$, con A riducibile, posso trovare una matrice di permutazione Π e scrivere

$$\Pi Ax = \Pi b$$

Le matrici di permutazioni sono matrici ortogonali, cioè rispettano la proprietà $\Pi \cdot \Pi^T = I$, pertanto è possibile utilizzare questo fatto per fare riuscire a permutare la matrice A ; in particolare, scriviamo Ax come AIx e sostituiamo l'equivalenza ottenuta in precedenza, così da ottenere:

$$\underbrace{\Pi A \Pi^T}_B \underbrace{\Pi x}_y = \underbrace{\Pi b}_c = By = c \rightarrow \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$$

Il sistema appena ottenuto è triangolare inferiore a **blocchi** con $y_1, c_1 \in \mathbb{C}^k$ e $y_2, c_2 \in \mathbb{C}^{n-k}$. Anche la risoluzione è abbastanza semplice, è infatti sufficiente risolvere il secondo sistema lineare e usare la soluzione per risolvere anche il primo. Il costo computazionale di questa risoluzione è

$$O(k^3 + (n - k)^3 + k(n - k)) = O(k^3 + (n - k)^3)$$

che nel caso ideale, quello in cui $k = n/2$, si ottiene $O(n^3/4)$. La particolarità di questo approccio è quella di poter applicare ricorsivamente la scomposizione attraverso matrici di permutazione fintanto che non si ottiene una matrice triangolare inferiore non a blocchi.

2 Metodi diretti

Il primo dei metodi che andiamo ad descrivere è il **metodo di Gauss**, un metodo **esatto** che permette di arrivare a una soluzione esatta del sistema lineare in un numero finito di passi. L'idea dell'algoritmo è quella di portare il sistema in forma triangolare inferiore per poi risolvere attraverso un processo di sostituzione all'indietro. Quindi, dato un sistema di **k equazioni** in **k incognite** nella forma:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1k}x_k &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2k}x_k &= b_2 \\ &\vdots \\ a_{k1}x_1 + a_{k2}x_2 + \cdots + a_{kk}x_k &= b_k. \end{aligned}$$

dove supponiamo la matrice dei coefficienti non singolare, così da garantire esistenza e unicità della soluzione, costruiamo un sistema equivalente nella forma $Rx = c$ in forma **triangolare superiore** attraverso una serie di prodotti e somme tra righe:

$$\begin{aligned} r_{11}x_1 + r_{12}x_2 + \cdots + r_{1n}x_n &= c_1 \\ r_{22}x_2 + \cdots + r_{2n}x_n &= c_2 \\ &\vdots \\ r_{nn}x_n &= c_n \end{aligned}$$

Quindi, l'idea dell'algoritmo è analoga all'idea che abbiamo visto nelle matrici a blocchi, con l'unica differenza che nel caso dell'algoritmo di Gauss non è necessario che la matrice sia **riducibile**. Una volta costruito un sistema in questa forma è possibile risolvere il sistema soltanto utilizzando operazioni aritmetiche di base

$$\begin{cases} x_n = \frac{c_n}{a_{nn}} \\ x_j = \frac{1}{u_{jj}} \left(c_j - \sum_{i=j+1}^n u_{ji}x_i \right) \end{cases}$$

Il secondo termine è in realtà molto semplice, il termine u_{jj} è coefficiente che moltiplicava l'elemento che vogliamo calcolare x_j , c_j è il termine noto sulla riga j -esima e la sommatoria è la somma dei coefficienti successivi a x_j (ecco perché $j+1$, è sufficiente immaginare di muoversi verso destra) moltiplicati per gli x_j calcolati in precedenza. Il costo di questa operazione corrisponde a

$$(n-j) \text{ moltiplicazioni} + (n-j-1) \text{ somme} + 1 \text{ somma} + 1 \text{ divisione} \quad (1)$$

Sommando tutto si ottiene $2(n-j) + 1$ operazioni, che sommate per n elementi danno come complessità computazionale $O(n^2)$.

L'algoritmo di Gauss ci permette quindi di giungere a un sistema lineare posto nella seguente forma; in particolare, l'idea è che alla j -esima iterazione si rimpiazza la j -esima equazione con la differenza tra tale equazione e un'altra equazione moltiplicata per un particolare coefficiente. In particolare, ad ogni iterazione si cerca di eliminare tutti i coefficienti che si trovano sotto all'elemento pivot a_{kk} andando a moltiplicare la riga k -esima per un coefficiente moltiplicatore, per poi sottrarlo alla riga j -esima di cui vogliamo azzerare l'elemento:

$$\boxed{A(j, :) - \frac{a_{jk}}{a_{kk}} \cdot A(k, :)}, \quad k+1 \leq j \leq n \wedge k = 1, \dots, n$$

Le quantità l_{ij} sono dette **moltiplicatori** e affinché sia possibile eseguire l'algoritmo è necessaria che venga rispettata la condizione $a_{jj}^{(j)} \neq 0, \forall j = 1, \dots, n-1$. Il problema principale dell'algoritmo di Gauss è l'eccessivo costo computazionale, supponendo che l'operazione di moltiplicazione abbia un costo computazionale di $O(2n^3/3)$ risulta che il costo complessivo dell'algoritmo di Gauss corrisponde a:

$$O\left(\frac{2}{3}n^3 + n^2\right) \approx O(n^3)$$

2.1 Fattorizzazione LU

Uno dei modi per risolvere un sistema lineare $Ax = b$, riducendo il costo operativo nella fase di risoluzione rispetto all'eliminazione di Gauss applicata direttamente, consiste nel determinare la **fattorizzazione LU** della matrice A . L'accezione **LU** indica che la matrice A viene scomposta nel prodotto di due matrici: una matrice L (*Lower*) triangolare inferiore con elementi unitari sulla diagonale principale, e una matrice U (*Upper*) triangolare superiore. L'operazione si basa sulla moltiplicazione di entrambi i membri dell'equazione $Ax = b$, **a sinistra**, per una sequenza di matrici dette **matrici elementari di Gauss**. Al passo j -esimo (con $1 \leq j \leq n-1$), la matrice elementare H_j assume la forma:

$$H_j = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{j+1,j} & \ddots & \\ & & \vdots & & 1 \end{bmatrix}, \quad \ell_{ij} = \frac{a_{ij}^{(j)}}{a_{jj}^{(j)}} \quad \text{per } i = j+1, \dots, n$$

Iterando il procedimento per le $n-1$ colonne della matrice, si ottiene l'equazione trasformata:

$$\left(\prod_{j=1}^{n-1} H_j \right) A = \left(\prod_{j=1}^{n-1} H_j \right) b$$

Definiamo U la matrice risultante dal prodotto $\left(\prod_{j=1}^{n-1} H_j \right) A$; essa corrisponde esattamente alla matrice triangolare superiore generata al termine dell'algoritmo di eliminazione di Gauss. Per ricavare la matrice L , è sufficiente applicare l'inversa del prodotto delle matrici elementari ad entrambi i membri:

$$A = \left(\prod_{j=1}^{n-1} H_j \right)^{-1} U = (H_1^{-1} H_2^{-1} \dots H_{n-1}^{-1}) U = LU$$

La matrice L coincide con la matrice dei **moltiplicatori**, ossia una matrice triangolare inferiore con tutti 1 sulla diagonale principale e i termini ℓ_{ij} al di sotto di essa. Questa proprietà è dimostrabile analizzando la struttura algebrica delle matrici H_j , le quali possono essere espresse come somma della matrice identità I e di una matrice nilpotente N_j :

$$N_j = \begin{bmatrix} 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & -\ell_{j+1,j} & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & -\ell_{nj} & \dots & 0 \end{bmatrix}$$

Dati due indici tali che $i < j$, il prodotto $N_i \cdot N_j$ è sempre identicamente nullo. Inoltre, l'inversa di ciascuna matrice elementare si ottiene semplicemente invertendo il segno dei moltiplicatori al suo interno (ossia $H_j^{-1} = I - N_j$), permettendo di calcolare la matrice L tramite una semplice somma algebrica, senza ricorrere al prodotto riga per colonna:

$$H_j^{-1} = I - N_j \implies L = \prod_{j=1}^{n-1} H_j^{-1} = I - \sum_{j=1}^{n-1} N_j$$

Dal punto di vista computazionale, il calcolo della fattorizzazione $A = LU$ richiede $O\left(\frac{2}{3}n^3\right)$ operazioni Floating Point (FLOPS). Il grande vantaggio del metodo risiede nella risoluzione del sistema lineare: una volta nota la scomposizione, il problema $Ax = b$ si riduce alla risoluzione sequenziale di due sistemi triangolari:

1. $Lc = b$ (risolvibile tramite sostituzione in avanti in tempo $O(n^2)$);

2. $Ux = c$ (risolubile tramite sostituzione all'indietro in tempo $O(n^2)$).

Qualora sia necessario risolvere molteplici sistemi lineari associati alla medesima matrice dei coefficienti A ma con termini noti b differenti, il costo dominante $O(n^3)$ viene affrontato una sola volta.

3 Pivoting

Supponiamo di voler risolvere il sistema lineare $Ax = b$, tuttavia andando a introdurre una piccola accortezza che riduce il rischio di avere moltiplicatori indefiniti o eccessivamente grandi e, quindi, difficilmente gestibili, sfruttando delle matrici particolari; questa tecnica prende il nome di **pivoting** sulle righe. Al passo j -esimo dell'algoritmo di Gauss stiamo cercando di annullare gli elementi che si trovano alla posizione $(j+1, i), \dots, (n, i)$ (tutti gli elementi sotto al pivot che stiamo considerando). Supponiamo che si presenti uno di questi due casi:

- $a_{jj}^{(k)} \approx 0$.
- $a_{jj}^{(k)} = 0$

Entrambe queste situazioni sono **casi indesiderati**: nel primo caso il moltiplicatore l_{ij} sarà eccessivamente grande, mentre, nel secondo caso risulta del tutto indefinito. Possiamo eliminare - o contenere - queste problematiche andando a sfruttare delle **matrici di permutazioni** che vadano a fare degli scambi tra le righe della matrice. In particolare, sia la riga j la riga contenente il pivot attuale, r la riga sotto a quella che contiene il pivot con l'elemento sottodiagonale di modulo maggiore:

$$|a_{rj}| = \max_{j \leq i \leq n} |a_{ij}^{(k)}|$$

Andiamo ad applicare una matrice di permutazione Π tale per cui la riga j e la riga r vengono scambiate. Facendo in questo modo abbiamo posto come **pivot** l'elemento di massimo modulo tra quelli sotto la diagonale. Tuttavia, quando lavoriamo con la fattorizzazione **LU**, non è altrettanto facile integrare questo meccanismo. Ricordiamo infatti che la fattorizzazione **LU** corrisponde a moltiplicare a sinistra i termini di $Ax = b$ per delle opportune matrici:

$$H_{n-1} \dots H_1 Ax = H_{n-1} \dots H_1 b$$

Se infatti andiamo a applicare, ad ogni passo dell'algoritmo, la matrice di permutazione e la matrice elementare di **Gauss**, otteniamo una situazione di questo tipo:

$$H_{n-1} \Pi_{n-1} \dots H_1 \Pi_1 Ax = H_{n-1} \Pi_{n-1} \dots H_1 \Pi_1 b \rightarrow LU = \Pi A$$

La matrice L contiene i moltiplicatori a cui sono stati applicati gli scambi sulle righe con la seguente regola: *se al passo j applico Π_j , applico lo scambio alle prime $j-1$ colonne di L* . Mentre la matrice Π contiene la composizione di tutte le permutazioni sulle righe di A . Un'osservazione che possiamo fare riguarda il calcolo del determinante di A : applicando gli sviluppi di Laplace la complessità computazionale è circa $O(n!)$, cioè un costo computazionale altissimo. Tuttavia, attraverso il teorema di Binet-Cauchy è possibile semplificare enormemente il calcolo del determinante:

$$\begin{aligned} \det(\Pi A) &= \det(LU) \\ \det(\Pi) \cdot \det(A) &= \det(L) \cdot \det(U) \\ \det(A) &= \frac{\det(L) \cdot \det(U)}{\det(\Pi)} = \frac{\det(U)}{\det(\Pi)} = \boxed{\det(U) \cdot (-1)^s} \end{aligned}$$

Dove s è il numero di scambi di riga che sono stati fatti nell'esecuzione dell'algoritmo di Gauss. Calcolare il determinante di una matrice sfruttando questo metodo porta la complessità computazionale a circa $O(2n^3/3)$.

4 Condizionamento di un sistema lineare

Qualunque metodo per la risoluzione di un sistema lineare produce una soluzione approssimata a causa degli errori di arrotondamento alle operazioni algebriche tra vettori in aritmetica finita. Tali errori vengono amplificati e trasmessi alla soluzione attraverso un meccanismo che dipende sia dall'algoritmo che dallo stesso sistema.

Supponiamo che sia x la soluzione esatta del sistema $Ax = b$. Ipotizziamo ora che risolvendo il sistema lineare in questione si ricavi una soluzione approssimata del tipo $x + \delta x$, dove δx è il vettore relativo agli errori di arrotondamento delle operazioni. Supponendo che esistano degli errori sui dati, corrispondenti a δA e δb , possiamo quindi dire che $x + \delta x$ è la soluzione esatta del problema perturbato

$$(A + \delta A)(x + \delta x) = (b + \delta b)$$

Il problema che ci poniamo è inverso, conoscendo infatti le perturbazioni sulla matrice dei coefficienti, ammettendo che esse siano entrambe piccole, ci piacerebbe che anche la perturbazione δx sia piccola. Ciò però non è vero in generale, è infatti sufficiente prendere questi due esempi di sistemi lineari perturbati per rendersene conto:

$$\begin{aligned} \begin{bmatrix} 3 & 4 \\ 3 & 4,00001 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= \begin{bmatrix} 7 \\ 7.00001 \end{bmatrix} \\ \begin{bmatrix} 3 & 4 \\ 3 & 3,99999 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= \begin{bmatrix} 7 \\ 7.00001 \end{bmatrix} \end{aligned} \quad (2)$$

Le soluzioni sono infatti perturbate nell'ordine delle unità, mentre le perturbazioni sui dati iniziali erano nell'ordine di 10^{-5} . Un algoritmo per cui le perturbazioni sono molto forti viene detto **instabile**, altrimenti, se le perturbazioni sono molto piccole, viene detto *stabile*. L'entità dell'errore relativo e dell'errore assoluto, definiti rispettivamente come:

$$\varepsilon_x = \frac{\|\delta x\|}{\|x\|} \quad \delta_x = \|\delta x\|$$

dipendono dalla sensibilità della soluzione alle perturbazioni dei dati A e b o, come si dice, dal *condizionamento* del sistema. Con il termine *condizionamento* di un sistema indichiamo l'attitudine che ha un dato problema a trasmettere le perturbazioni dei dati alla soluzione. Possiamo quindi cercare di dare una stima dall'alto di questi errori, trovando delle maggiorazioni che ci permettano di stimare il loro valore nel caso peggiore. Ipotizzando che $\delta A = 0$ risulta che:

$$A(x + \delta x) = b + \delta b$$

Sappiamo che per x , soluzione del sistema, vale che $Ax = b$; andando quindi a sostituire questa relazione ottiene

$$A\delta x = \delta b \rightarrow \|A\| \cdot \|\delta x\| \leq \|\delta b\| \quad \boxed{\|\delta x\| \leq \|A^{-1}\| \cdot \|\delta b\|}$$

Per dare una maggiorazione dell'errore relativo è necessario trovare una minorazione per il termine al denominatore $\|x\|$. Anche in questo caso è sufficiente considerare la relazione $Ax = b$, che riportata alle norme, equivale a:

$$\|b\| \leq \|x\| \cdot \|A\| \rightarrow \|x\| \geq \|A\| \cdot \|b\|$$

Andando a unire con la maggiorazione per l'errore assoluto si ottiene

$$\frac{\|\delta x\|}{\|x\|} \leq \boxed{\|A\| \cdot \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}}$$

Nel caso generale in cui la perturbazione su δA non sia nulla, è possibile utilizzare il seguente teorema, il quale definisce una maggiorazione generale per l'errore relativo sul risultato x :

Theorem 4.1. Nell'ipotesi che la matrice $A + \delta A$ sia **non singolare** e che, rispetto a una data norma, sia $\|\delta A\| < 1/\|A^{-1}\|$, vale la relazione

$$\varepsilon_x = \frac{\|\delta x\|}{\|x\|} \leq \frac{\mu(A)}{1 - \mu(A) \frac{\|\delta A\|}{\|A\|}} \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right)$$

dove $\mu(A) = \|A\| \cdot \|A^{-1}\|$

Quando il numero $\mu(A)$ è *molto grande* si ha una forte amplificazione del membro destro della disuguaglianza e l'errore relativo può essere anche molto grande. Pertanto utilizzeremo il valore di $\mu(A)$ come indicatore per il condizionamento del sistema o della matrice A e si indica tale numero come *numero di condizionamento* del sistema. Se $\mu(A) \gg 1$ la matrice A è **malcondizionata**, altrimenti, se $\mu(A)$ è *piccolo*, la matrice A è **bencondizionata**.

Tuttavia, calcolare il numero di condizionamento è un'operazione costosa e estremamente lunga. Conviene cercare di stimarlo attraverso un algoritmo che sia meno costoso dal punto di vista computazionale. Supponiamo di aver calcolato $x + \delta x$ con un qualunque metodo e calcoliamo il resto residuo $r = b - A\tilde{x}$, con $\tilde{x} = x + \delta x$. In generale, se sottraiamo dal resto residuo l'equazione $Ax = b$ si ottiene

$$\begin{aligned} A(x - \tilde{x}) &= r \\ x - \tilde{x} &= A^{-1}r \\ \|x - \tilde{x}\| &\leq \|A^{-1}\| \cdot \|r\|, \quad \|x\| = \frac{\|b\|}{\|A\|} \\ \frac{\|x - \tilde{x}\|}{\|x\|} &\leq \frac{\|A\| \cdot \|A^{-1}\| \cdot \|r\|}{\|b\|} = \mu(A) \cdot \frac{\|r\|}{\|b\|} \end{aligned}$$

L'equazione mostra che la dipendenza dell'errore finale da $\mu(A)$ è un fatto generale e mette in guardia dal ritenere buona un'approssimazione di x quando il corrispondente residuo sia "piccolo". In generale non si conosce la matrice inversa di A e quindi $\mu(A)$; tuttavia l'equazione introdotta può essere effettivamente utilizzata ricorrendo ad appositi procedimenti per il calcolo approssimato di $\mu(A)$. Vale inoltre che:

$$\mu(A) \approx 1 \quad \|r\| \text{ e } \frac{\|x - \tilde{x}\|}{\|x\|} \text{ comparabili} \quad (3)$$

$$\mu(A) \gg 1 \quad \frac{\|x - \tilde{x}\|}{\|x\| \cdot \|r\|} \approx \mu(A) \quad (4)$$

5 Metodi iterativi

I metodi iterativi sono una delle altre possibilità che abbiamo introdotto per la risoluzione dei sistemi lineari. Come anche già accennato all'inizio del capitolo, questa tipologia di metodi non restituisce una **soluzione esatta per il sistema**, ma una successione $\{x_k\}_{k \in \mathbb{N}}$ tale per cui

$$\lim_{k \rightarrow \infty} x_k = x$$

dove x rappresenta la soluzione esatta per il sistema. Il motivo per cui si sceglie di adottare questa tipologia di algoritmi è l'eccessivo costo computazionale dei metodi esatti, che è circa $O(n^3)$. Il primo metodo che prendiamo in esame è il **metodo di punto fisso**; l'idea è quella di partire dal sistema $Ax - b = 0$, riscrivendo un'equazione equivalente della forma

$$x^{(k+1)} = Hx^{(k)} + c, \quad H \in \mathbb{C}^{n \cdot n}, \quad c \in \mathbb{C}^n$$

Questa funzione coincide con la successione $\{x_k\}_{k \in \mathbb{N}}$ che avevamo introdotto all'inizio della sezione. Per ricavare la successione occorre considerare la decomposizione di A nella forma $A = A_1 + A_2$,

dove A_1 è una matrice **non singolare**. Andando a sostituire nell'equazione $Ax - b = 0$ risulta che

$$\begin{aligned} Ax &= b \\ (A_1 + A_2)x &= b \\ A_1x + A_2x &= b \\ A_1x &= b - A_2x \\ x &= \underbrace{-A_1^{-1}A_2x}_c + \underbrace{A_1^{-1}b}_c \end{aligned}$$

La relazione appena introdotta individua un *metodo iterativo* in cui, partendo da un vettore iniziale $x^{(0)}$, la soluzione viene approssimata utilizzando una successione $\{x_k\}$ di vettori. La matrice H viene detta *matrice di iterazione del metodo*. La convergenza del metodo è garantita dal seguente teorema

Theorem 5.1. Il metodo iterativo $x^{(k+1)} = Hx^{(k)} + c$ converge $\forall x^{(0)} \in \mathbb{C}^n$ se e solo se il raggio spettrale della matrice di iterazione H è strettamente minore di 1:

$$\rho(H) < 1 \iff \text{Convergenza del metodo}$$

Dimostrazione. Sia $\bar{x}$ la soluzione esatta del sistema, la quale soddisfa per definizione la relazione di punto fisso $\bar{x} = H\bar{x} + c$. Sottraendo membro a membro questa equazione con l'equazione del metodo iterativo, otteniamo:

$$\begin{aligned} \bar{x} - x^{(k+1)} &= H\bar{x} + c - (Hx^{(k)} + c) \\ \bar{x} - x^{(k+1)} &= H(\bar{x} - x^{(k)}) \end{aligned}$$

Definendo il vettore errore al passo k come $e^{(k)} = \bar{x} - x^{(k)}$, possiamo riscrivere la relazione come:

$$e^{(k+1)} = He^{(k)}, \quad k = 0, 1, 2, \dots$$

Applicando questa relazione per ricorrenza, ricaviamo l'errore al passo k -esimo in funzione dell'errore iniziale:

$$e^{(k)} = He^{(k-1)} = H^2e^{(k-2)} = \dots = H^ke^{(0)}$$

Affinché il metodo converga per ogni scelta del vettore iniziale $x^{(0)}$, l'errore deve tendere al vettore nullo al crescere di k . Si ha quindi:

$$\lim_{k \rightarrow \infty} e^{(k)} = \lim_{k \rightarrow \infty} (H^ke^{(0)}) = \left(\lim_{k \rightarrow \infty} H^k \right) e^{(0)} = 0$$

Per un noto teorema sulle matrici, la successione di matrici H^k converge alla matrice nulla ($\lim_{k \rightarrow \infty} H^k = 0$) se e solo se $\rho(H) < 1$. Questo conclude la dimostrazione. $\square$

A partire da questo teorema, è possibile fare due osservazioni fondamentali che potrebbero tornare molto utili per semplificarsi la vita in sede d'esame. La prima osservazione è diretta conseguenza del teorema di **Hirsh**, infatti se $\rho(H) \leq \|H\|$ e $\|H\| < 1$ per una qualsiasi norma, allora il metodo è sicuramente convergente. La seconda osservazione deriva invece dalla definizione di **determinante**; se infatti definiamo come determinante, il **prodotto degli autovalori**, allora se $|\det(H)| > 1$, vorrà dire che esiste almeno un autovalore che è maggiore di uno in modulo e pertanto anche il raggio spettrale della matrice sarà maggiore di uno, con conseguente impossibilità di convergenza.

Dalla dimostrazione del teorema di convergenza del metodo abbiamo ricavato un'interessante identità algebrica: $\|e^{(k)}\| = \|H^ke^{(0)}\|$. Andando ad applicare la disuguaglianza triangolare risulta che $\|e^{(k)}\| \leq \|H^k\| \cdot \|e^{(0)}\|$ da cui otteniamo che

$$\frac{\|e^{(k)}\|}{\|e^{(0)}\|} \leq \|H^k\|$$

Per $k \gg 1$ il modulo di $\|H^k\|$ tende ad avvicinarsi a $\rho(H)$. Questo vuol dire che prese due matrici di iterazioni H_1 e H_2 tali per cui $0 < \rho(H_1) < \rho(H_2) < 1$, possiamo dire che il primo metodo

converge in meno passi del secondo metodo. È possibile anche stimare il numero di operazioni necessarie affinché il metodo converga con un errore relativo di approssimazione minore di un certo δ , è sufficiente imporre che

$$\rho(H^k) \leq \delta, \quad k \geq \left\lceil \frac{\ln(\delta)}{\ln(\rho(H))} \right\rceil$$

Tuttavia, poiché il calcolo del raggio spettrale di una matrice è un'operazione alquanto complessa (è necessario trovare tutti gli autovalori), è più pratico controllare il verificarsi di una di queste due condizioni e terminare l'algoritmo quando viene verificata

$$\|x^{(k+1)} - x^{(k)}\| \leq \bar{\delta}$$

5.1 Esempi di metodi iterativi

I due esempi di metodi iterativi che studieremo all'interno di questo corso sono i metodi di **Jacobi** e **Gauss-Seidel**. L'idea alla base di entrambi questi metodi è quella di prendere una matrice $A \in \mathbb{C}^{n \times n}$ e di scomporla in tre sotto matrici D , E e F tali per cui

$$A = D - E - F$$

dove A è una **matrice diagonale**, B è una **matrice triangolare inferiore** e F una **matrice triangolare superiore** (all'interno di E e F troviamo gli elementi di A cambiati di segno). La differenza tra i due metodi sta nel modo in cui vengono gestite le matrici. Nel metodo di Jacobi abbiamo:

$$\begin{aligned} Ax &= b \\ (D - E - F)x &= b \\ Dx &= (E + F)x + b \\ x &= D^{-1}(E + F)x + D^{-1}b \end{aligned}$$

Da cui si ricava che la successione $\{x_k\}_{k \in \mathbb{N}}$ corrisponde a $x^{(k+1)} = D^{-1}(E + F)x^{(k)} + D^{-1}b$. Ovviamente, affinché il metodo sia possibile, la matrice D deve essere **non singolare**. Possiamo anche facilmente osservare come il costo computazionale di questo metodo sia minore rispetto al costo computazionale di un metodo esatto, come potrebbe essere il metodo di Gauss; ad ogni iterazione dobbiamo infatti calcolare un prodotto vettore per vettore ($O(n^2)$) e le soluzioni di un sistema diagonale ($O(n)$), ottenendo quindi un costo computazionale di $O(n^2)$. La matrice di iterazione H_J per il metodo di Jacobi è così definita

$$H_J = D^{-1}(E + F) = - \begin{bmatrix} a_{11}^{-1} & 0 & \dots & 0 \\ 0 & a_{22}^{-1} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn}^{-1} \end{bmatrix} \cdot \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ a_{21} & 0 & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & 0 \end{bmatrix} \rightarrow h_{ij} \begin{cases} -\frac{a_{kl}}{a_{kk}} & i \neq j \\ 0 & i = j \end{cases}$$

La componente x_i al passo $k + 1$ è calcolata isolando l'elemento diagonale a_{ii} dalla riga i -esima del sistema $Ax = b$. Si prende il termine noto b_i e si sottrae la **somma ponderata** di tutti gli elementi non diagonali a_{ij} (per $j \neq i$) della stessa riga, moltiplicati per i **corrispondenti valori dell'incognita al passo precedente** $x_j^{(k)}$. L'intera espressione risultante è poi divisa per l'elemento diagonale a_{ii} :

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right)$$

Notiamo immediatamente che utilizzando Jacobi non è possibile sovrascrivere alcuna componente del vettore $x^{(k)}$ finché non sono state calcolate tutte le componenti del punto $x^{(k+1)}$. Per quanto riguarda Gauss-Seidel invece occorre attuare una manipolazione dell'espressione iniziale leggermente

diversa

$$\begin{aligned}
Ax &= b \\
(D - E - F)x &= b \\
(D - E)x - Fx &= b \\
(D - E)x &= Fx + b \\
x &= \underbrace{(D - E)^{-1}F}_H x + \underbrace{(D - E)^{-1}b}_c
\end{aligned}$$

Il problema principale di questo metodo è calcolare l'inversa della matrice $(D - E)$, che è un'operazione il cui costo computazionale è molto oneroso. Tuttavia, possiamo sfruttare un modo alternativo per calcolare l'inversa di $(D - E)$, risolviamo il sistema lineare $(D - E)y = F$. Il vettore z sarà il risultato desiderato. Inoltre, possiamo anche fare un'ulteriore ottimizzazione del calcolo di $x^{(k+1)}$ che rimuove la problematica della memorizzazione di tutte le componenti del vettore $x^{(k)}$

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Utilizziamo le componenti calcolate fino al passo $(i - 1)$ di $x^{(k+1)}$, combinandole con le componenti di $x^{(k)}$ mancanti. A differenza del metodo di Jacobi quindi è possibile sovrascrivere $x^{(k)}$, ma è comunque necessario che $a_{ii} \neq 0$, $\forall i = 1, \dots, n$ (la matrice sia non singolare). Un teorema che garantisce la convergenza del metodo senza necessariamente andare a calcolare il raggio spettrale della matrice di iterazione è il seguente:

Theorem 5.2. Nel caso di matrici a predominanza diagonale forte, oppure di matrici irriducibili a predominanza diagonale debole, il raggio spettrale della matrice di iterazione è strettamente minore di 1 e, quindi, l'algoritmo converge.

Dimostrazione. Possiamo suddividere la dimostrazione in due parti, analizzando le due casistiche per il metodo di **Jacobi**. L'idea alla base della convergenza è dimostrare che il raggio spettrale della matrice di iterazione H_J sia strettamente minore di uno. La matrice di iterazione di Jacobi ha elementi nulli sulla diagonale principale e gli elementi fuori diagonale pari a $-a_{ij}/a_{ii}$. Calcoliamone la norma infinito:

$$\|H_J\|_\infty = \max_{1 \leq i \leq n} \sum_{\substack{j=1 \\ j \neq i}}^n \frac{|a_{ij}|}{|a_{ii}|} = \max_{1 \leq i \leq n} \frac{1}{|a_{ii}|} \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$$

Nel caso di matrici a predominanza diagonale **forte**, la dimostrazione è immediata: per definizione, in ogni riga la somma dei moduli degli elementi fuori diagonale è strettamente minore del modulo del singolo elemento sulla diagonale ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$). Di conseguenza, il rapporto è sempre strettamente minore di uno per ogni i . La norma infinito della matrice H_J sarà quindi minore di uno e, per le proprietà delle norme matriciali (o per il **Teorema di Hirsch**), anche il raggio spettrale lo sarà, garantendo la convergenza del metodo:

$$\rho(H_J) \leq \|H_J\|_\infty < 1$$

Consideriamo ora il caso delle matrici a predominanza diagonale **debole** e contemporaneamente **irriducibili**. Per la prima proprietà, sappiamo che $\|H_J\|_\infty \leq 1$. Dobbiamo escludere che un autovalore λ possa avere modulo $|\lambda| = 1$, e lo facciamo sfruttando il **Terzo Teorema di Gershgorin**. Per il metodo di **Jacobi**, tutti i cerchi di Gershgorin della matrice di iterazione sono centrati nell'origine e hanno raggio $R_i \leq 1$. Affinché un autovalore abbia modulo 1, trovandosi sul bordo dell'unione dei cerchi, per il Terzo Teorema di Gershgorin (grazie all'ipotesi di irriducibilità) esso dovrebbe trovarsi sul bordo di *tutti* i cerchi simultaneamente. Tuttavia, la definizione di predominanza diagonale debole implica l'esistenza di almeno una riga k per cui vale la disuguaglianza stretta; il corrispondente cerchio di Gershgorin avrà quindi raggio $R_k < 1$. Il suo bordo non intersecherà mai la circonferenza unitaria. Pertanto, è impossibile che $|\lambda| = 1$ appartenga al bordo di tutti i cerchi. Anche in questo caso risulta quindi $\rho(H_J) < 1$, il che conclude la prima parte della dimostrazione.

Per dimostrare la convergenza anche nel caso del metodo di Gauss-Seidel non è necessario suddividere la dimostrazione in due diverse parti. Supponiamo, per assurdo, che esista una particolare autocoppia (λ, x) tale per cui $|\lambda| \geq 1$; per definizione stessa di autocoppia, allora deve valere necessariamente che

$$\begin{aligned}\det(\lambda I - H_{GS}) &= 0 \\ \det(\lambda I - (D - E)^{-1}F) &= 0 \\ \det([D - E]^{-1}((D - E) - \lambda F)) &= 0 \\ \lambda^n \det((D - E)^{-1}) \det((D - E) - \lambda F) &= 0\end{aligned}$$

Siccome i primi due termini del prodotto sono necessariamente diversi da zero, allora l'unico termine che potrebbe annullarsi è proprio il termine $\det((D - E) - \lambda F)$. Tuttavia, se si analizza la struttura della matrice in un caso molto semplice, si nota immediatamente una cosa, la matrice ottenuta ha tutti gli elementi sopra la diagonale divisi per uno scalare maggiore di uno in modulo, quindi questa matrice conserva le proprietà di predominanza diagonale della matrice di partenza. Dai teoremi di Gershgorin sappiamo che matrici a predominanza diagonale forte oppure per matrici a predominanza diagonale debole, ma irrudicibili, sono tutte invertibili e quindi hanno determinante sicuramente diverso da zero. Si forma quindi un assurdo, in quanto non è possibile che esista un autovalore maggiore di uno per cui il termine $\det((D - E) - \lambda F)$ si annulla e, pertanto, possiamo concludere che tutti gli autovalori sono minori di uno e che il metodo converge. $\square$

Un teorema, di cui daremo l'enunciato senza alcuna dimostrazione, fornisce una relazione particolare tra raggio spettrale dei due metodi nel caso in cui la matrice A originale sia **tridiagonale**:

Theorem 5.3. Se A è una matrice tri-diagonale, ovvero $a_{ij} \neq 0, \forall (i, j) : |i - j| > 1$, allora vale la seguente relazione

$$\rho(H_{gs}) = \rho(H_j)^2$$

CAPITOLO 5

SISTEMI RETTANGOLARI

Un caso interessante di studio dei sistemi generali è il caso dei **sistemi rettangolari**, quindi quei sistemi lineari nella forma $Ax = b$ tali per cui $A \in \mathbb{C}^{m \times n}$, con $m \neq n$ (detti anche **sistemi rettangolari**). Diversamente dai sistemi quadrati, dove $m = n$, nei sistemi rettangolari dobbiamo fare un'ulteriore classificazione

- Sistemi **sotto-determinati**: sistemi dove il numero di equazioni è minore del numero di incognite.
- Sistemi **sovra-determinati**: sistemi dove il numero di equazioni è maggiore del numero di incognite.

Durante il corso ci limiteremo alla sola trattazione dei sistemi **sovra-determinati**, poiché più facile rispetto a quella dell'altra tipologia. In generale, anche per la risoluzione dei **sistemi rettangolari** è possibile sfruttare l'algoritmo di Gauss, che dopo n iterazioni porterà a una situazione di questo tipo

$$\begin{bmatrix} \star & \star & \dots & \star & \star \\ 0 & \star & \dots & \star & \star \\ 0 & 0 & \dots & \star & \star \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & \star \\ 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \\ d_1 \\ \vdots \\ d_{m-n} \end{bmatrix}$$

Uno delle condizioni necessarie affinché il sistema ammetta soluzione è che $d_i = 0$, $\forall i = n+1, \dots, m$, poiché se esistesse una $d_i \neq 0$ si avrebbe una situazione del tipo $0 = d_i \neq 0$ e quindi il sistema non ammetterebbe soluzione. Se tale condizione viene rispettata, è possibile limitarsi alla sola risoluzione del sistema quadrato composto dalle sole prime n righe della matrice A del sistema. Tuttavia, capita spesso che si verifichi la seguente situazione

$$\text{Rk}(A|b) > \text{Rk}(A)$$

che dal teorema di Rouché-Capelli garantisce la **non esistenza di soluzioni**.

1 Problema ai minimi quadrati

Nella maggior parte dei casi, per effetto del Teorema di Rouché-Capelli, i sistemi lineari **sovra-determinati** non ammettono **soluzione**. In termini puramente pratici e applicativi, questo fatto rappresenta un problema non trascurabile. Ci poniamo dunque la seguente domanda: se non

esiste una soluzione **esatta** al sistema lineare, è comunque ammissibile cercare di determinare una soluzione **approssimata** che **renda minimo il residuo**? La risposta è affermativa: possiamo cercare un vettore $\hat{x}$ tale per cui il residuo, definito come:

$$r = A\hat{x} - b$$

sia **minimo**. Se ammettiamo quindi la possibilità che la soluzione non sia esatta, il paradigma stesso del problema cambia: si passa infatti dal cercare l'unica soluzione del sistema lineare, a determinare una tra le infinite soluzioni possibili tale per cui il residuo sia il più piccolo possibile. In termini puramente matematici, possiamo descrivere questo nuovo approccio come un **problema di ottimizzazione** dove si cerca di determinare il minimo della norma del residuo (che ricordiamo essere un vettore):

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2, \quad \|b - Ax\|_2 = \sqrt{\sum_{i=1}^m r_i(x)^2}$$

ove il termine $r_i(x)$ è il residuo relativo alla componente i -esima della differenza $b - A\hat{x}$. La presenza della radice quadrata rappresenta però un ostacolo: gestire funzioni che presentano termini non lineari è relativamente difficile da un punto di vista analitico, specie nel caso in cui si debba lavorare con le derivate. Per aggirare questo ostacolo è possibile usare un trucco matematico: per definizione, la funzione **radice quadrata** è una funzione monotona **crescente** che non altera la posizione dei punti di minimo della funzione radicanda. Pertanto, il problema di ottimizzazione può essere formulato in un modo alternativo, molto più semplice da studiare:

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2^2, \quad \|b - Ax\|_2^2 = \|r(x)\|_2^2 = \sum_{i=1}^m r_i(x)^2 = \gamma(x)$$

La nostra **funzione obiettivo** corrisponde ora al quadrato della norma del residuo e può essere riscritta matricialmente come $\gamma(x) = (Ax - b)^T(Ax - b)$. Oltre ad avere una struttura algebrica che la rende particolarmente facile da trattare, la funzione $\gamma(x)$ gode di un'altra proprietà fondamentale: è **convessa**. Una funzione convessa e derivabile possiede un unico punto di minimo globale, localizzato esattamente dove il gradiente della funzione si annulla:

$$\nabla \gamma(x^*) = \nabla (\|r(x)\|_2^2) = 0$$

Possiamo sviluppare la derivata della funzione in due modi diversi: il primo sfruttando la sua forma matriciale compatta $\gamma(x) = (Ax - b)^T(Ax - b)$, e il secondo andando a calcolare le derivate parziali per i singoli residui al quadrato $r_i(x)^2$ e sommandole. Ipotizziamo di sfruttare il secondo metodo calcolando la derivata del quadrato del residuo i -esimo rispetto alla generica componente x_s :

$$\frac{\partial r_i^2}{\partial x_s}(x) = 2 \left[b_i - \sum_{j=1}^n a_{ij} \cdot x_j \right] (-a_{is})$$

Sviluppando il calcolo e sommando su tutte le componenti i , otteniamo la derivata parziale di $\gamma(x)$ rispetto a x_s :

$$(\nabla \gamma(x))_s = \frac{\partial \gamma(x)}{\partial x_s} = \sum_{i=1}^m \frac{\partial r_i(x)^2}{\partial x_s} = 2 \left[\sum_{i=1}^m a_{is} \sum_{j=1}^n a_{ij} x_j - \sum_{i=1}^m b_i a_{is} \right]$$

Osservando attentamente, notiamo che il primo termine corrisponde, nella struttura, alla componente s -esima del prodotto matriciale $A^T Ax$, ovvero $(A^T Ax)_s$; il secondo termine corrisponde invece alla componente s -esima del prodotto matriciale $A^T b$, ovvero $(A^T b)_s$. La derivata di $\gamma(x)$ rispetto alla componente x_s è quindi pari a $2(A^T Ax - A^T b)_s$. Estendendo il risultato a tutte le componenti per formare il vettore gradiente, si ottiene:

$$\nabla \gamma(x) = \boxed{2(A^T Ax - A^T b)}, \quad \nabla \gamma(x) = 0 \iff (A^T Ax - A^T b) = 0 \rightarrow \boxed{A^T Ax = A^T b}$$

Quest'ultimo è un sistema lineare $n \times n$, noto come **sistema delle equazioni normali**. Le soluzioni di questo sistema sono tutti e soli i punti di minimo della funzione $\gamma(x)$. Esiste un teorema che garantisce l'esistenza della soluzione $\hat{x}$ del sistema indipendentemente dalla scelta del vettore b :

Theorem 1.1. Se $A \in \mathbb{C}^{m \times n}$, con $m \geq n$, allora il sistema delle equazioni normali $A^H A x = A^H b$ ammette soluzione $\forall b \in \mathbb{C}^m$ e la sua soluzione è unica se e solo se:

$$\text{Rk}(A) = n$$

Dimostrazione. Per dimostrare che esiste effettivamente una soluzione al sistema delle equazioni normali, possiamo utilizzare due condizioni equivalenti: si può infatti dimostrare che $A^H b \in \text{Im}(A^H A)$, oppure, in modo del tutto analogo, che $\text{Rk}(A^H A) = \text{Rk}(A^H A | A^H b)$. Consideriamo il primo metodo; affinché $A^H b$ appartenga all'immagine di $A^H A$ è necessario che esista un particolare vettore x tale per cui $A^H A x = A^H b$. L'idea della dimostrazione è quindi quella di trovare un modo per scrivere il vettore $A^H b$ in funzione di un particolare vettore z moltiplicato a sinistra per la matrice $A^H A$. La prima considerazione che possiamo fare è che, dal Teorema Fondamentale dell'Algebra Lineare, lo spazio vettoriale $\mathbb{C}^m$ può essere scomposto in somma diretta ortogonale come:

$$\mathbb{C}^m = \text{Im}(A) \oplus^\perp \ker(A^H)$$

Siccome il nostro vettore b appartiene a tale spazio vettoriale complesso, possiamo scomporlo in due componenti: $b = b_1 + b_2$, dove $b_1 \in \text{Im}(A)$ e $b_2 \in \ker(A^H)$. Sostituendo questa relazione nell'espressione di $A^H b$ otteniamo:

$$A^H b = A^H (b_1 + b_2) = A^H b_1 + \cancel{A^H b_2} = A^H b_1$$

Infatti, se $b_2 \in \ker(A^H)$, per definizione stessa di nucleo il suo prodotto con A^H dovrà necessariamente essere nullo. Poiché $b_1 \in \text{Im}(A)$, allora per definizione di immagine esiste un particolare vettore z tale per cui $Az = b_1$. Sostituendo otteniamo:

$$A^H b = A^H b_1 = A^H A z$$

Essendo riusciti a esprimere il vettore $A^H b$ come il prodotto della matrice $A^H A$ per un vettore z , possiamo concludere che $A^H b \in \text{Im}(A^H A)$, il che garantisce che la soluzione al sistema delle equazioni normali esista sempre. La dimostrazione dell'unicità della soluzione richiede invece di verificare che la matrice dei coefficienti $A^H A$ sia invertibile, ovvero che il suo determinante sia diverso da zero. L'idea per giungere a tale conclusione è quella di applicare il **Teorema di Binet-Cauchy**, il quale postula che il determinante del prodotto di due matrici rettangolari può essere scritto come la somma dei prodotti dei determinanti di tutti i minori di ordine n delle rispettive matrici:

$$\det(A^H A) = \sum_{[J]} \det(A_{[J]}^H) \cdot \det(A_{[J]})$$

Poiché la matrice hermitiana è la trasposta coniugata di A , il determinante del minore associato alla scelta di indici $[J]$ della matrice A^H è uguale al coniugato del determinante del corrispondente minore della matrice A . Sostituendo si ottiene:

$$\det(A^H A) = \sum_{[J]} \overline{\det(A_{[J]})} \cdot \det(A_{[J]}) = \sum_{[J]} |\det(A_{[J]})|^2$$

Se applichiamo l'ipotesi di rango massimo ($\text{Rk}(A) = n$), esisterà sicuramente almeno un minore di ordine n con determinante diverso da zero. Di conseguenza, la somma di moduli al quadrato conterrà almeno un termine strettamente positivo, il che implica:

$$\det(A^H A) > 0$$

Pertanto, il determinante di $A^H A$ è sicuramente diverso da zero, il che garantisce l'unicità della soluzione. $\square$

Risolvere direttamente il sistema delle equazioni normali presenta però due criticità principali: un costo computazionale elevato, che si attesta intorno a $O(m \cdot n^2)$, e un numero di condizionamento della matrice $\mu(A^H A)$ molto alto (che nel caso particolare $m = n$ corrisponde a circa $\mu(A)^2$). A causa di quest'ultimo fattore, la risoluzione diretta dà spesso origine a un problema **malcondizionato**. Nella pratica, si preferisce quindi utilizzare altri metodi numericamente più stabili per risolvere il problema ai minimi quadrati, come la fattorizzazione **QR**.

2 Fattorizzazione QR

Il metodo delle equazioni normali presenta un problema fondamentale e di cui si deve tenere estremamente conto quando si deve risolvere un sistema sovra-determinato: il numero di condizionamento della matrice $A^T A$ è estremamente alto, in particolare, possiamo (senza dovizia di particolari) concludere che si attesta intorno al numero di condizionamento della matrice A al quadrato. Un valore di condizionamento così alto può portare a due problemi principali: il primo è la **perdita di precisione**, infatti si perde una cifra di precisione per ogni potenza di 10 del numero di condizionamento e, inoltre, con matrici **malcondizionate** vicine al limite di precisione di macchina possono apparire numericamente singolari (cioè con determinante pari a zero), impedendo del tutto la risoluzione del sistema $A^T A = A^T b$. L'idea per aggirare questo problema è quello di sostituire la matrice A , che potrebbe avere un numero di condizionamento molto alto, con una **fattorizzazione**, cioè un prodotto di due matrici Q ed R con delle proprietà molto particolari.

Definition 2.1. Sia $A \in \mathbb{C}^{m \times n}$, definiamo una **fattorizzazione QR** come una coppia di matrici (Q, R) tale per cui

$$A = Q \cdot R$$

con $Q \in \mathbb{C}^{m \times m}$ unitaria e $R \in \mathbb{R}^{m \times n}$ congrua alla seguente struttura

$$R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$$

con R_1 matrice triangolare quadrata.

La matrice Q è una matrice **unitaria**; questa tipologia di matrici rispetta una proprietà fondamentale che ci tornerà utile durante questo capitolo: è **invariante per norme**, cioè $\|QAQ^T\| = \|A\|$, per ogni norma. Durante il capitolo scorso, abbiamo visto un diverso tipo di fattorizzazione, cioè quella **LU**; anche se rispetto alla fattorizzazione **QR** troviamo diverse differenze. La prima differenza è che la fattorizzazione **LU** potrebbe non esistere, mentre la fattorizzazione **QR** esiste sempre (esiste un teorema che ne garantisce l'esistenza), indipendentemente dalla matrice A scelta. D'altra parte, sia la fattorizzazione **LU** che la fattorizzazione **QR** non sono uniche; infatti per una stessa matrice A possono esistere diverse fattorizzazioni possibili tali per cui $A = QR$. Questo tipo di fattorizzazione risulta estremamente utile nel calcolo della soluzione ottima al problema ai minimi quadrati, ipotizziamo infatti di sostituire la matrice A con la sua fattorizzazione **QR**:

$$\|Ax - b\|_2 = \|QRx - b\|_2 = \|Q(Rx - Q^H b)\|_2 = \|Rx - Q^H b\|_2$$

Poniamo $Q^H b = c$ e riscriviamo il problema sostituendo la matrice R con la sua decomposizione in blocchi (che ricordiamo essere composta da una matrice triangolare $R_1 \in \mathbb{C}^{n \times n}$ e una matrice di soli 0):

$$\left\| \begin{bmatrix} R_1 \\ 0 \end{bmatrix} x - \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} \right\| \rightarrow \|R_1 x - c_1\| + \|c_2\|$$

Considerando quindi la fattorizzazione **QR** della matrice e la successiva decomposizione della matrice R nelle sue due componenti, possiamo riscrivere il problema ai minimi quadrati come:

$$\min_{x \in \mathbb{C}^n} \|c_1 - R_1 x\|_2^2 + \|c_2\|_2^2$$

Il fattore $\|c_2\|_2^2$ è un termine che non dipende da x e quindi non deve essere tenuto in considerazione quando si cerca di ricavare la soluzione ottima $\hat{x}$. Tuttavia, è altresì vero che il termine $\|c_2\|_2^2$ rappresenta proprio il valore ottimo della soluzione al problema ai minimi quadrati, infatti per $\hat{x}$ soluzione ottima, il termine $\|c_1 - R_1 x\|_2^2$ si annulla e l'unico termine rimanente è proprio $\|c_2\|_2^2$.

2.1 Ricavare la fattorizzazione QR

Data una fattorizzazione QR, diventa quindi facile risolvere un sistema rettangolare approssimando la soluzione attraverso il problema dei minimi quadrati. Non è tuttavia così facile riuscire a determinare, data una matrice $A \in \mathbb{C}^{m \times n}$ la fattorizzazione QR corrispettiva. L'idea generale

alla base dell'**algoritmo** che permette di ricavare la fattorizzazione QR di una matrice è quello di **moltiplicare** la matrice A per particolari **matrici unitarie** H_j . In particolare, queste matrici hanno come obiettivo quello di aggiustare la **j-esima** colonna, andando a annullare tutti gli elementi che si trovano sotto all'elemento diagonale a_{jj} . Ovviamente, affinché un algoritmo di questo tipo funzioni è necessario che esista una particolare trasformazione del vettore colonna di A in grado di eliminare tutti gli elementi sotto-diagonali; a tal proposito definiamo un particolare tipo di vettori:

Lemma 2.1. Dato un vettore $x \in \mathbb{C}^n$ esiste sempre un vettore v tale per cui la matrice unitaria H_v moltiplicata per il vettore x è un vettore con tutti gli elementi nulli tranne il primo:

$$H_v \cdot x = \alpha \cdot e_1 = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

La scelta del vettore v può essere fatta in modo semplice andando a osservare il vettore x di cui vogliamo annullare gli elementi; in particolare, ipotizziamo x il vettore di cui vogliamo annullare gli elementi possiamo utilizzare la scelta: $v = x \pm \|x\|_2 \cdot e_1$.

Inoltre, poiché moltiplicare per matrici unitarie non cambia la norma $\|\cdot\|_2$ del vettore anche una volta moltiplicato per la matrice H_v allora è possibile affermare che $|\alpha| = \|x\|_2^2$. La scelta di queste matrici non è unica, esistono diverse possibilità; prendiamo in esame una particolare classi di matrici unitarie dette **matrici di householder**:

Definition 2.2. (Matrice di Householder) Dato un vettore $v \in \mathbb{C}^n$ definiamo la matrice di Householder H_v associata a v come:

$$H_v = I - 2 \frac{v \cdot v^H}{\|v\|_2^2}$$

Per calcolare la fattorizzazione QR attraverso le matrici di Householder andando a moltiplicare la matrice originale per una particolare matrice

$$H_k = \left[\begin{array}{c|c} I_{k-1} & 0 \\ \hline 0 & \tilde{H}_k \end{array} \right]$$

tale per cui $\tilde{H}_k$ è la matrice di householder relativa alla colonna k -esima della matrice al passo (k) , per tutte le colonne della matrice fintanto che non si arriva ad avere una matrice R come descritta in precedenza, con il primo blocco R_1 **triangolare superiore**. Ad esempio, al primo passo la matrice H_1 sarà costruita come

$$\tilde{H}_1 \cdot A = \tilde{H}_1 \left[\begin{array}{c|c|c} a_1 & \dots & a_n \end{array} \right] = \left[\begin{array}{c|c|c} \star & & \\ \vdots & a_2 & \dots & a_n \\ 0 & & \end{array} \right]$$

Successivamente calcolo la matrice H_2 e applico il medesimo ragionamento. A differenza della matrice H_1 però, la matrice H_2 dovrà essere strutturata in modo che la prima riga non venga alterata (in quanto già "aggiustata" dalla matrice H_1 al passo precedente. Quindi, moltiplico a sinistra per

$$H_2 = \left[\begin{array}{c|c} 1 & 0 \\ \hline 0 & \tilde{H}_2 \end{array} \right]$$

Con $\tilde{H}_2 \in \mathbb{C}^{(m-1) \times (m-1)}$ tale per cui la matrice aggiusta gli ultimi $m-1$ elementi della colonna 2 della matrice A . Quindi, al passo k dell'algoritmo la matrice avrà una forma del tipo. Al termine dell'algoritmo otteniamo la matrice R nella forma già descritta in precedenza, con R_1 triangolare **superiore**. Invece, per quanto riguarda la matrice Q dobbiamo andare a fare il prodotto di tutte le matrici di Householder trasposte

$$H_1 \dots H_n A = R \implies A = \underbrace{H_1^H \dots H_n^H}_Q R$$

Il vantaggio di utilizzare questa tipologia di matrici è il costo computazionale, infatti per calcolare la fattorizzazione QR di una matrice A il costo computazionale è circa $O(mn^2)$. Quindi, per risolvere il problema ai minimi quadrati utilizzando la fattorizzazione QR abbiamo un costo computazionale dato dalla somma dei seguenti termini

$$A = QR \quad O(mn^2) \quad (1)$$

$$c = Q^H b \quad O(mn) \quad (2)$$

$$R_1 x = c \quad O(n^2) \quad (3)$$

Quindi, il costo computazionale corrispondente è $O(mn^2)$. Sebbene il costo computazionale delle equazioni normali ($O(m^2n)$) sia comparabile a quello della fattorizzazione QR, **dal punto di vista numerico la fattorizzazione QR è preferibile**, in quanto mantiene il condizionamento del problema e produce soluzioni più stabili.

Parte III

Interpolazione

CAPITOLO 6

INTERPOLAZIONE POLINOMIALE

In questa parte del corso verranno trattati una nuova classe di problemi di approssimazione: i problemi di **interpolazione**. Supponiamo di avere una funzione $f : \mathbb{R}^n \rightarrow \mathbb{R}$ che non conosciamo direttamente, ma di cui conosciamo la valutazione $y_0, \dots, y_k \in \mathbb{R}$ in $k + 1$ punti $x_0, \dots, x_k \in \mathbb{R}^n$. Il problema che ci poniamo è quello di trovare una funzione $\gamma(x)$ tale per cui

$$\gamma(x_i) = y_i, \quad \forall i = 0, \dots, k$$

Tuttavia, non è sempre detto che sia possibile trovare una funzione che approssimi esattamente i vari $y_0, \dots, y_k \in \mathbb{R}$, possiamo anche accontentarci di trovare una funzione che passi sufficientemente vicino ad ogni punto (dove con *sufficientemente vicino* intendiamo una volta fissata una metrica specifica). Questo è il caso della famosissima **regressione lineare**, una tecnica estremamente utilizzata nell'ambito del machine learning, la cui idea è quella di approssimare una retta che sia quanto più vicina possibile a un set di punti che talvolta possono essere anche molto distanti tra di loro. In generale, non è possibile trovare un'approssimazione esatta in due casi:

- I dati sono affetti da rumore.
- I dati sono in grande numero.

Quando cerchiamo delle approssimazioni si cerca di costruire una funzione $f_k(x)$ che sia quanto più semplice possibile, idealmente costituita da sole funzioni elementari (semplici da gestire quindi). Quindi, dati $x_0, \dots, x_k \in \mathbb{R}^n$ e dati $y_0, \dots, y_k \in \mathbb{R}$ tali per cui $y_0 = f(x_0), \dots, y_k = f(x_k)$, definiamo il polinomio interpolante come

$$f_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_kx^k = \sum_{i=0}^k a_kx^k$$

Affinché sia possibile trovare il polinomio interpolante è necessario andare a trovare i coefficienti $a_0, \dots, a_k$. Il modo più semplice per farlo è andando a imporre la condizione $f_n(x_i) = y_i, \forall i = 0, \dots, k$

$$\begin{aligned} f_k(x_0) &= a_0 + a_1x_0 + a_2x_0^2 + \dots + a_kx_0^k = y_0 \\ f_k(x_1) &= a_0 + a_1x_1 + a_2x_1^2 + \dots + a_kx_1^k = y_1 \\ &\dots \\ f_k(x_k) &= a_0 + a_1x_k + a_2x_k^2 + \dots + a_kx_k^k = y_k \end{aligned}$$

Si nota immediatamente come sia possibile riscrivere questo insieme di equazioni come un sistema lineare nella forma $Va = y$:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^k \\ 1 & x_1 & x_1^2 & \dots & x_1^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{k-1} & x_{k-1}^2 & \dots & x_{k-1}^k \\ 1 & x_k & x_k^2 & \dots & x_k^k \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{k-1} \\ a_k \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{k-1} \\ y_k \end{bmatrix}$$

La matrice in questione è detta **matrice di Vandermode** relativa ai punti $x_0, \dots, x_k$. Il vantaggio principale di aver riportato il nostro problema di interpolazione ad un sistema lineare è la possibilità di verificare immediatamente se la soluzione esiste o meno, oltre a verificare se sia unica. Infatti, se il polinomio interpolante esiste, allora il sistema ammette una soluzione e, di conseguenza, la matrice di Vandermode è invertibile. La matrice di Vandermode, vista la sua struttura, ha il determinante sempre diverso da 0:

$$\det(V) = \prod_{k \geq j \geq i \geq 0} (x_j - x_i) \neq 0$$

in pratica è come fare il prodotto delle differenze tra nodi distinti dove il nodo a sinistra ha indice maggiore di quello a destra. Quindi, la matrice di Vandermode risulta essere sempre invertibile e, di conseguenza, possiamo assicurare l'esistenza e l'unicità del polinomio interpolante. Tuttavia la matrice di Vandermode ha un numero di condizionamento che cresce all'aumentare del numero dei nodi di interpolazione, di fatto con $k \geq 10$ il problema diventa malcondizionato.

L'idea che andiamo a sfruttare è la seguente: il condizionamento è una proprietà che dipende dalla matrice stessa, ma non è detto che non la si possa riscrivere in una forma che garantisca un numero di condizionamento migliore; è sufficiente cambiare la base nello spazio di polinomi. Infatti fino ad adesso abbiamo supposto che la base $\mathcal{B}$ sui cui abbiamo costruito i polinomi di interpolazione fosse quella definita come

$$\mathcal{B} = \{1, x, x^2, \dots, x^k\}$$

Potremmo però pensare di prendere una base diversa da quest'ultima, una base $\mathcal{B}^* = \{q_0(x), \dots, q_k(x)\}$. In questo capitolo analizzeremo due possibili scelte:

- Base di Lagrange.
- Base di Newton.

1 Base di Lagrange

La prima base che analizziamo per costruire i polinomi interpolatori è la **base di Lagrange**. È un approccio molto usato anche nell'integrazione numerica, dove serve per definire i polinomi che andranno poi integrati analiticamente per calcolare le approssimazioni. Dato un insieme di $k + 1$ nodi, i polinomi che compongono la base di Lagrange si definiscono in questo modo:

$$\mathcal{L} = \{l_0(x), \dots, l_k(x)\}, \quad \text{dove} \quad l_j(x) = \prod_{\substack{i=0 \\ i \neq j}}^k \frac{x - x_i}{x_j - x_i}$$

A prima vista la formula può sembrare un po' pesante, ma gode di una proprietà comodissima (che di fatto si comporta come un delta di Kronecker): se valutiamo il polinomio $l_j(x)$ in un nodo generico x_i , il risultato sarà esattamente 1 se $i = j$, e varrà 0 in tutti gli altri nodi. Questo ci dà un vantaggio pratico enorme quando andiamo a calcolare il polinomio interpolatore globale $p_k(x)$ usandolo come combinazione lineare di questi polinomi di base. Imponendo la condizione di interpolazione per un nodo generico x_i (ovvero vogliamo che $p_k(x_i) = y_i$), otteniamo:

$$p_k(x_i) = \sum_{j=0}^k c_j l_j(x_i) = y_i$$

Visto che $l_j(x_i)$ si azzerava per tutti i termini della sommatoria tranne quando $j = i$ (dove vale 1), l'intera somma collassa su un unico termine. Il risultato è quindi immediato:

$$c_i = y_i$$

In pratica, il grande vantaggio della base di Lagrange è che i coefficienti del polinomio sono banalmente i valori delle ordinate (y) dei nostri punti di interpolazione, evitandoci così di dover risolvere un sistema lineare per trovarli.

2 Base di Newton

La base di Lagrange, per quanto sia estremamente semplice ed elegante, nasconde un grosso difetto pratico: la mancanza di "espandibilità". Se, dopo aver calcolato il polinomio interpolatore, decidiamo di aggiungere anche un solo nuovo nodo per migliorare l'approssimazione, siamo costretti a buttare via tutto e ricalcolare da zero ogni singolo polinomio $l_j(x)$. Per ovviare a questo spreco computazionale introduciamo una nuova base: la **base di Newton**.

$$\mathcal{N} = \{n_0(x), n_1(x), \dots, n_k(x)\}, \quad n_j(x) = \prod_{i=0}^{j-1} (x - x_i)$$

Come si può notare, il polinomio $n_j(x)$ ha grado j e dipende esclusivamente dai primi j nodi $(x_0, \dots, x_{j-1})$. Questo significa che se aggiungiamo un nodo all'insieme, i vecchi polinomi della base restano validi; ci basterà calcolare solo un nuovo polinomio aggiuntivo e il suo rispettivo coefficiente. A differenza del metodo di Lagrange, però, i coefficienti c_j non sono semplicemente le y_i dei nodi, ma si calcolano attraverso un meccanismo ricorsivo. Nello specifico, i coefficienti prendono il nome di **differenze divise**:

$$c_j = f[x_0, \dots, x_j], \quad \text{per } j = 0, \dots, k$$

Le differenze divise si costruiscono in modo iterativo (ricordando i rapporti incrementali):

$$\begin{cases} f[x_i] = f(x_i) & \text{(Ordine 0)} \\ f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} & \text{(Ordine 1)} \\ f[x_i, \dots, x_{i+j}] = \frac{f[x_{i+1}, \dots, x_{i+j}] - f[x_i, \dots, x_{i+j-1}]}{x_{i+j} - x_i} & \text{(Ordine generico } j) \end{cases}$$

Queste quantità godono di tre proprietà teoriche fondamentali:

- **Simmetria:** Il valore della differenza divisa non cambia se permutiamo l'ordine dei nodi. Ad esempio, $f[x_0, x_1, x_2] = f[x_1, x_0, x_2]$.
- **Legame con le derivate:** Se la funzione originale f è derivabile k volte in un intervallo I che contiene tutti i nodi, allora esiste un punto $\xi \in I$ tale che $f[x_0, \dots, x_k] = f^{(k)}(\xi)/k!$. Questa è una diretta conseguenza del Teorema del Valor Medio e ci dice che le differenze divise sono essenzialmente delle approssimazioni numeriche delle derivate.
- **Estensione per coincidenza (limite):** Se due o più nodi tendono a coincidere (ad esempio $x_1 \rightarrow x_0$), la differenza divisa del primo ordine tende esattamente al valore della derivata prima nel nodo: $f[x_0, x_0] = f'(x_0)$. (Questo concetto è alla base dell'interpolazione di Hermite).

All'apparenza, calcolare a mano questi indici annidati sembra un incubo. In pratica, però, il processo si riduce alla compilazione di una semplice tabella triangolare, dove ogni elemento si calcola a partire dai due che ha immediatamente a sinistra:

x_i	Ord. 0 (f)	Ord. 1 (DD_1)	Ord. 2 (DD_2)	Ord. 3 (DD_3)
x_0	$f(x_0)$			
x_1	$f(x_1)$	$f[x_0, x_1]$		
x_2	$f(x_2)$	$f[x_0, x_2]$	$f[x_0, x_1, x_2]$	
x_3	$f(x_3)$	$f[x_0, x_3]$	$f[x_0, x_1, x_3]$	$f[x_0, x_1, x_2, x_3]$

Traducendo la tabella in una matrice pratica da programmare, un generico elemento $a_{i,j}$ (situato alla riga i e colonna j) si calcola facendo la differenza tra l'elemento sulla sua stessa riga a sinistra e quello subito sopra, diviso per la distanza tra il nodo attuale e il nodo "più vecchio" coinvolto:

$$a_{i,j} = \frac{a_{i,j-1} - a_{i-1,j-1}}{x_i - x_{i-j}}$$

Si procede iterando per righe dall'alto verso il basso e per colonne da sinistra verso destra. I coefficienti c_j del nostro polinomio interpolatore non saranno altro che gli elementi sulla **diagonale principale** di questo triangolo (ovvero $f(x_0), f[x_0, x_1], f[x_0, x_1, x_2]$, ecc.). Per concludere, possiamo fare un paio di osservazioni utili:

- **Crollo del grado:** Se durante i calcoli ci accorgiamo che tutti gli elementi di una certa colonna j sono identici, le differenze della colonna successiva saranno inevitabilmente tutte 0. Questo significa che i coefficienti successivi si annullano e il grado effettivo del polinomio interpolatore sarà pari a j .
- **Sistema triangolare:** Se impostassimo il problema dell'interpolazione come un classico sistema lineare imponendo la condizione $p(x_i) = y_i$ sui nodi della base di Newton, noteremmo che i polinomi di base via via si annullano (perché contengono fattori $(x_i - x_i) = 0$). La matrice dei coefficienti del sistema risulterebbe quindi **triangolare inferiore**, rendendo il sistema risolvibile per semplice sostituzione in avanti.

3 Interpolazione osculatoria di Hermite

Questo tipo di interpolazione si differenzia da quelle studiate precedentemente poiché prevede, oltre alle valutazioni della funzione $f(x)$ nei nodi $x_0, \dots, x_k$, anche le valutazioni della derivata prima della funzione nei medesimi punti. In particolare, dati $k+1$ punti a due a due distinti, in corrispondenza dei quali sono noti $2k+2$ valori reali:

$$f(x_0), \dots, f(x_k) \quad \text{e} \quad f'(x_0), \dots, f'(x_k)$$

il problema dell'interpolazione secondo Hermite consiste nel determinare un unico polinomio interpolante $H_{2k+1}(x)$, di grado al più $2k+1$, tale che:

$$H_{2k+1}(x_i) = f(x_i) \quad \text{e} \quad H'_{2k+1}(x_i) = f'(x_i) \quad \forall i = 0, \dots, k$$

Preliminarmente, presentiamo il risultato fondamentale che garantisce l'esistenza e l'unicità di tale polinomio osculatorio:

Theorem 3.1. (Esistenza e unicità) Se $f \in C^1(I)$ e i nodi sono a due a due distinti ($\forall i \neq j : x_i \neq x_j$), allora esiste sempre ed è unico il polinomio osculatorio di Hermite $H_{2k+1}(x)$ per f sull'insieme di punti I .

I polinomi osculatori rappresentano una generalizzazione dei classici polinomi di interpolazione di Lagrange. Questo approccio può essere esteso anche a condizioni più stringenti, imponendo la coincidenza delle derivate fino a un ordine superiore generico m :

$$P^{(m)}(x_i) = f^{(m)}(x_i)$$

Nel caso d'interesse del corso, ovvero con vincoli limitati alla funzione e alla sua derivata prima, il polinomio cercato viene espresso come combinazione lineare degli elementi di una base opportuna:

$$H_{2k+1}(x) = \sum_{i=0}^k h_{0i}(x)f(x_i) + \sum_{i=0}^k h_{1i}(x)f'(x_i)$$

Il polinomio è dunque composto dalla somma di due contributi. Per garantire che $H_{2k+1}(x)$ soddisfi le condizioni di interpolazione, i polinomi di base $h_{0i}(x)$ e $h_{1i}(x)$ (entrambi di grado $2k+1$) devono

soddisfare le seguenti proprietà nei nodi x_j :

$$\begin{aligned} h_{0i}(x_j) &= \begin{cases} 1 & \text{se } i = j \\ 0 & \text{se } i \neq j \end{cases} & h'_{0i}(x_j) &= 0 \\ h_{1i}(x_j) &= 0 & h'_{1i}(x_j) &= \begin{cases} 1 & \text{se } i = j \\ 0 & \text{se } i \neq j \end{cases} \end{aligned}$$

La base dello spazio dei polinomi di grado $2k+1$ adatta a questo problema non è la base canonica dei monomi, bensì la base definita da:

$$\mathcal{B} = \{h_{00}(x), \dots, h_{0k}(x), h_{10}(x), \dots, h_{1k}(x)\}$$

I coefficienti della combinazione lineare sono i valori noti della funzione e delle sue derivate. Di conseguenza, il problema algoritmico si riduce alla determinazione esplicita dei polinomi di base $h_{0i}(x)$ e $h_{1i}(x)$. Sfruttando i polinomi di base di Lagrange $l_i(x)$, essi possono essere scritti nella forma:

$$\begin{aligned} h_{0i}(x) &= [1 - 2l'_i(x_i)(x - x_i)] l_i^2(x) \\ h_{1i}(x) &= (x - x_i) l_i^2(x) \end{aligned} \quad \text{con} \quad l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^k \frac{x - x_j}{x_i - x_j}$$

Infine, è possibile fornire una stima analitica dell'errore di troncamento introdotto dall'interpolazione di Hermite, sotto opportune ipotesi di regolarità della funzione:

Theorem 3.2. (Errore di interpolazione di Hermite) Sia $f \in C^{2k+2}(I)$, allora per ogni $x \in I$ vale la seguente relazione:

$$f(x) - H_{2k+1}(x) = \Omega(x) \frac{f^{(2k+2)}(\xi)}{(2k+2)!}$$

dove $\Omega(x) = \prod_{j=0}^k (x - x_j)^2$ e il punto ξ appartiene all'intervallo aperto generato dai nodi e dal punto x : $\xi \in (\min\{x_0, \dots, x_k, x\}, \max\{x_0, \dots, x_k, x\})$.

CAPITOLO 7

APPROSSIMAZIONE A TRATTI E FUNZIONI SPLINE

Fino ad ora abbiamo visto problemi di interpolazione dove la base, e quindi la funzione interpolante, sono dei **polinomi** interpolati considerando l'insieme dei punti $x_0, x_1, \dots, x_k$ come **unico intervallo** di interpolazione. Tuttavia, entrambe queste condizioni possono essere rimosse, è possibile infatti ipotizzare di suddividere l'intervallo dei punti in vari sotto-intervalli $[x_{i-1}, x_i]$ e successivamente interpolare su ognuno di essi, ottenendo una funzione $F(x)$ risultante dall'unione delle funzioni $f_i(x)$ su ogni intervallo ed è anche possibile ipotizzare di utilizzare delle basi per la funzione interpolante che non siano necessariamente polinomiali, ma anche esponenziali, logaritmiche, etc. In particolare, durante questo capitolo prenderemo in esame due diversi approcci per risolvere il problema di interpolazione:

- Interpolazione **lineare a tratti**.
- Interpolazione ai **minimi quadrati**

Come anche nel caso dell'interpolazione osculatoria di hermite, quando si approssima una funzione $f(x)$ ammettendo anche la possibilità che l'interpolazione non sia **esatta** su tutti i punti dati, è necessario e fondamentale valutare quello che è l'**errore di approssimazione**, cioè quanto ognuno dei punti dati, valutato relativamente alla funzione di interpolazione, si discosta dalla sua valutazione esatta. Senza fare alcuna dimostrazione o considerazione, possiamo enunciare un risultato che fornisce una formula esplicita per il calcolo di questo errore:

Theorem 0.1. (Errore di interpolazione) Consideriamo $k + 1$ punti distinti, sia inoltre una funzione $f : \mathbb{R} \rightarrow \mathbb{R}$ con $f \in C^{k+1}(I)$ con I intervallo che contiene i nodi. Se $p_k(x)$ è il polinomio di interpolazione di grado al più k che interpola i punti $(x_0, f(x_0)), \dots, (x_k, f(x_k))$, allora

$$f(x) - p_k(x) = \Pi(x) \frac{f^{(k+1)}(\xi)}{(k+1)!}, \quad \Pi(x) = \prod_{j=1}^k (x - x_j), \quad \xi \in \left[\min_{0 \leq i \leq k} x_i, \max_{0 \leq i \leq k} x_i \right]$$

Quindi, conoscendo una stima della derivata $k + 1$ -esima di $f(x)$ e una stima di $\Pi(x)$ è possibile calcolare una stima dell'errore del polinomio di interpolazione quando si valuta $p_k(x)$ nei nodi di interpolazione.

Ovviamente, ogni buon metodo di interpolazione che non interpola esattamente i punti ha come obiettivo quello di **minimizzare** questo errore; in termini matematici, si cerca una funzione interpolante $p_k(x)$ per l'insieme dei $k + 1$ nodi di interpolazione, tale per cui l'errore di approssimazione corrisponde al minimo possibile:

$$\min_{p_k(x)} |p_k(x) - f(x)|, \quad a = \min\{x_0, \dots, x_k, x\}, \quad b = \max\{x_0, \dots, x_k, x\}$$

Osservando quanto detto fino ad ora una delle soluzioni più immediate che potrebbe venire in mente per minimizzare l'errore è quella di *aumentare il numero di nodi di interpolazione*. Tuttavia una risoluzione di questo tipo può generare diversi problemi (non è sempre detto che sia facile o possibile valutare la funzione da interpolare su un maggior numero di punti) e, anche in caso fosse possibile determinare nuovi nodi, non è comunque detto che l'errore diminuisca. Esiste infatti un fenomeno, detto **fenomeno di Rounge**, secondo il quale anche aumentando il numero di nodi non è comunque sempre possibile azzerare l'errore: indicando con $\mathcal{N}_n$ l'insieme dei nodi dell'interpolazione per il grado k , anche supponendo che i nodi vengano scelti in modo abbastanza uniforme, è possibile dimostrare che, per ogni successione di insiemi di $\mathcal{N}_n$, esiste una funzione $f(x)$ continua su $[a, b]$ tale per cui

$$\lim_{k \rightarrow \infty} |p_k(x) - f(x)| \neq 0$$

Inoltre, possiamo anche facilmente dimostrare che non esiste una scelta di nodi *universalmente corretta*, ma che tuttavia esistono scelte che possono essere *migliori* di altre.

1 Interpolazione a tratti

Come accennato nell'introduzione, la prima idea per ridurre quanto più possibile l'errore di interpolazione consiste nell'effettuare l'interpolazione su singoli intervalli $[x_{i-1}, x_i]$, i cui estremi appartengono all'insieme dei nodi di interpolazione. Ovviamente, prima di operare una suddivisione, occorre definire un ordinamento stretto tra i nodi, così da evitare che i singoli intervalli possano sovrapporsi:

$$a = x_0 < x_1 < \dots < x_k = b$$

Si definisce quindi *polinomiale a tratti* su $[a, b]$ una funzione $F(x)$ che all' i -esimo sotto-intervallo $[x_i, x_{i+1}]$ fa corrispondere un polinomio $s_i(x)$ di grado prefissato n . Un esempio molto semplice è l'interpolazione lineare a tratti, in cui a ogni sotto-intervallo si fa corrispondere un diverso segmento di **retta**. Uno dei problemi principali dell'interpolazione polinomiale a tratti deriva dall'elevato numero di parametri necessari per descrivere la funzione $F(x)$. Ipotizzando infatti di suddividere l'intervallo in k sotto-intervalli e di associare a ciascuno delle funzioni polinomiali di grado n (caratterizzate quindi da $n + 1$ coefficienti), diventa necessario determinare $k(n + 1)$ parametri. La determinazione di tale numero di parametri è un'operazione complessa; inoltre, nella maggior parte dei casi, calcolare la funzione interpolante con così tanti parametri liberi (ovvero con un numero eccessivo di **gradi di libertà**) porta spesso a ricavare funzioni irregolari nei nodi di giunzione. L'idea fondamentale è quindi quella di definire classi di funzioni particolari che mantengano la struttura dei polinomi a tratti, introducendo al contempo opportuni vincoli di regolarità: prendiamo in esame il caso delle funzioni spline.

Definition 1.1. (Funzioni **spline**) Date $k + 1$ coppie (x_j, y_j) , si definisce *spline cubica* (di grado 3) relativa ai nodi $x_0, x_1, \dots, x_k$ una funzione $S_3(x)$ dotata delle seguenti caratteristiche:

- In ogni sotto-intervallo $[x_i, x_{i+1}]$, la restrizione $s_i(x)$ è un polinomio di grado minore o uguale a 3.
- $S_3(x_i) = y_i, \forall i = 0, \dots, k$ (condizione di interpolazione).
- $S_3(x) \in C^2([a, b])$.

Una delle proprietà salienti delle funzioni spline è proprio la loro regolarità globale, la quale introduce delle condizioni fondamentali affinché la funzione $s_{i-1}(x)$, ristretta all'intervallo $[x_{i-1}, x_i]$, si raccordi in modo regolare con la funzione successiva $s_i(x)$ definita su $[x_i, x_{i+1}]$ in corrispondenza del nodo interno x_i :

$$\begin{cases} s'_{i-1}(x_i) = s'_i(x_i) \\ s''_{i-1}(x_i) = s''_i(x_i) \end{cases} \quad \text{per } i = 1, \dots, k - 1$$

Intuitivamente, possiamo interpretare questa condizione come il vincolo geometrico di non avere punti "spigolosi" sul confine tra due intervalli adiacenti, garantendo una curvatura armoniosa.

Per determinare univocamente una spline cubica su k sotto-intervalli, è necessario calcolare i $4k$ parametri incogniti che costituiscono i singoli polinomi $s_0(x), \dots, s_{k-1}(x)$. A tale scopo, imponiamo tutte le condizioni precedentemente descritte:

$$\begin{cases} s_{i-1}(x_{i-1}) = y_{i-1} & i = 1, \dots, k \\ s_{i-1}(x_i) = y_i & i = 1, \dots, k \\ s'_{i-1}(x_i) = s'_i(x_i) & i = 1, \dots, k-1 \\ s''_{i-1}(x_i) = s''_i(x_i) & i = 1, \dots, k-1 \end{cases}$$

Le equazioni così ottenute sono in totale $4k-2$, numero insufficiente per determinare univocamente i $4k$ parametri: il sistema lineare risulta quindi sotto-determinato. Affinché sia possibile rendere la matrice del sistema quadrata, e dunque algebricamente risolvibile in modo univoco, occorre imporre altre due condizioni aggiuntive agli estremi dell'intervallo $[a, b]$. Queste condizioni definiscono la tipologia di **spline** scelta; si distinguono principalmente tre varianti:

- **Spline naturale**: le derivate seconde sul bordo esterno si annullano:

$$S''_3(x_0) = S''_3(x_k) = 0$$

- **Spline periodica**: le derivate prime e seconde agli estremi coincidono:

$$\begin{cases} S'_3(x_0) = S'_3(x_k) \\ S''_3(x_0) = S''_3(x_k) \end{cases}$$

- **Spline vincolata** (o clamped): le derivate prime sui bordi coincidono con i valori noti della derivata della funzione originale $f(x)$:

$$\begin{cases} S'_3(x_0) = f'(x_0) \\ S'_3(x_k) = f'(x_k) \end{cases}$$

Fino a questo momento non è stata enunciata la garanzia matematica dell'esistenza di tale soluzione. Introduciamo quindi un risultato classico, formale ma privo di dimostrazione, relativo all'esistenza e all'unicità della spline cubica:

Theorem 1.1. (Teorema di esistenza e unicità della spline) Dati $x_0, \dots, x_k$ nodi distinti su $[a, b]$ e i rispettivi valori $y_0, \dots, y_k$, una volta fissata una delle tre condizioni al contorno descritte (naturale, periodica o vincolata), esiste un'unica funzione spline $S_3(x)$ soddisfacente i vincoli di interpolazione e regolarità.

Il calcolo esplicito della spline, una volta determinate le condizioni, si può impostare efficacemente esprimendo l' i -esimo polinomio $s_i(x)$ sull'intervallo $[x_i, x_{i+1}]$ in funzione delle derivate seconde nei nodi (indicate con $m_i = S''_3(x_i)$):

$$\begin{aligned} s_i(x) = & \frac{y_i(x_{i+1} - x) + y_{i+1}(x - x_i)}{x_{i+1} - x_i} \\ & - \frac{(x_{i+1} - x_i)(x_{i+1} - x)}{6} \left[m_i \left(1 - \frac{(x_{i+1} - x)^2}{(x_{i+1} - x_i)^2} \right) \right] \\ & - \frac{(x_{i+1} - x_i)(x - x_i)}{6} \left[m_{i+1} \left(1 - \frac{(x - x_i)^2}{(x_{i+1} - x_i)^2} \right) \right] \end{aligned}$$

Nel caso di nodi equi-spaziati con passo $h = x_{i+1} - x_i$, l'espressione si semplifica notevolmente e i coefficienti incogniti delle derivate seconde $m_1, \dots, m_{k-1}$ per una *spline naturale* (ove $m_0 = m_k = 0$) si determinano risolvendo un sistema lineare tridiagonale a diagonale strettamente dominante della forma:

$$\begin{bmatrix} 4 & 1 & 0 & \cdots & 0 \\ 1 & 4 & 1 & \ddots & \vdots \\ 0 & 1 & 4 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & 4 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ \vdots \\ m_{k-1} \end{bmatrix} = \frac{6}{h^2} \begin{bmatrix} y_2 - 2y_1 + y_0 \\ y_3 - 2y_2 + y_1 \\ \vdots \\ y_{k-1} - 2y_{k-2} + y_{k-3} \\ y_k - 2y_{k-1} + y_{k-2} \end{bmatrix}$$

2 Interpolazione ai minimi quadrati

Uno dei problemi principali derivati dai metodi di interpolazione che abbiamo visto fino a questo momento deriva dal fatto che, data $f : \mathbb{R} \rightarrow \mathbb{R}$ e date le coppie $(x_0, y_0), \dots, (x_k, y_k)$, dove k è molto grande, diventa molto difficile determinare un polinomio $p_k(x)$ tale per cui $p_k(x_i) = y_i$, $\forall i = 1, \dots, k$, usando come base quella dei polinomi. Tuttavia, è possibile rilassare due condizioni che abbiamo implicitamente utilizzato fino a questo momento, è infatti possibile ammettere la possibilità di non passare esattamente per tutti i punti, limitandosi a definire una tolleranza massima in termini di **distanza** tra la funzione $p_k(x_i)$ e il valore $y_i = f(x_i)$, inoltre, è anche possibile ammettere che le funzioni base per la costruzione del nostro polinomio interpolante non siano più i soli polinomi, ma anche altre funzioni non lineari che abbiamo visto durante i corsi di analisi. Rilassando queste due condizioni siamo quindi in grado di generalizzare il problema dell'interpolazione, andando a scrivere il polinomio interpolante come **combinazione lineare** dei coefficienti c_j e delle funzioni modello $\phi_j(x)$ non necessariamente polinomiali:

$$\Phi(x) = \sum_{j=1}^m c_j \phi_j(x), \quad m \leq k$$

Il principale vantaggio di un approccio di questo tipo è che è necessario un minor numero di funzioni modelli rispetto a quelle che sono necessarie quando utilizziamo la base polinomiale. Tuttavia, il fatto che la funzione Φ non interpoli esattamente tutti i punti y_i crea un problema fondamentale: l'errore, definito come la distanza sul piano tra $\Phi(x_i)$ e y_i , deve essere reso quanto più piccolo possibile; idealmente, dovrebbe essere il minimo possibile. Possiamo allora definire i nostri coefficienti c_j come i coefficienti tali per cui l'errore di approssimazione per ogni singola coppia $(\Phi(x_i), y_i)$ è minimo:

$$c = \min_{c \in \mathbb{R}^{m+1}} \gamma(c), \quad \gamma(c_0, \dots, c_m) = \sum_{i=0}^k [\Phi(x_i) - y_i]^2$$

Otteniamo quindi un vero e proprio problema di ottimizzazione, dove l'obiettivo è quello di minimizzare una particolare funzione $\gamma(c)$ che contiene gli scarti quadratici medi delle valutazioni della funzione Φ ; possiamo anche notare una proprietà fondamentale di questo problema, andando a espandere la funzione $\gamma(c)$ si ottiene:

$$\gamma(c) = \sum_{i=0}^k \left[\sum_{j=0}^m \phi_j(x_i) c_j - y_i \right]^2 \rightarrow \gamma(x) = \sum_{j=0}^k r_j(c)^2 = \|r(c)\|_2^2$$

Si può quindi notare un'analogia immediata con il problema ai minimi quadrati visto durante il capitolo sui sistemi lineari. A partire da questa analogia possiamo riscrivere il problema sotto forma di sistema lineare usando le funzioni modello valutate nei diversi punti x_i :

$$A = \begin{bmatrix} \phi_0(x_0) & \dots & \phi_m(x_0) \\ \vdots & \ddots & \vdots \\ \phi_0(x_k) & \dots & \phi_m(x_k) \end{bmatrix}, \quad c = \begin{bmatrix} c_0 \\ \vdots \\ c_m \end{bmatrix}, \quad b = \begin{bmatrix} y_0 \\ \vdots \\ y_k \end{bmatrix}$$

Data l'analogia con il problema ai minimi quadrati dei sistemi lineari, sono valide le considerazioni che abbiamo visto nel relativo capitolo. In particolare, il problema può essere risolto usando sia il metodo delle **equazioni normali** e altresì il metodo della **fattorizzazione QR**. Anche il teorema che garantisce l'esistenza della soluzione e la sua unicità, nel caso di $\text{Rk}(A) = n$, è valido per l'interpolazione ai minimi quadrati. Occorre tuttavia fare alcune precisazioni a livello teorico

Parte IV

Integrazione numerica

CAPITOLO 8

FORMULE DI QUADRATURA E GRADO DI PRECISIONE

Una delle principali categorie di problemi che andiamo a studiare durante il corso riguarda i **problemi di integrazione**. Ipotizziamo infatti di prendere una generica funzione $f(x)$, un generico intervallo $[a, b]$ e di voler calcolare il valore dell'integrale della funzione lungo l'intervallo di valori:

$$\int_a^b f(x) \, dx$$

Una delle domande che è sempre importante porsi quando si procede nella risoluzione di integrali con il metodo esatto è se effettivamente sia possibile trovare una primitiva in forma elementare e se, anche nel caso in cui sia possibile, sia facile da trovare. Alcuni esempi di funzioni per cui non esiste una primitiva in forma elementare sono: funzioni **ellittiche** o delle funzioni **in più variabili** o della curva **gaussiana**. Inoltre, potrebbe anche non essere nota a priori la funzione $f(x)$, ma solo delle valutazioni di quest'ultima in una serie di punti $x_0, x_1, \dots, x_n$. In tutti questi casi è dunque possibile cercare di **approssimare** una soluzione, cioè cercare di risolvere l'integrale sfruttando delle formule alternative che non richiedano di passare dalla **primitiva**. Durante questo capitolo quindi, tratteremo alcuni metodi che permettono di trovare delle soluzioni ai problemi di approssimazione degli integrali, scritti in questa forma:

$$I(\rho \cdot f) = \int_a^b \rho(x) \cdot f(x) \, dx, \quad \rho(x) \geq 0, \quad \rho : [0, b] \rightarrow \mathbb{R}^n$$

Dove ρ è detta **funzione peso**. Da un punto di vista matematico la funzione peso rappresenta una distribuzione di probabilità e la sua utilità deriva dal fatto che permette di esprimere, sotto forma di funzione, l'andamento di $f(x)$; si pensi al caso in cui si deve integrare una funzione nell'intervallo $[0, +\infty]$ che però decade molto velocemente man mano che $x \rightarrow \infty$, grazie alla funzione peso siamo in grado di specificare che i nodi da posizionare devono essere vicini allo zero, per esempio. Essendo $\rho(x)$ una distribuzione di probabilità, è possibile definire i valori attesi, detti anche **momenti della funzione peso**, come:

$$m = \int_a^b x^n \rho(x) \, dx < +\infty$$

Possiamo vedere questi *momenti* come una misura che descrive come il peso è distribuito sull'intervallo. Generalmente, durante tutto il capitolo, assumeremo sempre che $\rho(x) = 1$. Abbiamo quindi detto che l'idea alla base del calcolo approssimato degli integrali, è quella di determinare una formula che sia in grado di stabilire il valore dell'integrale senza passare dalla primitiva; definiamo questa formula come **formula di quadratura**.

Definition 0.1. (formula di quadratura) Dati $a, b \in \mathbb{R}$ e $\rho(x)$, definiamo J_n la formula di quadratura su $n + 1$ nodi con pesi $a_0, \dots, a_n$, la funzione

$$J_n : C([a, b]) \rightarrow \mathbb{R}, \quad J_n = \sum_0^n f(x_i) \cdot a_i, \quad J_n : \mathbb{R}^{n+1} \rightarrow \mathbb{R}$$

dove n è il grado del polinomio interpolante che viene utilizzato per costruire la formula di quadratura.

Una volta ricavati i nodi di integrazione e i pesi, la formula di quadratura è univocamente determinata per f . Una volta definita la **formula di quadratura** è possibile calcolare altrettanto univocamente, data una funzione f e un intervallo $[a, b]$, il valore di $I(f, [a, b])$ attraverso la formula $J_n(f)$. Tuttavia, ogni qualvolta si determina *qualcosa* di approssimato, è fondamentale capire se quella ricavata è una buona approssimazione o meno; definiamo quindi l'**errore** di una formula di quadratura $J_n(f)$ come:

$$E_n(f) = \int_a^b f(x)\rho(x) dx - J_n(f), \quad E_n : C([a, b]) \rightarrow \mathbb{R}$$

Ad un primo sguardo la formula potrebbe sembrare poco funzionale, infatti per calcolare l'errore è necessario aver valutato l'integrale in forma esatta, che è quello che vogliamo evitare usando la **formula di quadratura**. In realtà questa definizione non viene generalmente applicata e, inoltre, esiste a tal proposito un teorema che fornisce una formula effettivamente applicabile per determinare l'errore di una **formula di quadratura**. L'errore è un ottimo indicatore per quanto riguarda la precisione di una formula di quadratura, tuttavia però non è immediato, poiché dipende dalla funzione rispetto a cui andiamo a calcolarlo. Dobbiamo quindi trovare un altro indice che permetta di specificare in che misura una formula di quadratura è accurata:

Definition 0.2. (Grado di precisione) Data J_n **formula di quadratura** si definisce con $m \in \mathbb{N}$ il grado di precisione, definito come l'intero tale per cui

$$\boxed{E_n(1) = E_n(x) = \dots = E_n(x^m) = 0}$$

$$\boxed{E(x^{m+1}) \neq 0}$$

La prima condizione implica che, essendo le basi dei polinomi integrate in modo esatto, allora anche tutti i polinomi composti su tali basi sono integrati esattamente; ad esempio, se una formula di quadratura ha grado di precisione 3, allora ogni polinomio nella forma $a_0 + a_1x + a_2x^2$ sarà integrato esattamente. In generale, durante il corso utilizzeremo due diverse formule di quadratura:

- **Formula dei trapezi.** È la più semplice delle formule di quadratura di Newton-Cotes chiuse. L'idea geometrica è quella di approssimare la funzione $f(x)$ nell'intervallo $[a, b]$ tramite il segmento di retta che unisce i punti $(a, f(a))$ e $(b, f(b))$, calcolando poi l'area del trapezio risultante. La formula corrisponde a:

$$J_1(f) = \frac{b-a}{2} [f(a) + f(b)]$$

Tra tutte le formule di quadratura che utilizzano esclusivamente i due estremi dell'intervallo come nodi, questa formula è l'unica che massimizza il grado di precisione, che risulta pari a 1.

- **Formula di Cavalieri-Simpson.** Questa formula introduce un terzo nodo equispaziato coincidente con il punto medio dell'intervallo. L'interpretazione geometrica è analoga a quella dei trapezi, ma in questo caso la funzione $f(x)$ viene approssimata mediante l'unica parabola (polinomio di grado 2) passante per i tre punti d'intersezione. La formula di quadratura di Cavalieri-Simpson corrisponde a:

$$J_2(f) = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

Fissati questi tre nodi specifici, la formula di Cavalieri-Simpson massimizza il grado di precisione. Grazie alla simmetria del nucleo d'errore, essa guadagna un ordine di precisione "gratuito", integrando esattamente tutti i polinomi di grado minore o uguale a 3.

All'inizio del capitolo è stato detto che una formula di quadratura è univocamente definita da **nodi** e **pesi**. Ipotizziamo quindi di aver fissato i nodi, che ipotizziamo essere $n+1$, e di voler determinare i **pesi**, anch'essi ovviamente $n+1$, tali per cui il grado di precisione è almeno pari al grado del polinomio. Affinché il grado di precisione sia pari almeno al grado del polinomio, è necessario che gli errori di integrazione delle basi polinomiali x^i fino al grado n siano nulli. Portando tutto a sistema si ottiene un sistema lineare nella forma:

$$\begin{cases} E_n(1) = 0 \\ E_n(x) = 0 \\ \vdots \\ E_n(x^n) = 0 \end{cases} = \begin{cases} m_0 = a_0 + \dots + a_n \\ m_1 = a_0 x_0 + a_1 x_1 + \dots + a_n x_n \\ \vdots \\ m_n = a_0 x_0^n + a_1 x_1^n + \dots + a_n x_n^n \end{cases} \rightarrow \begin{bmatrix} 1 & 1 & \dots & 1 \\ x_0 & x_1 & \dots & x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_0^n & x_1^n & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ \vdots \\ m_n \end{bmatrix}$$

Il sistema appena trovato ha una struttura del tipo $Va = m$, dove la matrice V presenta una struttura simile alla matrice di Vandermonde di cui è stato ampiamente trattato nel capitolo scorso sull'interpolazione. L'unica differenza tra le due matrici è che rispetto alla matrice di Vandermonde originale, quella che abbiamo ricavato adesso non è altro che la **trasposta** dell'originale. Tuttavia, vale che:

$$\det(V) = \det(V^T) = \prod_{\substack{i=0 \\ i>j}}^n (x_i - x_j)$$

Il determinante di questa matrice è quindi diverso da zero e, per questo motivo, il sistema ammette sicuramente una e una sola soluzione. Quindi, fissati i nodi esiste ed è unica la formula di quadratura per f rispetto ai nodi $x_0, \dots, x_n$ il cui grado di precisione risulta almeno n ; per dimostrare che il grado di precisione è esattamente n si deve dimostrare che $E(x^{n+1}) \neq 0$. Una generalizzazione del problema, maggiormente complessa, è quella in cui oltre a dover determinare i pesi, si devono anche determinare i nodi della formula di quadratura così da massimizzare il grado di precisione della formula. In questo caso le incognite del sistema sono non più $n+1$, ma $2n+2$ e quindi è necessario imporre $2n+2$ equazioni; anche in questo caso si impone, in modo del tutto simile al caso con $n+1$ equazioni, che $E_n(1) = E_n(x) = \dots = E_n(x^{2n+1}) = 0$. Il sistema così ottenuto è quindi corrispondente a:

$$\begin{cases} a_0 + a_1 + \dots + a_n = 0 \\ a_0 x_0 + a_1 x_1 + \dots + a_n x_n = 0 \\ \vdots \\ a_0 x_0^{2m+1} + a_1 x_1^{2m+1} + \dots + a_n x_n^{2n+1} = 0 \end{cases} \quad (1)$$

La differenza fondamentale rispetto al sistema precedente è che non è possibile dire altrettanto facilmente che la soluzione esiste, poiché il determinante non è fondamentalmente esprimibile attraverso una formula immediata e non è possibile dimostrare che sia diverso da 0 indipendentemente dai coefficienti scelti. Tuttavia però, esiste un risultato che postula l'esistenza di una soluzione per il sistema:

Theorem 0.1. Il sistema (1) ammette sempre un'unica soluzione, per ogni scelta di nodi, di ρ e di $[a, b]$ tale per cui il grado di precisione è almeno pari a $2n+1$.

L'unica formula di quadratura su $[a, b]$ che verifica il sistema (1), ovvero, l'unica formula di quadratura che massimizza il grado di precisione ottimizzando sia sui nodi che sui pesi, è chiamata **formula Gaussiana su $[a, b]$ con $n+1$ nodi**.

1 Errore nelle formule di quadratura

Il problema è che calcolare l'errore di ogni singola formula di quadratura è un'operazione che dal punto di vista computazionale è molto costoso. Inoltre, per come è stato definito l'errore

nel caso precedente, affinché sia possibile calcolarlo è necessario conoscere la valutazione esatta dell'integrale che si vuole approssimare. Nel caso ideale sarebbe quindi conveniente riuscire a trovare delle formule che permettano di dare una stima qualitativa all'errore in modo del tutto indipendente dall'integrale di partenza. A tal propositivo viene in aiuto un teorema noto come **teorema di Peano**:

Theorem 1.1. (Teorema di Peano) Supponendo che la formula di quadratura J_n abbia grado di precisione esatto m e che la funzione $f \in C^{m+1}([a, b])$, allora l'errore si può scrivere nella forma:

$$E_n(f) = I(f\rho) - J_n(f) = \frac{1}{m!} \int_a^b f^{(m+1)}(t)G(t) dt$$

Dove la funzione $G(t)$, detta **nucleo di Peano**, è definita nel seguente modo:

$$G(t) = E_n((x-t)_+^m) = I(\rho(x)(x-t)_+^m) - J_n((x-t)_+^m)$$

essendo la funzione potenza troncata definita come:

$$(x-t)_+^m = \begin{cases} (x-t)^m & \text{se } x \geq t \\ 0 & \text{se } x < t \end{cases}$$

Come notiamo dalla formula, la presenza di un termine $f^{(m+1)}(t)$ causa non pochi problemi dal punto di vista computazionale, infatti per m molto grandi il calcolo della derivata $(m+1)$ -esima risulta un'operazione molto onerosa, tutt'al più se esso si trova pure all'interno di un integrale. Si può d'altra parte sfruttare un'idea esemplificativa: se la funzione $G(t)$ non cambia di segno nell'intervallo allora è possibile applicare un teorema che permette di rimuovere il termine derivato dall'integrale, il **teorema della media integrale**. Applicando questa esemplificazione si ottiene che:

$$E_n(f) = \frac{1}{m!} f^{(m+1)}(\varepsilon) \int_a^b G(t) dt$$

Siccome l'integrale del nucleo di Peano è del tutto indipendente dalla funzione f , è possibile imporre la condizione $f(x) = x^{(m+1)}$ e valutare l'errore corrispondente, andando a rimuovere del tutto la dipendenza rispetto alla funzione di partenza

$$E_n(x^{(m+1)}) = \frac{1}{m!} (m+1)! \int_a^b G(t) dt \implies \int_a^b G(t) \rho(x) dt = \frac{E_n(x^{m+1})}{m+1}$$

Tutte queste considerazioni possono essere fatte convergere all'interno di un unico corollario fondamentale, il quale introduce una formula esplicita per il calcolo dell'errore di integrazione per una generica funzione $f(x)$ la cui valutazione esatta non è nota e facilmente determinabile:

Lemma 1.2. Se $f \in C^{m+1}([a, b])$ e $G(t)$ non cambia di segno in $[a, b]$, allora

$$E_n(f) = \frac{E_n(x^{m+1})}{m+1} \cdot f^{(m+1)}(\varepsilon), \quad \varepsilon \in [a, b]$$

Nel caso in cui si cerchi di ottenere una maggiorazione del tipo $|E_n(f)| \leq M$ è possibile utilizzare

$$M = \left[\max_{x \in [a, b]} |f^{(m+1)}(x)| \cdot \frac{E_n(x^{m+1})}{m+1} \right]$$

Il lemma appena enunciato potrebbe sembrare inutile all'apparenza, ma per la maggior parte delle formule che studieremo noi questa ipotesi è più che valida. A partire da questo corollario è possibile andare a sviluppare delle formule generali per valutare l'errore di approssimazione nel caso delle

formule dei trapezi e di Cavalieri-Simpson. Prendiamo in esame il caso della formula dei trapezi

$$\begin{aligned} E_1(f) &= \frac{f^{(2)}(\varepsilon)}{2} E_1(x^2) \\ &= \frac{f^{(2)}(\varepsilon)}{2} \left[\int_a^b x^2 dx - \frac{b-a}{2} (a^2 + b^2) \right] \\ &= \frac{f^{(2)}(\varepsilon)}{2} \left[\frac{b^3 - a^3}{3} - \frac{b-a}{2} (a^2 + b^2) \right] \\ &= \frac{f^{(2)}(\varepsilon)}{2} \left[-\frac{(b-a)^3}{6} \right] = -\frac{(b-a)^3}{12} f^{(2)}(\varepsilon) \end{aligned}$$

Questo stesso ragionamento può essere esteso anche per il calcolo dell'errore nella formula di quadratura di **Cavalieri-Simpson** si ottiene una formula dell'errore corrispondente a:

$$E_3(f) = \boxed{-\frac{(b-a)^5}{2880} \cdot f^{(4)}(\varepsilon)}, \quad \varepsilon \in [a, b]$$

Un tipo particolare di formule di quadratura, sono le cosiddette formule di quadratura **interpolatorie**. L'idea alla base di queste formule è quella di sostituire la funzione **integranda** con il corrispettivo **polinomio interpolatorio** scritto nella base di **Lagrange**. Matematicamente, quindi, al posto di integrare la funzione $f(x)$, la cui struttura potrebbe essere non nota a priori eccetto - eccetto per un certo numero $n+1$ di coppie $(x_0, f(x_0)), \dots, (x_n, f(x_n))$ - si integra il polinomio di Lagrange $L_n(x)$:

$$\int_a^b f(x) dx \approx \int_a^b L_n(x) dx$$

Se la funzione da integrare $f(x)$ è un polinomio di grado minore o uguale a n , allora il polinomio interpolatore di Lagrange coincide esattamente con la funzione stessa e, pertanto, non vi è alcun errore di approssimazione. Inoltre, siccome il polinomio interpolatore è scritto usando la **base dei polinomi** ed ha grado pari ad n , allora integra esattamente tutti i polinomi di grado fino ad n . Possiamo quindi concludere che una formula di quadratura interpolatoria, la cui funzione $L_n(x)$ ha grado n , ha anche **grado di precisione** pari ad almeno n .

2 Formule di Newton-Cotes

Una prima tipologia di formule di quadratura interpolatorie sono le formule di **Newton-Cotes**. Si definisce una formula di **Newton-Cotes** una **formula di quadratura** interpolatoria, tale per cui i nodi di integrazione sono **equispaziati**, cioè possono essere scritti come $x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_n = b$ (dove h corrisponde all'ampiezza dell'intervallo $(b-a)/n$), e tale per cui si utilizza la base di Lagrange per costruire il polinomio interpolatorio. Se l'insieme dei nodi include anche gli estremi dell'intervallo, si parla di **formule chiuse**, mentre se l'insieme dei nodi non include gli estremi, si parla di **formule aperte**. Il peso i -esimo è ricavabile valutando $I(l_i(x))$ lungo l'intervallo $[a, b]$:

$$a_i = \int_a^b l_i(x) dx$$

Una delle proprietà delle formule di **Newton-Cotes** è che, fintanto che i nodi del polinomio interpolatorio sono al più sette, allora i pesi a_i sono a **segno costante**. Appena però i nodi diventano ≥ 7 allora i pesi diventano a segno alternato e le formule tendono ad avere un comportamento **instabile**. Uno dei vantaggi principali delle formule di **Newton-Cotes**, deriva proprio dalla proprietà per cui l'espressione della formula dell'errore di **Newton-Cotes** rimuove il problema di dover calcolare direttamente $E_n(x^{m+1})$; l'espressione è infatti data da

$$E_n(f) = \begin{cases} c_n \cdot h^{n+2} \cdot f^{n+1}(\varepsilon) & n \text{ dispari} \\ c_n \cdot h^{n+3} \cdot f^{n+2}(\varepsilon) & n \text{ pari} \end{cases}$$

Una formula di **Newton-Cotes** basata su un polinomio interpolatore con **grado pari a n** ha un grado di precisione che dipende dalla parità di n : se n è dispari, allora il grado di precisione è esattamente n , mentre se n è pari, allora il grado di precisione della formula di quadratura è pari a $n + 1$.

3 Integrazione su intervalli

Il problema principale delle formule di **Newton-Cotes** deriva principalmente dal fatto che l'errore ha una dipendenza diretta rispetto a una potenza dell'ampiezza dell'intervallo $b - a$. Se prendiamo due punti relativamente distanti, allora l'errore di integrazione tende ad aumentare in modo considerevole, rendendo l'approssimazione poco precisa. Per risolvere questo problema possiamo approssimare il problema in due diversi modi: **aumentando il numero di nodi, aumentando il numero di sotto-intervalli**. Il primo approccio possibile è quindi quello di aumentare il numero di nodi di integrazione, tuttavia, una soluzione di questo tipo presenta non pochi problemi: non è infatti detto che sia possibile conoscere la valutazione della funzione in più punti e, inoltre, non è detto che l'approssimazione diventi più precisa; è infatti possibile dimostrare che per $n \geq 7$ le formule di Newton-Cotes diventano numericamente instabili. Il secondo approccio, cioè quello di suddividere l'intervallo $[a, b]$ in L sotto-intervalli equi-dimensionati nella forma $[x_{i-1}, x_i]$, interpolare così da ottenere una formula di interpolazione per ogni sotto-intervallo, integrare singolarmente su ognuno di essi l'interpolante e infine sommare tutti gli integrali ottenuti, è invece migliore nell'ottica del voler aumentare la precisione della formula. Supponiamo quindi di adottare la seconda metodologia. Si procede andando a suddividere l'intervallo di integrazione in $L + 1$ nodi equispaziati e successivamente, per ogni intervallo $[x_{i-1}, x_i]$, si applica una formula di quadratura interpolatoria (ad esempio **trapezi** o **Cavalieri-Simpson**). L'integrale finale sarà quindi dato dalla somma di tutte le approssimazioni ottenuti sui diversi sotto-intervalli:

$$\int_a^b f(x) dx = \sum_{i=1}^L \int_{x_{i-1}}^{x_i} f(x) dx \approx \sum_{i=1}^L J_n^{[x_{i-1}, x_i]}(f) \quad (2)$$

Se da una parte, questo approccio, ci garantisce maggiore precisione nel calcolo dell'integrale, dall'altra parte ci obbliga a calcolare la valutazione della funzione in un certo numero di punti interni all'intervallo di integrazione; in particolare, il numero di punti in cui è necessario calcolare la funzione è pari a $L + 1 + (n - 1) \cdot L = nL + 1$. Questo procedimento permette quindi di generalizzare la definizione di errore che abbiamo visto nel paragrafo sulle formule di Newton-Cotes; in particolare, l'errore totale sarà dato dalla somma degli errori su ogni singolo sotto-intervallo:

$$\begin{aligned} E_n^{(G)} &= \sum_{i=1}^L c \cdot h^\alpha \cdot f^{(\alpha-1)}(\varepsilon_i) \\ &= c \cdot h^\alpha \sum_{i=1}^L f^{(\alpha-1)}(\varepsilon_i) \\ &= c \cdot h^\alpha \cdot L \cdot \left[\frac{1}{L} \sum_{i=1}^L f^{(\alpha-1)}(\varepsilon_i) \right] \\ &= \boxed{c \cdot h^\alpha \cdot L \cdot f^{(\alpha-1)}(\varepsilon)} \end{aligned}$$

A partire da questa formula dell'errore generalizzato per le formule di quadratura, è possibile distinguere due casi, una formula dell'errore generalizzato nel caso di n (numero di nodi) pari e nel caso di n dispari:

$$E_n^{(G)} = \begin{cases} c_n \frac{(b-a)^{n+2}}{L^{n+1}} f^{n+1}(\varepsilon) & \text{n dispari} \\ c_n \frac{(b-a)^{n+3}}{L^{n+2}} f^{n+2}(\varepsilon) & \text{n pari} \end{cases}$$

A partire da queste due formule è possibile generalizzare anche la stima dell'errore per le due principali formule di quadratura che abbiamo visto durante questo corso: la formula dei **trapezi**

e la formula di **Cavalieri-Simpson**. In particolare, è sufficiente sostituire i valori nella rispettiva formula dell'errore ($n = 1$ per i trapezi, $n = 2$ per Cavalieri-Simpson), ottenendo:

Formula dei trapezi	$-\frac{(b-a)^3}{12L^2}f^{(2)}(\varepsilon), \quad \varepsilon \in [a, b]$
Formula di Simpson	$-\frac{(b-a)^5}{2880L^4}f^{(4)}(\varepsilon), \quad \varepsilon \in [a, b]$

Tutte le formule di quadratura interpolatorie ottenute applicando la formula base iterativamente su un certo numero L di sotto-intervalli, sono dette **formule di Newton-Cotes generalizzate** (o composte) e possono essere calcolate sommando i contributi dei singoli sotto-intervalli. Nel caso della formula dei trapezi, si ottiene la seguente formula generalizzata:

$$J_1^G(f) = \sum_{i=1}^L \frac{b-a}{2L} (f(x_{i-1}) + f(x_i)) = \frac{b-a}{2L} \sum_{i=1}^L (f(x_{i-1}) + f(x_i)) =$$

$$\frac{b-a}{2L} \left(f(x_0) + 2 \sum_{i=1}^{L-1} f(x_i) + f(x_L) \right)$$

Procedendo nello stesso modo per la formula di **Cavalieri-Simpson**, ricordando che i pesi per ogni sotto-intervallo sono proporzionali a 1, 4, 1, si ottiene la seguente formula generalizzata:

$$J_2^G(f) = \sum_{i=1}^L \frac{b-a}{6L} \left[f(x_{i-1}) + 4f\left(\frac{x_{i-1} + x_i}{2}\right) + f(x_i) \right] =$$

$$\frac{b-a}{6L} \left[f(x_0) + 4 \sum_{i=1}^L f\left(\frac{x_{i-1} + x_i}{2}\right) + 2 \sum_{i=1}^{L-1} f(x_i) + f(x_L) \right]$$

NOTA: Queste due parti sono state trattate nell'anno 2025/2026

4 Formule Gaussianhe

Le formule gaussiane sono forme interpolatorie ad alto grado di precisione. Per calcolare queste formule di quadratura è necessario risolvere dei sistemi non lineari con un numero di parametri estremamente alto, oppure, è possibile ricavare tali formule attraverso la tecnica dei **polinomi ortogonali**. I nodi delle formule gaussiane si possono scrivere come zeri di particolari polinomi, detti polinomi ortogonali. Più precisamente sono gli zeri di $\{q_n\}_{n \in \mathbb{N}}$, $\deg(q_n) = n$ tale per cui il prodotto scalare $\langle q_n(x), p(x) \rangle$ rispetto a un generico polinomio è sempre zero, $\forall p(x) : \deg(p) < n$. Per lo spazio vettoriale di riferimento il prodotto scalare è così definito

$$\int_a^b q_n(x)r(x)\rho(x)dx$$

Si definisce quindi la classe di polinomi ortogonali Π^* come l'insieme dei polinomi algebrici, a coefficienti reali, ortogonali rispetto al prodotto scalare

$$\Pi^* = \{q_i(x) \mid \deg(q_i) = i; \langle q_i, q_j \rangle = h_i \delta_{ij}\}, \quad i, j = 1, 2, \dots$$

I coefficienti h_i indicano le **costanti di normalizzazione**. Dati $[a, b]$ e $\rho(x)$, gli elementi di Π^* costituiscono una base per Π ; per essi valgono le seguenti proprietà:

- $\langle p, q_n \rangle = 0, \quad \forall p \in \Pi_{n-1}$.
- $q_n(x)$ ha n zeri reali e distinti in $]a, b[$.

5 Formule di quadratura che coinvolgono derivate

Una formula di quadratura può essere strutturata anche conoscendo, oltre alle valutazioni della funzione nei punti, le valutazioni della derivata della funzione in quei punti. Si possono generalizzare anche i concetti di grado di precisione, di scelta dei nodi e pesi ottimali anche per formule di quadratura del tipo:

$$\int_a^b f(x) \, dx \approx \sum_{i=1}^{m_1} f(x_i) + \sum_{i=1}^{m_2} f^{(1)}(x_i) + \cdots + \sum_{i=1}^{m_q} f^{(q)}(x_i)$$

Il procedimento è del tutto analogo a quello visto anche nel caso di altre formule: si calcolano i pesi in modo da avere $E_n(x) = \cdots = E_n(x^m) = 0$ e si risolve il sistema lineare che viene generato dall'imposizione di questi vincoli. Le incognite del sistema, e quindi la soluzione saranno i pesi della formula.

Parte V

Equazioni non lineari

CAPITOLO 9

DEFINIZIONI E METODI ITERATIVI

In questo capitolo si affrontano quelle che in gergo vengono chiamate **equazioni non lineari**. Un'equazione non lineare è, in termini puramente intuitivi, una particolare equazione dove i termini dell'incognita non sono **lineari**, cioè non sono elementi della base dei polinomi o derivati. In termini matematici, tuttavia, la definizione è più rigorosa: sia $f : \mathbb{R} \rightarrow \mathbb{R}$ una funzione continua su un certo intervallo $\mathcal{I} \subseteq \mathbb{R}$ e si supponga che $f(x)$ non sia della forma $f(x) = a_1x + a_0$ con $a_0, a_1 \in \mathbb{R}$. Si definisce l'uguaglianza:

$$f(x) = 0$$

come **equazione non lineare** nell'incognita x . Il problema che affrontiamo durante questo capitolo è proprio di determinare gli zeri (o radici) di una generica **equazione non lineare** espressa nella forma che abbiamo appena introdotto. La necessità di studiare dei metodi approssimati per risolvere questo genere di equazioni, a differenza del caso degli integrali, è che nel caso delle **equazioni non lineari** non è proprio possibile in certi casi trovare la soluzione **direttamente**. In generale, quando parliamo di **risolvere direttamente** si intende riuscire a determinare le radici dell'equazione utilizzando un numero finito di operazioni, come ad esempio la formula risolutiva delle equazioni di secondo grado. Si supponga però di prendere la seguente equazione non lineare:

$$f(x) = x \log(x) + \frac{1}{3} \arctan(x) = 0$$

Si nota immediatamente che non esiste alcuna formula risolutiva diretta, quindi un insieme finito di operazioni, che permetta di giungere velocemente ad una delle possibili radici di questa equazione. D'altronde, anche nel caso delle equazioni polinomiali non sempre è possibile trovare una soluzione attraverso un metodo diretto, infatti per tutte le equazioni polinomiali di grado **superiore** a cinque non esiste alcuna formula diretta. Una soluzione possibile, che non richiede alcun metodo numerico, è quella del metodo grafico: si rappresentano cioè le funzioni e se ne osserva le intersezioni; tuttavia questo metodo funziona solo per equazioni molto semplici, con pochi termini e con grafici facilmente rappresentabili. Rimane dunque possibile solo l'opzione di utilizzare un metodo **iterativo** che permetta di calcolare numericamente un'**approssimazione** o, in certi casi, anche la radice vera e propria di un'equazione $f(x) = 0$. In generale, possiamo definire un metodo iterativo come una formula del tipo

$$x_{n+1} = \Phi(x_n, \dots, x_1, x_0)$$

Dove $x_0, x_1, \dots, x_n$ solo valori dati. Nello specifico, questi metodi sono detti **metodi a k punti** e la funzione Φ prende il nome di **funzione di iterazione del metodo**; un metodo a k punti per cui la funzione Φ rimane costante, è detto **metodo stazionario a k punti**. La funzione di iterazione è una funzione che non ha una definizione specifica, in termini puramente astratti è possibile definire un numero infinito di funzioni di iterazione per una generica equazione $f(x) = 0$. Tuttavia, ciò che fondamentalmente differenzia i metodi tra di loro è il concetto di **convergenza**, infatti di tutte le possibili funzioni di iterazione ne esiste una parte che non è in grado di portare ad

almeno una delle radici del polinomio; in gergo questi metodi sono detti *non convergenti*. Inoltre, se si considerano i metodi che invece convergono ad almeno una delle radici, ne possiamo fare una classificazione in termini di **bontà** del metodo, utilizzando dei parametri come l'**ordine di convergenza**; tuttavia questi concetti verranno introdotti più avanti. Dobbiamo infatti, prima di tutto, definire formalmente il concetto di convergenza:

Definition 0.1. (Convergenza di un metodo iterativo) una successione $\{x_n\}_{n \in \mathbb{N}}$ risultante da un metodo iterativo è detta convergente se e solo se

$$\lim_{k \rightarrow \infty} x_n = \alpha$$

dove α è una radice di $f(x)$

Una prima differenza rispetto al caso dei sistemi lineari riguarda proprio la caratterizzazione del punto iniziale. Nei sistemi lineari la convergenza era fondamentalmente indipendente dalla scelta del punto iniziale x_0 ed era solo relativa alla matrice di iterazione, mentre nelle equazioni **non lineari** la scelta del punto iniziale diventa fondamentale per capire se un metodo converge, infatti due scelte diverse di x_0 possono portare a risultati diversi; diciamo che la convergenza è **locale** e non **globale**. In realtà, durante la trattazione, verranno introdotti anche dei teoremi che garantiscono una convergenza "globale" relativamente a un determinato intervallo, ma in termini generali possiamo comunque dire che la convergenza è **locale**. La scelta del punto iniziale diventa quindi anch'esso a sua volta un problema che deve essere risolto con opportuni mezzi, in particolare, la scelta del punto iniziale deve essere fatta in modo da ottenere un punto iniziale quanto più possibile vicino alla radice. L'idea è quindi quella di usare dei metodi già visti durante il corso di analisi matematica:

- **Teorema dei valori intermedi:** Supponendo $f \in C^1([a, b])$ e $a, b \in \mathbb{R}$, se $f(a) \cdot f(b) < 0$ allora esiste almeno una radice $\alpha \in \mathbb{R}$ della funzione f ,
- **Studio del segno della derivata.**
- **Separazione grafica:** Se $f(x) = g(x) - h(x)$ potrebbe essere conveniente (se $g(x)$ e $h(x)$ hanno grafico e proprietà note) scrivere $f(x) = 0$ come $g(x) = h(x)$ per poi studiare le intersezioni tra le due funzioni.

Come abbiamo detto in precedenza, ipotizzando di avere determinato diversi metodi iterativi che convergono ad una delle radici α del polinomio, è possibile anche classificare questi metodi in funzioni della loro efficacia, o per meglio dire, della velocità con cui convergono. Possiamo quindi introdurre il concetto di **ordine di convergenza**:

Definition 0.2. (Ordine di convergenza) Data una successione $\{x_n\}$ convergente ad una radice α , si ponga $e_n = x_n - \alpha$; se esistono due numeri reali $p \geq 1$ e $C \neq 0$ tali che sia

$$\lim_{n \rightarrow \infty} \frac{|e_{n+1}|}{|e_n|^p} = C$$

si dice che la successione ha **ordine di convergenza** p e **fattore di convergenza** C

L'ordine di convergenza è quindi, nella sua definizione, una misura di quanto velocemente un particolare metodo tende ad avvicinarsi alla radice dell'equazione $f(x) = 0$, maggiore è l'ordine di convergenza e maggiore è la velocità con cui questo avvicinamento avviene. Se l'ordine di convergenza è pari ad uno si parla di convergenza lineare, mentre se la convergenza è maggiore di 1 si dice **convergenza super-lineare**. Prima dei metodi di punto fisso, che sono la principale classe di metodi che verranno analizzati durante questo corso, è conveniente studiare due metodi per approssimare una radice α di un'equazione non lineare:

- **Metodo di Bisezione.**
- **Metodo delle Secanti.**

Entrambi i metodi sono relativamente semplici da affrontare, ma rappresentano comunque un primo approccio ad un certo tipo di metodi iterativi per equazioni non lineari.

1 Metodo di bisezione

Il *metodo di bisezione* è il più semplice metodo iterativo per approssimare gli zeri reali di una funzione $f(x)$. L'idea del metodo, intuitivamente, è molto facile; attraverso un approccio divide-et-impera si suddivide un particolare intervallo in cui sappiamo essere contenuta la radice esattamente al centro e successivamente si considera il semi-intervallo in cui abbiamo ragionevole modo di pensare sia contenuta la radice. L'algoritmo procede in questo modo fintanto che non si giunge a una soluzione della nostra equazione non lineare. Per determinare se all'interno di un intervallo è presumibile che ci sia una radice della nostra equazione è sufficiente fare ricorso al teorema di esistenza degli zeri:

Theorem 1.1. (Teorema di esistenza degli zeri) Sia $f(x)$ una funzione continua su un certo intervallo $[a, b]$. Se la funzione assume valori opposti agli estremi dell'intervallo, cioè se $f(a)f(b) < 0$, allora esiste sicuramente un punto $c \in]a, b[$ tale per cui $f(c) = 0$.

Quindi, una volta determinato l'intervallo $[a, b]$ sui cui sappiamo essere vero l'enunciato del teorema di esistenza degli zeri, si procede a suddividerlo in due intervalli $[a, c_1]$ e $[c_1, b]$. A questo punto si verifica la validità del teorema di esistenza degli zeri sui due intervalli e scegliendo come intervallo di riferimento quello per cui il teorema è valido. L'algoritmo si ferma nel momento in cui si trova la radice, cioè, quando si procede nel valutare il prodotto tra il nuovo estremo e i due vecchi estremi dell'intervallo, si ottiene zero. Il passo del metodo può essere espresso facilmente nel seguente modo:

$$x_{n+1} = \frac{x_n + \hat{x}_n}{2}, \quad \hat{x}_n = \begin{cases} x_{n-1} & f(x_n)f(x_{n-1}) < 0 \\ \hat{x}_{n-1} & \text{altrimenti} \end{cases}$$

Ipotizziamo quindi di prendere una funzione $f(x)$ che sappiamo essere continua almeno sull'intervallo $[a, b]$. Ipotizziamo anche che sia verificata la condizione di esistenza degli zeri, cioè che $f(a)f(b) < 0$ e proviamo a applicare qualche passo dell'algoritmo di **bisezione**. Supponendo che $a = x_0$ e che $b = x_1$, allora l'idea è quella di definire un certo valore di x_1 come

$$x_2 = \frac{x_1 - x_0}{2} \iff [a, b] = [x_0, x_2] \cup [x_2, x_1]$$

A questo punto si prova a verificare la condizione di esistenza degli zeri sugli estremi dei due semi-intervalli, si verifica cioè se $f(a)f(x_2) < 0$ oppure se $f(b)f(x_2) < 0$. Il semi-intervallo su cui deve essere applicato il nuovo passo del metodo sarà l'intervallo che rispetta, per l'appunto, la condizione di esistenza degli zeri. Se si verifica che $f(x_1) = 0$, allora sicuramente x_1 corrisponde a una radice del polinomio ed è quindi inutile proseguire iterando l'algoritmo; questa è la condizione di *stop* principale dell'algoritmo di bisezione. Possiamo anche concludere che, anche se $f(x_1) \neq 0$, è comunque un'approssimazione migliore di quelle precedenti. Iterando un altro passo otteniamo che l'approssimazione x_3 della radice sarà data da:

$$x_3 = \begin{cases} \frac{x_2 + x_1}{2} & f(x_2)f(x_1) < 0 \\ \frac{x_0 + x_2}{2} & f(x_2)f(x_0) < 0 \end{cases} = \frac{x_2 + \hat{x}_2}{2}, \quad \hat{x}_2 = \begin{cases} x_1 & f(x_2)f(x_1) < 0 \\ x_0 & f(x_2)f(x_0) < 0 \end{cases}$$

Possiamo caratterizzare il metodo di bisezione come un metodo a due punti **non stazionario**. Abbiamo infatti definito un metodo come **stazionario** solo quando la sua funzione Φ non cambia di segno; tuttavia, nel caso del metodo di bisezione è possibile prendere intervalli tali per cui la funzione Φ oscilla di segno durante i passi dell'algoritmo. Inoltre, data la sua natura, possiamo anche definire una formula per maggiorare l'errore della formula al passo n -esimo:

$$|x_{n+1} - \alpha| \leq \frac{1}{2^n}(b - a) \rightarrow n \geq \log_2 \left(\frac{b - a}{\varepsilon} \right)$$

2 Metodo delle secanti

Un altro metodo di risoluzione per le equazioni non lineari consiste nel metodo delle **secanti**; questo metodo è fondamentalmente una semplificazione del metodo di newton (che vedremo più

avanti) e che rimuove la necessità di calcolare la derivata prima della funzione, andando a sostituirla con il rapporto incrementale di due punti x_0, x_1 . Ipotizzando quindi di avere una funzione $f(x)$ continua su un certo intervallo, e due punti iniziali x_0, x_1 tali per cui $x_0 < \alpha < x_1$, allora è possibile approssimare le radici utilizzando il metodo delle secanti con funzione di iterazione:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

In termini generali, se x_0, x_1 sono delle buone scelte di punti iniziali, il metodo garantisce sempre la convergenza con un determinato ordine. In particolare, vale il seguente risultato:

Theorem 2.1. (Teorema di Jeeves) Se $f \in C^2([a, b])$ con la radice $\alpha \in [a, b]$ e $f'(\alpha) \neq 0$, allora se x_0, x_1 sono sufficientemente vicine ad α le iterate del metodo delle secanti convergono ad α , con ordine di convergenza pari alla **sezione aurea**.

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.62$$

Se il metodo di bisezione aveva una caratterizzazione molto intuitiva ed immediata, il metodo delle secanti ha un'interpretazione geometrica molto elegante. L'idea del metodo è quella di tracciare il segmento che congiunge $f(x_0)$ e $f(x_1)$; il nuovo punto corrisponde al punto dove il segmento tracciato interseca l'asse delle ascisse. A questo punto, determinato x_2 è possibile procede facendo la stessa cosa: si traccia la secante tra $f(x_1)$ e $f(x_2)$, l'intersezione con l'asse delle ascisse sarà il nuovo punto.

3 Metodi iterativi ad un punto

Fino ad ora abbiamo visto metodi iterativi molto semplici, che sfruttano due diversi punti per calcolare le iterazioni del metodo. Possiamo tuttavia anche definire dei metodi che utilizzano un singolo punto, cioè dei metodi iterativi dove il valore del punto x_{n+1} dipende esclusivamente dal valore di x_n valutato in una specifica funzione di iterazione Φ . In particolare, data l'equazione $f(x) = 0$, si può sempre costruire una funzione $\Phi(x)$ tale che l'equazione iniziale sia equivalente alla seguente:

$$x = \Phi(x) \rightarrow x_{n+1} = \Phi(x_n)$$

Questi metodi sono detti **metodi iterativi ad un punto**. Un primo esempio di costruzione di un metodo iterativo ad un punto prevede di scegliere la funzione di iterazione come $\Phi(x) = x - \hat{f}(x)g(x)$, dove $g(x)$ è una funzione arbitraria e $\hat{f}(x)$ è una funzione che si annulla in tutti i punti corrispondenti a zeri di $f(x)$. Questi metodi vengono anche detti **metodi di punto fisso**, in quanto, ipotizzando che $x = \alpha$ sia una radice di $f(x)$ si ottiene:

$$\Phi(\alpha) = \alpha - g(\alpha)f(\alpha) = \alpha$$

In questi metodi, quindi, cercare le radici di una generica equazione corrisponde a determinare i punti fissi di α . All'inizio del capitolo avevamo detto che esistono due problemi relativamente a ogni metodo iterativo per equazioni lineari: **convergenza** del metodo e **ordine di convergenza** del metodo. Infatti, quando definiamo un particolare metodo α dobbiamo essere sicuri che converga e, ammesso che lo faccia, quanto veloce è in grado di farlo. Non è infatti sempre detto che un metodo converga o che lo faccia con una velocità tale che sia ottimale per i nostri obiettivi. Esiste un teorema, a tal proposito, generale, che definisce una serie di condizioni sufficienti per la convergenza locale del metodo:

Theorem 3.1. (Teorema di convergenza locale) Sia α un punto fisso di $\Phi(x)$ interno ad un intervallo $\mathcal{I}$ sul quale $\Phi(x)$ sia **derivabile** con continuità e si supponga che esistano due numeri positivi ρ e K con $K < 1$, tali che $\forall x \in [\alpha - \rho, \alpha + \rho] \subset \mathcal{I}$ si verifichi la condizione

$$|\Phi'(x)| \leq K$$

allora per $\forall x_0 \in [\alpha - \rho, \alpha + \rho]$ la successione x_n è sempre contenuta all'interno dell'intervallo $[\alpha - \rho, \alpha + \rho]$, esiste una sola radice nell'intervallo $\alpha \in [\alpha - \rho, \alpha + \rho]$ e la successione x_n converge sempre alla suddetta radice α .

Dimostrazione. Il modo migliore per dimostrare questo teorema è analizzare singolarmente le tre diverse implicazioni:

- i) **Invarianza dell'intervallo:** Si dimostra per induzione. Scegliamo un $x_0 \in [\alpha - \rho, \alpha + \rho]$ come passo base. Come passo induttivo, ammettiamo per ipotesi che valga $|x_n - \alpha| \leq \rho$ per un certo n , e dimostriamo che x_{n+1} è anch'esso contenuto nel medesimo intervallo. Usando il Teorema di Lagrange:

$$|x_{n+1} - \alpha| = |\Phi(x_n) - \Phi(\alpha)| = |\Phi'(\xi)| \cdot |x_n - \alpha|, \quad \xi \in [x_n, \alpha]$$

Essendo per ipotesi $|\Phi'(\xi)| \leq K$, otteniamo:

$$|\Phi'(\xi)| \cdot |x_n - \alpha| \leq K \cdot |x_n - \alpha| \leq K \cdot \rho < \rho$$

Quindi $|x_{n+1} - \alpha| < \rho$, il che ci permette di concludere che $x_{n+1} \in [\alpha - \rho, \alpha + \rho]$.

- ii) **Convergenza:** Per dimostrare che il metodo converge, riprendiamo l'espansione del termine $|x_{n+1} - \alpha|$:

$$|x_{n+1} - \alpha| = |\Phi(x_n) - \Phi(\alpha)| = |\Phi'(\xi)| \cdot |x_n - \alpha| \leq K \cdot |x_n - \alpha|$$

Procedendo per ricorrenza ed espandendo allo stesso modo i termini $x_n, x_{n-1}, \dots, x_1$, otteniamo la relazione:

$$|x_{n+1} - \alpha| \leq K^{n+1} \cdot |x_0 - \alpha| \leq K^{n+1} \cdot \rho$$

Passando al limite per $n \rightarrow \infty$, poiché $K \in (0, 1)$, si ha che $K^{n+1} \cdot \rho \rightarrow 0$. Siccome la distanza tra x_{n+1} e α tende a zero, possiamo concludere che la successione generata dal metodo converge al punto fisso.

- iii) **Unicità:** Dimostriamo l'unicità per assurdo. Ipotizziamo che esistano due punti fissi distinti α e $\tilde{\alpha}$ all'interno dell'intervallo $[\alpha - \rho, \alpha + \rho]$. Allora vale che:

$$|\alpha - \tilde{\alpha}| = |\Phi(\alpha) - \Phi(\tilde{\alpha})| = |\Phi'(\xi)| \cdot |\alpha - \tilde{\alpha}| \leq K \cdot |\alpha - \tilde{\alpha}|$$

dove ξ è un punto compreso tra α e $\tilde{\alpha}$. Siccome $\alpha \neq \tilde{\alpha}$, la quantità $|\alpha - \tilde{\alpha}|$ è strettamente maggiore di zero. Dividendo entrambi i membri per $|\alpha - \tilde{\alpha}|$ otterremo $1 \leq K$. Poiché l'ipotesi iniziale impone $K < 1$, si è formata una palese contraddizione. Possiamo quindi concludere che non possono esistere altri punti fissi all'interno dell'intervallo.

□

Il teorema definisce un risultato fondamentale, che torna molto spesso utile anche nelle prove d'esame. Infatti, è sufficiente che sia verificata una proprietà relativamente semplice, cioè che la derivata prima del metodo sia minore di 1, affinché il metodo converga. In tanti esercizi delle prove d'esame viene chiesto, per l'appunto, di verificare se un metodo converge una volta determinate le radici dell'equazione; risolvere un esercizio di questo tipo vuol dire sostanzialmente determinare se la derivata della funzione di iterazione del metodo è minore di 1 valutandola nella radice. La relazione che lega il fatto che se $|\Phi'(\alpha)| < 1$, allora la convergenza del metodo è locale, è facilmente dimostrabile attraverso il calcolo di $|x_{n+1} - \alpha|$:

$$|x_{n+1} - \alpha| = |\Phi(x_n) - \Phi(\alpha)| = |\Phi'(\xi)| \cdot |x_n - \alpha| < |x_n - \alpha|$$

Si nota immediatamente che se $|\Phi'(x)| < 1$, allora l'errore assoluto dell'approssimazione, definito come la differenza tra l'approssimazione x_{n+1} e la radice α , non viene amplificato, ma viene anzi decrementato, rendendo di fatto il metodo convergente; ovviamente supposto che la scelta del punto iniziale sia coerente con l'enunciato del teorema sulla convergenza. Se infatti la scelta del punto iniziale non viene fatta in modo opportuno è probabile che il metodo non risulti convergente, ma altresì divergente.

D'altra parte, il concetto stesso di convergenza non è di fatto univoco; ipotizziamo infatti di prendere un'equazione $f(x) = 0$ e un certo intervallo $[a, b]$ tale per cui, detta α radice di $f(x)$, risulta che $\alpha \in]a, b[$ (centrato nell'intervallo). Supponiamo ora di prendere un punto x_0 iniziale tale per cui il metodo sicuramente converge, allora iterando il metodo si possono ottenere due tipi di convergenza:

- Convergenza **monotona**. Se $\Phi'(x) > 0$ nell'intervallo $[\alpha - \rho, \alpha + \rho]$ allora vuol dire che il punto x_{n+1} sarà contenuto sempre all'interno dell'intervallo $[x_n, \alpha]$ (o $[\alpha, x_n]$)
- Convergenza **alternata**. Se $\Phi'(x) < 0$ nell'intervallo $[\alpha - \rho, \alpha + \rho]$ allora il punto x_{n+1} si troverà sempre nel semi-intervallo opposto rispetto al valore precedente x_n .

Fino ad ora abbiamo però introdotto un teorema relativo alla convergenza del metodo, cioè un teorema che garantisce solo che il teorema converge, ma non quanto velocemente lo faccia. Infatti, come detto innumerevoli volte, non è sufficiente sapere se un metodo converge, ma anche con quale velocità lo fa; in termini matematici, è fondamentale conoscere anche l'**ordine di convergenza**. Esiste anche in questo caso un teorema relativo proprio al calcolo dell'ordine di convergenza di un metodo iterativo:

Theorem 3.2. (Ordine di convergenza) Sia $\Phi(x) \in C^p(I)$ in un intorno I di α , dove $\alpha \in I$ è un punto fisso di $\Phi(x)$ ($\Phi(\alpha) = \alpha$). La successione definita da $x_{n+1} = \Phi(x_n)$ ha ordine di convergenza esattamente pari a p se e solo se le derivate fino alla $(p-1)$ -esima si annullano in α e la derivata p -esima è diversa da zero.

$$\text{Ordine di convergenza } p \iff \Phi'(\alpha) = \dots = \Phi^{(p-1)}(\alpha) = 0 \quad \text{e} \quad \Phi^{(p)}(\alpha) \neq 0$$

Dimostrazione. Prima di analizzare le due implicazioni, ricordiamo una proprietà fondamentale dei limiti derivante dalla definizione stessa di ordine di convergenza p : se la successione ha ordine p , allora esiste una costante $C \neq 0$ tale per cui $\lim_{n \rightarrow \infty} \frac{|x_{n+1} - \alpha|}{|x_n - \alpha|^p} = C$. Di conseguenza, scelto un qualsiasi esponente intero r strettamente minore dell'ordine di convergenza ($1 \leq r < p$), si ha sempre che:

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - \alpha|}{|x_n - \alpha|^r} = \lim_{n \rightarrow \infty} \left(\frac{|x_{n+1} - \alpha|}{|x_n - \alpha|^p} \cdot |x_n - \alpha|^{p-r} \right) = C \cdot 0 = 0$$

poiché il termine $|x_n - \alpha|^{p-r}$ tende a zero avendo esponente positivo ($p-r > 0$). Dimostriamo ora le due implicazioni:

($\implies$) Se la successione ha ordine di convergenza pari a p , allora dimostriamo che tutte le derivate della funzione di iterazione valutate in α si annullano fino all'ordine $p-1$. Procediamo per induzione sull'indice della derivata r , per ogni $1 \leq r \leq p-1$: Per il passo base ($r=1$), sviluppiamo $\Phi(x_n)$ con la formula di Taylor arrestata all'ordine 0 con resto di Lagrange nell'intorno di α :

$$x_{n+1} = \Phi(x_n) = \Phi(\alpha) + \Phi'(\xi_n)(x_n - \alpha)$$

Sottraendo $\alpha = \Phi(\alpha)$ a entrambi i membri e dividendo per $(x_n - \alpha)$, si ottiene il rapporto $\frac{x_{n+1} - \alpha}{x_n - \alpha} = \Phi'(\xi_n)$. Passando al limite per $n \rightarrow \infty$ (da cui $\xi_n \rightarrow \alpha$), il membro sinistro deve fare 0 per la proprietà sui limiti vista in precedenza (essendo il denominatore di grado $1 < p$), mentre il membro destro tende a $\Phi'(\alpha)$ per continuità. Ricaviamo quindi $\Phi'(\alpha) = 0$. Per il passo induttivo, supponiamo che le derivate siano nulle fino all'ordine $r-1$, ovvero $\Phi'(\alpha) = \dots = \Phi^{(r-1)}(\alpha) = 0$ con $r < p$. Sviluppando $\Phi(x_n)$ con Taylor fino all'ordine $r-1$ con resto di Lagrange di ordine r otteniamo:

$$x_{n+1} = \Phi(x_n) = \Phi(\alpha) + \sum_{i=1}^{r-1} \frac{\Phi^{(i)}(\alpha)}{i!} (x_n - \alpha)^i + \frac{\Phi^{(r)}(\xi_n)}{r!} (x_n - \alpha)^r$$

Grazie all'ipotesi induttiva, tutti i termini della sommatoria centrale svaniscono. Sottraendo α e dividendo per $(x_n - \alpha)^r$ l'equazione si riduce a $\frac{x_{n+1} - \alpha}{(x_n - \alpha)^r} = \frac{\Phi^{(r)}(\xi_n)}{r!}$. Passando al limite per $n \rightarrow \infty$, il membro sinistro fa zero poiché l'esponente al denominatore soddisfa la condizione $r < p$, implicando direttamente che $\frac{\Phi^{(r)}(\alpha)}{r!} = 0$, da cui $\Phi^{(r)}(\alpha) = 0$. Per il principio di induzione abbiamo provato che tutte le derivate fino a $p-1$ sono nulle. Il fatto che $\Phi^{(p)}(\alpha) \neq 0$ discende per assurdo dalla definizione stessa di ordine di convergenza, poiché se fosse zero l'ordine effettivo della successione sarebbe pari almeno a $p+1$, contraddicendo l'ipotesi iniziale.

($\Leftarrow$) Se assumiamo che $\Phi'(\alpha) = \dots = \Phi^{(p-1)}(\alpha) = 0$ e $\Phi^{(p)}(\alpha) \neq 0$, allora dimostriamo che l'ordine di convergenza risultante della successione è proprio p . Sviluppiamo la funzione del punto fisso $\Phi(x_n)$ in serie di Taylor nell'intorno di α , arrestandoci all'ordine $p - 1$ con il resto di Lagrange di ordine p :

$$x_{n+1} = \Phi(x_n) = \Phi(\alpha) + \sum_{i=1}^{p-1} \frac{\Phi^{(i)}(\alpha)}{i!} (x_n - \alpha)^i + \frac{\Phi^{(p)}(\xi_n)}{p!} (x_n - \alpha)^p$$

Tenendo conto che $\Phi(\alpha) = \alpha$ e applicando l'ipotesi di annullamento di tutte le derivate fino alla $(p - 1)$ -esima, l'espressione si riduce a $x_{n+1} - \alpha = \frac{\Phi^{(p)}(\xi_n)}{p!} (x_n - \alpha)^p$. Dividendo per $(x_n - \alpha)^p$ e passando al modulo e al limite per $n \rightarrow \infty$ (per cui $x_n \rightarrow \alpha$ implica $\xi_n \rightarrow \alpha$), per via della continuità della derivata p -esima si ottiene:

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - \alpha|}{|x_n - \alpha|^p} = \lim_{n \rightarrow \infty} \frac{|\Phi^{(p)}(\xi_n)|}{p!} = \frac{|\Phi^{(p)}(\alpha)|}{p!}$$

Poiché per ipotesi la quantità $\Phi^{(p)}(\alpha)$ è diversa da zero, il limite del rapporto degli errori converge a una costante asintotica non nulla e finita, soddisfacendo pienamente la definizione di ordine di convergenza pari a p .

□

In un certo senso è possibile fare un'analogia con i metodi di quadratura visti per gli integrali; infatti, in quel caso avevamo definito il **grado di precisione** della formula di quadratura come quel valore di m tale per cui l'errore sul $E(1) = E(x) = \dots = E(x^m) = 0$ e $E(x^{m+1}) \neq 0$. Le differenze rispetto al caso integrale stanno principalmente nel fatto che si utilizzano le derivate e che si deve traslare tutto di un esponente in meno, infatti una formula ha ordine di convergenza p se $\Phi^{(1)}(\alpha) = \Phi^{(2)}(\alpha) = \dots = \Phi^{(p-1)}(\alpha) = 0$ e $\Phi^{(p)}(\alpha) \neq 0$. Anche questo risultato è estremamente utile per risolvere esercizi d'esame, è infatti sufficiente procedere con il calcolo delle divisioni successive della formula, valutando poi il risultato per $x = \alpha$ e fermandosi solo nel momento in cui si trova un valore diverso da zero. L'ordine di derivazione per cui la valutazione della funzione di iterazione del metodo è diversa da zero corrisponde proprio all'ordine di convergenza del metodo stesso.

Convergenza globale

Uno dei problemi che ci siamo portati dietro fino ad ora relativo proprio alla convergenza dei metodi iterativi, è il fatto che il teorema di convergenza garantisce fondamentalmente solo una convergenza **locale** del metodo. Nella pratica questa proprietà rappresenta un problema non indifferente, infatti è necessario determinare preliminarmente un punto iniziale che sia sufficientemente vicino alla radice, aumentando il costo necessario per poter iterare il metodo. In alcuni casi, nello specifico, per funzioni di iterazioni del metodo che rispettano una determinata proprietà, esiste un teorema che garantisce proprio la convergenza "globale" (anche se in senso molto lato) del metodo:

Theorem 3.3. (Teorema per la convergenza globale) Se $f(x) \in C^1([a, b])$, $\Phi(x) \in [a, b] \forall x \in [a, b]$ e $|\Phi'(x)| < 1$ allora esiste un unico punto fisso $\alpha \in [a, b]$ e il metodo iterativo $x_{n+1} = \Phi(x_n)$ converge $\forall x_0 \in [a, b]$.

Si nota immediatamente che rispetto ai teoremi visti in precedenza, questo teorema garantisce una convergenza quantomeno estesa ad un intervallo molto più ampio rispetto all'interno della radice, rendendo di fatto più semplice risolvere il problema derivato dalla necessità di trovare un punto iniziale che sia sufficientemente vicino alla radice.

4 Metodo di newton

Tra i metodi iterativi, il più conosciuto e forse anche quello che ricopre una maggiore importanza, è il metodo di **Newton**. L'idea di questo metodo è quella di applicare l'iterazione $\Phi(x) = x - g(x)f(x)$ andando a utilizzare come funzione $g(x)$ l'inverso della derivata di $f(x)$. Ovviamente, un metodo

di questo tipo richiede che esista un certo intorno di α , dove α è lo zero dell'equazione, tale per cui la funzione è **derivabile** con **continuità** in tutto l'intorno. Definiamo quindi la funzione di iterazione e il metodo iterativo come:

$$\Phi(x) = x - \frac{f(x)}{f'(x)} \rightarrow x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

L'iterata x_{n+1} è individuata, geometricamente, dal punto di incontro dell'asse delle ascisse con la tangente alla curva $y = f(x)$ nel punto $(x_n, f(x_n))$. Anche in questo caso l'interpretazione intuitiva è infatti molto semplice, è sufficiente che ad ogni passo si consideri il punto $(x_n, f(x_n))$, si tracci la tangente a quel particolare punto per la funzione $f(x)$ e si determini dove quest'ultima interseca l'asse delle ascisse, restituendo in uscita il punto successivo. È importante notare che il metodo delle secanti è una semplificazione del metodo di Newton, dove al posto della derivata si pone il rapporto incrementale tra due punti. La particolarità del metodo di Newton, che è anche ciò che lo rende così appetibile, deriva dai suoi teoremi di convergenza:

Theorem 4.1. (Convergenza del Metodo di Newton) Sia $f \in C^2([a, b])$ e sia $\alpha \in (a, b)$ tale che $f(\alpha) = 0$. Se $f'(\alpha) \neq 0$ (ovvero α è una radice semplice), allora esiste un intorno $I = [\alpha - \rho, \alpha + \rho] \subseteq [a, b]$ tale che, per ogni punto iniziale $x_0 \in I$, la successione generata dal metodo di Newton $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ converge ad α con ordine di convergenza almeno quadratico ($p \geq 2$). Inoltre, vale la seguente relazione asintotica:

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - \alpha|}{|x_k - \alpha|^2} = \left| \frac{f''(\alpha)}{2f'(\alpha)} \right|$$

Dimostrazione. Il metodo di Newton può essere visto come un metodo di iterazione di punto fisso $x_{k+1} = \Phi(x_k)$, dove la funzione di iterazione è definita come $\Phi(x) = x - \frac{f(x)}{f'(x)}$. Per determinare l'ordine di convergenza, calcoliamo le derivate di $\Phi(x)$ e valutiamole nel punto fisso α . Dimostriamo i due aspetti del teorema:

- **(Ordine di convergenza)** Calcoliamo la derivata prima $\Phi'(x)$ applicando la regola di derivazione del quoziente al termine frazionario:

$$\Phi'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{(f'(x))^2} = 1 - \frac{(f'(x))^2 - f(x)f''(x)}{(f'(x))^2} = \frac{f(x)f''(x)}{(f'(x))^2}$$

Poiché per ipotesi α è una radice di $f(x)$ (ossia $f(\alpha) = 0$) e $f'(\alpha) \neq 0$, valutando la derivata nel punto fisso si annulla il numeratore: $\Phi'(\alpha) = 0$. Essendo $\Phi'(\alpha) = 0$, il teorema sull'ordine di convergenza ci garantisce che la successione converge localmente con ordine almeno $p = 2$ (superlineare).

- **(Fattore asintotico)** Per ricavare la relazione del limite, dobbiamo studiare la derivata seconda di $\Phi(x)$. Applicando nuovamente la regola del quoziente a $\Phi'(x)$ si ottiene:

$$\Phi''(x) = \frac{[f'(x)f''(x) + f(x)f'''(x)](f'(x))^2 - [f(x)f''(x)] \cdot 2f'(x)f''(x)}{(f'(x))^4}$$

Valutando $\Phi''(x)$ in α e ricordando che $f(\alpha) = 0$, tutti i termini in cui compare $f(\alpha)$ si annullano, semplificando drasticamente l'espressione:

$$\Phi''(\alpha) = \frac{[f'(\alpha)f''(\alpha) + 0](f'(\alpha))^2 - 0}{(f'(\alpha))^4} = \frac{(f'(\alpha))^3 f''(\alpha)}{(f'(\alpha))^4} = \frac{f''(\alpha)}{f'(\alpha)}$$

Sviluppiamo ora $\Phi(x_k)$ in serie di Taylor centrata in α , arrestandoci all'ordine 1 con resto di Lagrange di ordine 2:

$$x_{k+1} = \Phi(x_k) = \Phi(\alpha) + \Phi'(\alpha)(x_k - \alpha) + \frac{\Phi''(\xi_k)}{2}(x_k - \alpha)^2$$

Sapendo che $\Phi(\alpha) = \alpha$ e $\Phi'(\alpha) = 0$, l'equazione diventa $x_{k+1} - \alpha = \frac{\Phi''(\xi_k)}{2}(x_k - \alpha)^2$. Dividendo entrambi i membri per $(x_k - \alpha)^2$, passando al modulo e calcolando il limite per $k \rightarrow \infty$ (che implica $\xi_k \rightarrow \alpha$), otteniamo infine:

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - \alpha|}{|x_k - \alpha|^2} = \lim_{k \rightarrow \infty} \frac{|\Phi''(\xi_k)|}{2} = \frac{|\Phi''(\alpha)|}{2} = \left| \frac{f''(\alpha)}{2f'(\alpha)} \right|$$

il che conclude la dimostrazione. Se $f''(\alpha) \neq 0$ l'ordine è esattamente 2, se $f''(\alpha) = 0$ l'ordine sarà strettamente maggiore di 2.

□

Anche in questo caso ci troviamo di fronte al solito problema che abbiamo ampiamente discusso in precedenza, infatti anche in questo caso la convergenza è garantita al netto di una scelta corretta del punto iniziale x_0 , che deve essere sufficientemente vicino alla radice per fare in modo che il metodo converga. Tuttavia questo metodo viene anche caratterizzato da un secondo teorema di convergenza, che garantisce una convergenza altresì globale:

Theorem 4.2. (Teorema di convergenza globale del metodo di Newton) Se $\alpha \in [a, b] \subset \mathbb{R}$ radice semplice di $f \in C^2([a, b])$ e se $f'(x)$ e $f''(x)$ hanno segno costante in $[a, b]$ allora il metodo di Newton converge $\forall x_0 \in [a, b]$ tale per cui $f(x_0)f''(x_0) > 0$.

Nel caso specifico del metodo di Newton possiamo dare una caratterizzazione ancora diversa; in particolare, si possono garantire le ipotesi necessarie affinché il teorema sia valido studiando attentamente il segno di $f(x)$ e delle sue derivate prime e seconde: si osserva come se f è concava, il metodo di newton converge partendo da un punto x_0 in cui $f(x_0) < 0$, viceversa, il metodo di newton converge partendo da un punto x_0 in cui $f(x_0) > 0$

Radici multiple

Un caso particolare di equazioni $f(x) = 0$ è rappresentato da tutti quei casi in cui le radici di una particolare equazione sono multiple, si presentano cioè con una molteplicità algebrica $r > 1$. Esistono diverse motivazioni per cui il caso delle radici multiple è fondamentalmente problematico, la prima di queste motivazioni deriva proprio dal fatto che se $f \in C^r([a, b])$ e se α è radice di ordine r allora tutte le derivate della funzione $f(x)$ fino all'ordine r valutate in α valgono esattamente zero. Questo fatto è fortemente problematico in quanto limita la velocità di convergenza del metodo, che non è più quadratica, ma bensì lineare; la condizione per la convergenza quadratica è che $f(\alpha) = 0$ e che $f'(\alpha) \neq 0$, tuttavia però le proprietà delle radici multiple ci dicono esplicitamente che $f'(\alpha) = 0$. L'idea che dobbiamo seguire è quindi quella di ridefinire il metodo; ipotizzando di conoscere la molteplicità della particolare radice si ha la seguente variante:

$$x_{n+1} = x_n - r \frac{f(x_n)}{f'(x_n)}$$

Il metodo così ottenuto permette di ripristinare la convergenza quadratica del metodo.

CAPITOLO 10

SISTEMI NON LINEARI

Uno dei possibili modi con cui è possibile generalizzare la teoria che abbiamo visto in questo momento è quella dei sistemi **non lineari**, cioè sistemi di equazioni della forma $f_i(x) = 0$, dove $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$, i cui termini non sono lineari. In particolare, possiamo definire il nostro sistema di equazioni in forma alternativa come una generica funzione $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ di cui vogliamo determinare il vettore $x \in \mathbb{R}^n$ tale per cui $F(x) = 0$:

$$\begin{cases} f_1(\alpha_0, \dots, \alpha_n) = 0 \\ f_2(\alpha_0, \dots, \alpha_n) = 0 \\ \vdots \\ f_n(\alpha_0, \dots, \alpha_n) = 0 \end{cases}$$

Rispetto al capitolo scorso, cioè quello in cui venivano trattate le equazioni non lineari, quando si lavora con **sistemi di equazioni non lineari** è necessario generalizzare i concetti relativi a **convergenza** e **ordine di convergenza**. Infatti, le definizioni usate per le equazioni lavorano con scalari e non con vettori, è fondamentale quindi adattare entrambe le definizioni dello scorso capitolo usando la notazione matriciale, sostituendo cioè i **valori assoluti** con le **norme dei vettori**:

Definition 0.1. (Successione convergente e metodo convergente) La successione $\{x_k\}_{k \in \mathbb{N}}$ converge ad α se e solo se

$$\lim_{k \rightarrow \infty} \|x_k - \alpha\|_2 = 0$$

inoltre la convergenza si dice di ordine p se e solo se

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - \alpha\|_2}{\|x_k - \alpha\|_2^p} = C$$

con $C < +\infty$ e $C \neq 0$.

Sempre in analogia con quanto detto nel capitolo scorso, il sistema appena introdotto può essere scritto in una forma equivalente, la quale consente di approssimare una soluzione α come punto fisso di una opportuna funzione di iterazione $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$ scritta nella forma:

$$\Phi(x) = x - G(x)f(x)$$

dove $x \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$ e $G(x)$ è una matrice $n \times n$, **non singolare** almeno in un **intorno** della radice $\alpha \in \mathbb{R}^n$. Questa matrice può essere vista come una particolare matrice della forma:

$$\begin{bmatrix} G_{11}(x) & \dots & G_{1n}(x) \\ \vdots & \ddots & \vdots \\ G_{n1}(x) & \dots & G_{nn}(x) \end{bmatrix}$$

Il terzo concetto che abbiamo ancora da generalizzare per questi sistemi è quello di convergenza. Infatti, se nel caso delle equazioni non lineari, avevamo una funzione di iterazione $\Phi(x)$, la cui derivata era a sua volta una funzione $\Phi'(x)$ in una singola variabile, ora abbiamo una situazione leggermente diversa. Per come abbiamo infatti generalizzato il concetto di funzione di iterazione, possiamo dimostrare che la funzione Φ non è altro che una matrice e per cui il concetto di convergenza usato fino a questo momento non è più coerente. Il teorema di convergenza per sistemi non lineari corrisponde quindi a:

Theorem 0.1. (Teorema di convergenza per **sistemi non lineari**) Sia $\Phi \in C^1(\Omega)$ con $\Omega \subset \mathbb{R}^n$ e sia $\alpha \in \Omega$ tale che $\Phi(\alpha) = \alpha$, se $\rho(J\Phi(\alpha)) < 1$ allora $\exists \delta > 0 : \forall x_0$ con $\|x_0 - \alpha\|_2 < \delta$ la successione

$$x_{n+1} = \Phi(x_n)$$

risulta convergente. Inoltre, se esiste un $k \in (0, 1)$ tale per cui $\|J\Phi(x)\|_2 \leq k$ allora sono valide le stesse considerazioni del caso $\rho(J\Phi(\alpha)) < 1$ e il metodo converge.

Quindi, affinché la convergenza del metodo sia garantita è fondamentalmente possibile verificare una tra queste due condizioni, o che in un intorno di α la norma della matrice sia minore di uno o che per $x = \alpha$ il raggio spettrale della matrice è minore di 1.

1 Metodo di Newton-Raphson

L'estensione naturale del metodo di Newton nel caso di $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ è il metodo di **Newton-Raphson**. L'idea, del tutto analoga a quella del metodo di Newton classico, è che se $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ sono derivabili con continuità e $Jf(x)$ è non singolare in un intorno Ω della radice α allora la successione $x_{n+1} = \Phi(x_n)$ può essere scritta come

$$x_{n+1} = x_n - [Jf(x_n)]^{-1} \cdot f(x_n)$$

Il problema principale dell'estensione dei metodi iterativi al dominio di $\mathbb{R}^n$ è che cambia anche la definizione di derivata, a cui si sostituisce quella di **matrice Jacobiana** di una funzione. Data una funzione $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$, si definisce **matrice jacobiana** di $f(x)$ la matrice $Jf(x) \in \mathbb{R}^{n \times n}$ tale per cui:

$$Jf(x) = \begin{bmatrix} \frac{\partial f_1(x)}{\partial x_1} & \cdots & \frac{\partial f_n(x)}{\partial x_1} \\ \vdots & & \vdots \\ \frac{\partial f_1(x)}{\partial x_n} & \cdots & \frac{\partial f_n(x)}{\partial x_n} \end{bmatrix}$$

Quindi, una volta calcolata il **Jacobiano**, diventa possibile implementare il metodo di **Newton-Raphson**. Tuttavia però, la definizione di $\Phi(x_k)$ richiede che venga calcolato l'inverso del **jacobiano**, che è un'operazione alquanto complessa, specie per ordini della matrici molto grandi. Nella pratica, infatti, non si inverte mai la matrice **Jacobiana**, è preferibile infatti risolvere prima il sistema $Jf(x_n)y = f(x_n)$ e successivamente calcolare il passo del metodo. Così come per il metodo di **Newton**, anche nel metodo di **Newton-Raphson** troviamo un teorema di convergenza locale:

Theorem 1.1. (Teorema di Convergenza locale) Se $f \in C^2(\Omega)$, con $\Omega \subseteq \mathbb{R}^n$, se $\alpha \in \Omega$ è una radice di $f(x)$, quindi $f(\alpha) = 0$, e infine se $Jf(x)$ non è singolare almeno nell'intervallo Ω allora $\exists S \subset \Omega$ e $\beta > 0$ tale per cui la successione $\{x_n\}_{n \in \mathbb{N}}$ del metodo di **Newton-Raphson** soddisfa

$$\|x_{k+1} - \alpha\|_2 \leq \beta \|x_k - \alpha\|_2, \quad \forall x_0 \in S$$

inoltre la convergenza del metodo è superlineare ($p \geq 2$).

Come abbiamo detto, il calcolo del **jacobiano** è già di per se un'operazione relativamente complessa, senza contare fondamentalmente il calcolo dell'inversa del **Jacobiano**. È dunque possibile ottenere dei metodi semplificati affinché sia più facile arrivare a una soluzione senza dover necessariamente passare per il calcolo di questa matrice. Tuttavia è fondamentale tenere conto anche del fatto che per questi metodi iterativi, il teorema di convergenza del metodo **Newton-Raphson** non è più applicabile. Possiamo definire quattro diversi metodi alternativi al metodo di **Newton-Raphson**:

- **Metodo di Newton-Raphson semplificato:** In questa variante il jacobiano della matrice viene valutato una sola volta nel punto iniziale e utilizzato ad ogni iterazione del sistema; in particolare

$$x_{m+1} = x - [Jf(x_0)]^{-1} f(x_m)$$

Facendo in questo modo è anche possibile calcolare la fattorizzazione LU della matrice $Jf(x_0)$ per poi utilizzarla per risolvere il sistema lineare $Jf(x_0)y = f(x_m)$. Il costo complessivo di questa variante diventa quindi pari al costo per risolvere il sistema lineare $O(n^2)$ più il costo di valutare la funzione $f(x)$ nel punto x_m .

- **Metodo di Jacobi-Newton:** In questa variante del metodo di newton invece si costruisce una matrice D che sostituisca la matrice $Jf(x)$ nel calcolo del passo. Questa matrice D sarà diagonale (così da portare il costo per valutare il sistema lineare a $O(m)$) e avrà una forma

$$D = \text{diag}\left(\frac{\partial f_1(x_m)}{\partial x_1}, \dots, \frac{\partial f_m(x_m)}{\partial x_m}\right)$$

Quindi, il costo sarà composto dal costo della risoluzione del sistema e dal costo della valutazione della funzione $f(x)$ nel punto x_m . La formula esplicita risulta quindi essere

$$x_{k+1} = \text{diag}(Jf(x))$$

- **Metodo Gauss-Seidel:** Agisce esattamente come il metodo **Jacobi** per l'aggiornamento della componente $x_1^{(k+1)}$, usando successivamente le componenti di $x_1^{(k+1)}$ già calcolate per calcolare le successive.
- **Metodi quasi newton:** l'idea di tutti questi metodi è quella di sostituire $Jf(x)$ con *qualcosa* che richieda meno valutazioni di funzioni non lineari e/o con cui è più facile risolvere questi sistemi.

Parte VI

Problemi agli autovalori

CAPITOLO 11

METODO DELLE POTENZE

Il problema che viene affrontato durante questo capitolo è il problema del calcolo degli autovalori; data una matrice $A \in \mathbb{C}^{n \times n}$ si cercano di determinare le autocopie (λ, v) di A . Uno degli esempi classici, che giustifica lo studio degli algoritmi per determinare gli autovalori, sono i cosiddetti **problemi agli autovalori** e, tra questi problemi, troviamo anche il famoso algoritmo del **page rank** (di cui si trova un approfondimento nell'appendice). In generale distinguiamo due classi di algoritmi

- i) Algoritmi che permettono di ricavare una singola autocoppia.
- ii) Algoritmi che permettono di trovare tutte le autocopie.

In questo capitolo studieremo la prima categoria di algoritmi, in particolare, studieremo il **metodo delle potenze** e il **metodo delle potenze invertito**. Questi due algoritmi permettono di ricavare, rispettivamente, l'autovalore di modulo massimo e minimo.

1 Metodo delle potenze

Ipotizziamo di avere una matrice $A \in \mathbb{C}^{n \times n}$ che sia anche **diagonalizzabile** con autovalori $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_m|$ e chiamiamo $v_1, \dots, v_m$ gli autovettori associati a tali autovalori. Siccome A è diagonalizzabile, allora abbiamo che l'insieme $\{v_1, \dots, v_m\}$ rappresenta una base per $\mathbb{C}^n$ e allora, preso un vettore $z_0 \in \mathbb{C}^n$ possiamo scrivere

$$z = \sum_{i=1}^m c_i v_i \Rightarrow Az = A \sum_{i=1}^m c_i v_i = \sum_{i=1}^m c_i \underbrace{Av_i}_{\lambda_i v_i} = \sum_{i=1}^m c_i \lambda_i v_i$$

Andando quindi ad applicare la matrice A sul vettore z e sfruttando la definizione di autovalore possiamo far comparire all'interno dell'espressione l'autovalore i -esimo. Se moltiplicassimo per $k \in \mathbb{N}$ volte la matrice A per z si otterrebbe

$$\begin{aligned} A^k z &= \sum_{i=1}^m c_i \lambda_i^k v_i = c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \dots + c_m \lambda_m^k v_m = \\ &= \lambda_1^k \left(c_1 v_1 + c_2 \left[\frac{\lambda_2}{\lambda_1} \right]^k v_2 + \dots + c_m \left[\frac{\lambda_m}{\lambda_1} \right]^k v_m \right) \end{aligned}$$

aumentando le potenze di k i rapporti tra autovalori tendono a diminuire, diventando molto piccoli; per $k \rightarrow \infty$ otteniamo $A^k z = \lambda_1^k (c_1 v_1)$. Esiste un teorema, fondamentale, su cui si basa interamente il metodo delle potenze

Theorem 1.1. (Convergenza del Metodo delle Potenze) Sia $A \in \mathbb{C}^{n \times n}$ una matrice hermitiana ($A = A^H$) e supponiamo che i suoi autovalori, ordinati per modulo decrescente, presentino un autovalore dominante: $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$. Consideriamo un vettore iniziale $z^{(0)} \in \mathbb{C}^n$ tale che non sia ortogonale all'autovettore $v^{(1)}$ associato a λ_1 , ovvero $[v^{(1)}]^H z^{(0)} \neq 0$. Se scegliamo un indice $h \in \{1, \dots, n\}$ tale per cui la coordinata h -esima dell'autovettore dominante sia non nulla ($v_h^{(1)} \neq 0$), allora la successione generata dal metodo delle potenze $z^{(n)} = Az^{(n-1)}$ soddisfa le seguenti proprietà di convergenza:

$$\lim_{n \rightarrow \infty} \frac{z^{(n)}}{z_h^{(n)}} = \tilde{v}^{(1)} \quad \text{e} \quad \lim_{n \rightarrow \infty} \frac{[z^{(n)}]^H A z^{(n)}}{[z^{(n)}]^H z^{(n)}} = \lambda_1$$

dove $\tilde{v}^{(1)}$ è un multiplo dell'autovettore dominante la cui componente h -esima vale 1, e il rapporto a destra è il quoziente di Rayleigh che converge all'autovalore esatto.

Dimostrazione. Procediamo con la dimostrazione analizzando il comportamento della successione $z^{(n)}$ per $n \rightarrow \infty$. Poiché la matrice A è hermitiana, per il teorema spettrale essa è diagonalizzabile e i suoi autovettori $\{v^{(1)}, \dots, v^{(n)}\}$ formano una base ortonormale di $\mathbb{C}^n$. Possiamo quindi esprimere il vettore di partenza $z^{(0)}$ come combinazione lineare di tale base, scrivendo $z^{(0)} = \sum_{j=1}^n c_j v^{(j)}$. I coefficienti di questo sviluppo si ottengono tramite il prodotto scalare, per cui $c_1 = [v^{(1)}]^H z^{(0)}$. L'ipotesi iniziale del teorema ci garantisce proprio che questo prodotto sia diverso da zero, assicurandoci che $c_1 \neq 0$. Valutiamo ora il vettore all' n -esima iterazione. Applicare ripetutamente il metodo delle potenze partendo da $z^{(0)}$ equivale a calcolare $A^n z^{(0)}$. Sfruttando la linearità e la definizione di autovalore, per cui $A^n v^{(j)} = \lambda_j^n v^{(j)}$, otteniamo:

$$z^{(n)} = A^n z^{(0)} = A^n \sum_{j=1}^n c_j v^{(j)} = \sum_{j=1}^n c_j \lambda_j^n v^{(j)}$$

Mettendo in evidenza la potenza dell'autovalore dominante λ_1^n , l'espressione diventa:

$$z^{(n)} = \lambda_1^n \left[c_1 v^{(1)} + \sum_{j=2}^n c_j \left(\frac{\lambda_j}{\lambda_1} \right)^n v^{(j)} \right]$$

Per evitare i problemi computazionali legati al termine λ_1^n , che potrebbe divergere a infinito se in modulo è maggiore di 1, studiamo la successione dividendo l'intero vettore $z^{(n)}$ per la sua h -esima componente $z_h^{(n)}$. Scrivendo il rapporto e passando al limite per $n \rightarrow \infty$, abbiamo:

$$\lim_{n \rightarrow \infty} \frac{z^{(n)}}{z_h^{(n)}} = \lim_{n \rightarrow \infty} \frac{\lambda_1^n \left[c_1 v^{(1)} + \sum_{j=2}^n c_j \left(\frac{\lambda_j}{\lambda_1} \right)^n v^{(j)} \right]}{\lambda_1^n \left[c_1 v_h^{(1)} + \sum_{j=2}^n c_j \left(\frac{\lambda_j}{\lambda_1} \right)^n v_h^{(j)} \right]}$$

I termini λ_1^n a numeratore e denominatore si elidono. Inoltre, per l'ipotesi di autovalore dominante ($|\lambda_1| > |\lambda_j|$ per $j \geq 2$), il rapporto $\left| \frac{\lambda_j}{\lambda_1} \right|$ è strettamente minore di 1, per cui elevandolo a potenza $n \rightarrow \infty$ esso tende a 0, facendo svanire tutte le sommatorie. Resta unicamente:

$$\lim_{n \rightarrow \infty} \frac{z^{(n)}}{z_h^{(n)}} = \frac{c_1 v^{(1)}}{c_1 v_h^{(1)}} = \frac{v^{(1)}}{v_h^{(1)}} = \tilde{v}^{(1)}$$

Questo prova la prima parte della tesi, ovvero che la successione normalizzata converge a un multiplo esatto dell'autovettore dominante. Infine, per dimostrare la convergenza dell'autovalore, consideriamo il quoziente di Rayleigh $R(z^{(n)})$. Trattandosi di un rapporto omogeneo di grado zero, la moltiplicazione o divisione del vettore $z^{(n)}$ per uno scalare non nullo è ininfluenza sul risultato finale:

$$\lim_{n \rightarrow \infty} R(z^{(n)}) = \lim_{n \rightarrow \infty} \frac{[z^{(n)}]^H A z^{(n)}}{[z^{(n)}]^H z^{(n)}} = \lim_{n \rightarrow \infty} \frac{\left[\frac{z^{(n)}}{z_h^{(n)}} \right]^H A \left[\frac{z^{(n)}}{z_h^{(n)}} \right]}{\left[\frac{z^{(n)}}{z_h^{(n)}} \right]^H \left[\frac{z^{(n)}}{z_h^{(n)}} \right]}$$

Sostituendo il limite ricavato in precedenza, per cui il rapporto normalizzato converge a $\tilde{v}^{(1)}$, e ricordando che $A\tilde{v}^{(1)} = \lambda_1 \tilde{v}^{(1)}$, l'espressione si riduce a:

$$\lim_{n \rightarrow \infty} R(z^{(n)}) = \frac{[\tilde{v}^{(1)}]^H (A\tilde{v}^{(1)})}{[\tilde{v}^{(1)}]^H \tilde{v}^{(1)}} = \frac{[\tilde{v}^{(1)}]^H \lambda_1 \tilde{v}^{(1)}}{[\tilde{v}^{(1)}]^H \tilde{v}^{(1)}} = \lambda_1 \frac{[\tilde{v}^{(1)}]^H \tilde{v}^{(1)}}{[\tilde{v}^{(1)}]^H \tilde{v}^{(1)}} = \lambda_1$$

Si conclude che il quoziente di Rayleigh converge esattamente all'autovalore dominante λ_1 , portando a termine la dimostrazione. $\square$

Uno dei problemi principali del metodo delle potenze è che la successione $z^{(k+1)} = Az^{(k)}$ può soffrire di problemi relativi ad **underflow** o **overflow**, dovuti proprio al fatto che l'elevazione a potenza di una matrice per valori molto grandi dell'esponente può portare ad avere dei vettori considerevolmente grandi o considerevolmente piccoli. L'idea, quando si implementa il codice per iterare questo metodo, è quindi quella di normalizzare il vettore ad ogni passo, dividendolo proprio per la sua norma due. Al netto di ciò, possiamo stimare il costo computazionale dell'algoritmo su tre diversi elementi di costo:

- **Prodotto Matrice-Vettore.** Questo è senza dubbio il passo più costoso dell'algoritmo, il cui costo è circa pari ad $O(n^2)$.
- **Calcolo della norma.** La norma due (o infinito) della matrice non è altro che la somma dei quadrati dei suoi elementi più una radice, il suo costo è quindi pari a $O(n)$.
- **Normalizzazione.** La normalizzazione consiste nel dividere ogni componente del vettore per uno scalare, di conseguenza il costo computazionale è pari a $O(n)$.

Il costo computazionale totale per una singola iterazione è esattamente pari ad $O(n^2)$. Il costo complessivo di tutto l'algoritmo è dunque pari a $K \times O(n^2)$, dove K è il numero delle iterazioni. Uno degli altri problemi derivati dalla natura stessa dell'algoritmo, è quello relativo al criterio di arresto; scegliere infatti un criterio di arresto del tipo

$$\|x^{(k+1)} - x^{(k)}\| \leq \text{tol}$$

Porta ad un problema non trascurabile, che si verifica quando l'autovalore di modulo massimo è negativo in termini di modulo. Se ipotizziamo che la nostra successione sia arrivata a un k tale per cui $x_k \approx c_1 \lambda_1^k v_1$, allora:

$$x^{(k+1)} = \frac{y^{(k+1)}}{\|y^{(k+1)}\|} = \frac{Ax^{(k)}}{\|Ax^{(k)}\|} = \frac{c_1 \lambda_1^{k+1} v_1}{|c_1 \lambda_1^{k+1} v_1|} = \left(\frac{\lambda_1}{|\lambda_1|} \right)^{k+1} \frac{c_1}{|c_1|} v_1$$

Se l'autovettore di modulo massimo è negativo, questa successione è a segni alterni e quindi impone una condizione stop del nostro algoritmo che si basi sulla differenza tra due passi successivi della successione è oltremodo sbagliato. Due criteri di arresto che è invece possibile utilizzare per implementare questo algoritmo sono:

$$\begin{aligned} R(z^{(n)}) - R(z^{(n-1)}) &< \text{Tol} \\ \|Az^{(n)} - R(z^{(n)})z^{(n)}\|_2 &< \text{Tol} \end{aligned}$$

Commenti sulle ipotesi del teorema

Il fatto che A sia **hermitiana** non è una condizione strettamente necessaria, il teorema funziona anche se la matrice è semplicemente **diagonalizzabile**. Inoltre, un'altra proprietà molto interessante del metodo delle potenze è che generando casualmente il vettore $z^{(0)}$, la probabilità che non sia ortogonale all'autovettore v_1 è pari circa a uno.

2 Metodo delle potenze inverse

Una delle possibili varianti del metodo delle potenze è quella che permette, invece di calcolare l'autovalore di modulo massimo, di calcolare l'autovalore di modulo minimo. L'idea alla base di questo metodo deriva dal fatto che, data una generica matrice non singolare A i cui autovalori sono ordinati come $|\lambda_1| \geq |\lambda_2| \geq \dots > |\lambda_n|$ (assumendo che l'autovalore di modulo minimo sia unico), gli autovalori della matrice A^{-1} risultano essere:

$$\frac{1}{|\lambda_n|} > \frac{1}{|\lambda_{n-1}|} \geq \dots \geq \frac{1}{|\lambda_1|}$$

Poiché l'inversa di A condivide gli stessi autovettori di A , è sufficiente applicare il metodo delle potenze classico ad A^{-1} . L'autovalore dominante di A^{-1} sarà $\frac{1}{\lambda_n}$, dal quale si può facilmente ricavare λ_n calcolando il quoziente di Rayleigh direttamente con la matrice originale A , una volta che il vettore ha raggiunto la convergenza:

$$\tilde{\lambda}_n = \frac{[z^{(k)}]^H A z^{(k)}}{[z^{(k)}]^H z^{(k)}}$$

A livello algoritmico, il passo per calcolare la nuova direzione non viene eseguito calcolando esplicitamente la matrice inversa (operazione troppo costosa e numericamente sconsigliata), bensì ponendo $y^{(k+1)} = A^{-1}z^{(k)}$, che equivale a risolvere a ogni iterazione il sistema lineare:

$$A y^{(k+1)} = z^{(k)}$$

Se si risolvesse questo sistema da zero a ogni iterazione, il costo sarebbe di $O(n^3)$ per ogni passo. Per ottimizzare il processo, si utilizza un trucco algebrico fondamentale: si precalcola la fattorizzazione **LU** della matrice A una sola volta prima di avviare il ciclo iterativo.

Questa operazione iniziale costa $O(n^3)$. Tuttavia, grazie a questa fattorizzazione, a ogni iterazione successiva sarà sufficiente effettuare solo una sostituzione in avanti e una all'indietro, abbassando il costo del singolo passo iterativo a $O(n^2)$. Di conseguenza, indicando con K il numero di iterazioni necessarie a convergere, il costo computazionale complessivo del metodo non sarà una proibitiva $O(K \cdot n^3)$, ma si ridurrà a:

$$O(n^3) + O(K \cdot n^2)$$

3 Metodo delle potenze inverse con shift

Un'estensione estremamente utile è il **metodo delle potenze inverse con shift** (o traslazione). L'idea è quella di applicare l'algoritmo non alla matrice di partenza A , ma alla matrice traslata $(A - \mu I)$, dove $\mu \in \mathbb{C}$ è uno scalare scelto opportunamente. Le proprietà dell'algebra lineare garantiscono che questa nuova matrice mantenga inalterati gli autovettori, ma i suoi autovalori subiscano una traslazione, assumendo la forma $(\lambda_i - \mu)$. Andando quindi ad operare con il metodo delle potenze inverse sulla matrice traslata, gli autovalori del problema diventeranno della forma $\frac{1}{\lambda_i - \mu}$. Di conseguenza, l'autovalore dominante che il metodo andrà a isolare sarà quello con il denominatore più piccolo possibile: questo significa che l'algoritmo convergerà infallibilmente verso l'autovalore di A **più vicino in assoluto** al valore dello shift μ . Questo approccio permette di estrarre non solo gli estremi dello spettro, ma qualsiasi autovalore "centrale" di cui si conosca una stima iniziale.

Algoritmicamente, la procedura ricalca in modo quasi identico quella del metodo delle potenze inverse classico: ad ogni passo iterativo k , la nuova direzione viene calcolata risolvendo il sistema lineare:

$$(A - \mu I) y^{(k+1)} = z^{(k)}$$

Esattamente come in precedenza, l'ottimizzazione computazionale si ottiene precalcolando la fattorizzazione **LU** della matrice traslata $(A - \mu I)$ una sola volta all'inizio dell'algoritmo. In questo modo, si sostiene il costo di $O(n^3)$ unicamente fuori dal ciclo, riducendo il costo di ogni singola iterazione a $O(n^2)$ tramite le semplici sostituzioni in avanti e all'indietro.

CAPITOLO 12

METODO QR

Uno dei metodi che permette di calcolare tutti gli autovalori della matrice, a differenza dei metodi delle potenze e delle potenze inverse, è il **metodo QR** per il **calcolo degli autovalori** (da non confondersi con il metodo **QR** per il problema ai minimi quadrati). L'idea del metodo è, in linea teorica, quella di generare una successione di matrici $\{A_n\}$ tali per cui A_n è una matrice **simile** (cioè con medesimi autovalori) alla matrice di partenza. Se portiamo $n \rightarrow \infty$ possiamo definire come A_∞ la matrice:

$$A_\infty = \lim_{n \rightarrow \infty} A_n$$

Come abbiamo visto, il metodo QR porta la matrice A a convergere verso una matrice triangolare superiore A_∞ (che chiameremo T). La particolarità di una matrice triangolare superiore è che i suoi autovalori si leggono banalmente sulla diagonale principale, per cui $\lambda_i = t_{ii}$. Supponiamo che gli autovalori siano tutti distinti, ovvero $t_{ii} \neq t_{jj}$ per $i \neq j$.

1 Calcolo degli autovettori di una matrice triangolare

È particolarmente facile calcolare gli autovettori di questa matrice. Cerchiamo infatti un vettore $v^{(i)}$ che sia soluzione del sistema lineare omogeneo:

$$(T - t_{ii}I)v^{(i)} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

Per risolvere questo sistema, il metodo più efficace è analizzare la struttura della matrice $(T - t_{ii}I)$ partizionandola a blocchi. In particolare, isoliamo la sottomatrice quadrata superiore di dimensione $(i-1) \times (i-1)$ (che chiameremo T_1), la parte superiore della colonna i -esima (che chiameremo z), e dividiamo l'autovettore di conseguenza. Il sistema assume questa forma a blocchi:

$$\begin{bmatrix} T_1 & z & * \\ 0 & 0 & * \\ 0 & 0 & * \end{bmatrix} \begin{bmatrix} v_1 \\ \gamma \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Analizziamo gli elementi di questo partizionamento:

- T_1 è la sottomatrice triangolare superiore $(i-1) \times (i-1)$ che si ottiene prendendo la parte in alto a sinistra di $(T - t_{ii}I)$.
- z è il vettore colonna di dimensione $(i-1)$ contenente gli elementi strettamente sopra la diagonale della i -esima colonna.
- L'elemento diagonale in posizione (i, i) è zero, poiché abbiamo sottratto t_{ii} .

- * indica dei blocchi generici della matrice che non ci interessano, in quanto verranno annullati dai vettori nulli.
- v_1 è la porzione di autovettore (di dimensione $i - 1$) che dobbiamo determinare.
- γ è la componente i -esima dell'autovettore, ed è uno scalare.
- 0 è il vettore nullo che riempie tutte le restanti componenti dell'autovettore (da $i + 1$ fino a n).

Eseguendo il prodotto riga per colonna limitatamente al primo blocco, otteniamo l'equazione vettoriale:

$$T_1 v_1 + \gamma z = 0 \implies T_1 v_1 = -\gamma z$$

Per trovare un autovettore associato (che è definito a meno di una costante moltiplicativa), si pone arbitrariamente lo scalare $\gamma = 1$. L'equazione si semplifica quindi in:

$$T_1 v_1 = -z \implies v_1 = -T_1^{-1} z$$

Resta da calcolarsi come risolvere un sistema $(i - 1) \times (i - 1)$ con matrice triangolare superiore. Trattandosi di un sistema già triangolarizzato, è sufficiente procedere per sostituzione all'indietro (back-substitution), il cui costo computazionale è di $O(i^2)$.

Per trovare tutti gli n autovettori della matrice T , sarà necessario ripetere questo procedimento per ogni autovalore, portando il costo computazionale totale a:

$$O\left(\sum_{i=1}^{n-1} i^2\right) = O(n^3)$$

In conclusione, abbiamo trovato un modo estremamente efficiente per calcolare sia gli autovalori che gli autovettori di A_∞ . Come dimostrato nei passaggi precedenti del metodo QR, per ottenere le approssimazioni degli autovettori della matrice originaria A , sarà sufficiente moltiplicare gli autovettori di T appena trovati per la matrice di trasformazione totale accumulata, ottenendo l'autovettore finale $u = \tilde{Q}v^{(i)}$.

2 Come si definisce la successione $\{A_n\}$

All'inizio del capitolo abbiamo definito la successione $\{A_n\}$ come una successione di matrici tali per cui, ogni singola matrice originata dalla successione è una matrice simile ad A . Tuttavia, occorre chiedersi come sia effettivamente definita questa successione in termini pratici; ipotizziamo quindi di scegliere come punto iniziale A_0 la matrice A . A questo punto possiamo procedere nel seguente modo

$$\begin{aligned} A_0 &= Q_0 R_0 \\ A_1 &= R_0 Q_0 \end{aligned}$$

Quindi, per calcolare la successiva matrice di iterazione è sufficiente calcolare il prodotto al contrario delle matrici della fattorizzazione **QR** del passo precedente. Generalizzando quindi questo passo ad un indice n -esimo della successione, si ottiene che

$$\begin{cases} A_0 = A \\ A_{n-1} = Q_{n-1} \cdot R_{n-1} \\ A_n = R_{n-1} \cdot Q_{n-1} \end{cases}$$

Il vantaggio principale di questa successione è proprio deriva dal fatto che tutte le matrici originate dalla successione sono tra di loro simili e, pertanto, mantengono gli stessi autovalori. Lo possiamo dimostrare facilmente attraverso pochi passaggi matematici:

$$\begin{aligned} A_{n+1} &= R_n Q_n \\ &= Q_n^H Q_n R_n Q_n \\ &= Q_n^H A_n Q_n \end{aligned}$$

Siccome, per definizione di matrice hermitiana, l'inversa è uguale alla trasposta coniugata, allora questa definizione è concorde con quella di matrice simile, ergo, la matrice A_{n+1} è simile alla matrice A_n . Sviluppando questo prodotto per ricorrenza si ottiene che

$$A_{n+1} = (Q_n^H \dots Q_1^H) \cdot A \cdot (Q_1 \dots Q_n) = \tilde{Q}_n^H \cdot A \cdot \tilde{Q}_n$$

Visto che A_{n+1} è una matrice simile ad A , i suoi autovettori saranno nella forma $\tilde{Q}_n^H \cdot v$ e quindi, affinché sia possibile ricavare gli autovettori della matrice di partenza, è necessario moltiplicare quelli della matrice simile per $\tilde{Q}_n$. Fondamentalmente, possiamo concludere che l'idea del metodo **QR** consiste nel generare la successione $\{A_n\}$ fino a un certo $\tilde{n}$ sufficientemente alto, tale che la matrice corrispondente è molto vicina ad essere in forma triangolare superiore. Si calcolano quindi gli autovalori e gli autovettori di questa matrice e infine si moltiplicano gli autovettori appena calcolati per $\tilde{Q}_n$ per ottenere le approssimazioni degli autovettori di A . Il problema principale di questo metodo, tuttavia, è che ogni sua iterazione ha un costo di $O(n^3)$; questo costo ha due origini fondamentali: calcolarsi la fattorizzazione **QR** e calcolarsi il prodotto $R_n Q_n$. L'idea per ridurre questo costo è quella di fare un *pre-processing* sulla matrice, in modo da portare il costo di una singola iterazione ad un valore di $O(n^2)$. L'osservazione cruciale è che se la matrice A ha una sola sottodiagonale in cui gli elementi sono diversi da zero, la matrice è in forma di Hessenberg e valgono le seguenti proprietà:

- Il costo computazionale di calcolare la fattorizzazione **QR** e il prodotto RQ è uguale e corrispondente ad $O(n^2)$.
- Il prodotto $R \cdot Q$ è sua volta una matrice in forma di **Hessenberg**. Inoltre, se A_0 è in forma di **Hessenberg**, allora anche A_n è nella medesima forma.

Nel caso in cui la matrice non fosse nella forma di **Hessenberg**, esiste un risultato che asserisce che data una generica matrice A , è sempre possibile trovare una matrice W e una matrice H in forma di Hessenberg tali per cui:

$$A = WHW^H$$

In generale, per trovare queste due matrici, è possibile utilizzare un algoritmo molto simile a quello utilizzato per ricavare la fattorizzazione **QR** della matrice A .

Parte VII

Appendice

APPENDICE A

RICAVARE LA MATRICE DI PERMUTAZIONE PER UNA MATRICE RIDUCIBILE

Data una matrice $A \in \mathbb{C}^{n \times n}$ che sia **riducibile**, quindi tale per cui il suo grafo associato **non è fortemente connesso**, si vuole trovare una matrice di permutazione Π tale per cui $D = \Pi A \Pi^T$ è triangolare a blocchi. Prendiamo come esempio la matrice

$$A = \begin{bmatrix} 3 & 0 & 6 \\ 2 & 4 & 7 \\ 3 & 0 & 0 \end{bmatrix}$$

Ricordiamo che la matrice di permutazione Π non è unica: partendo da una medesima matrice riducibile è possibile ottenere diversi matrici di permutazione. L'idea dell'algoritmo è quella di andare a compilare una tabella dove indichiamo: **nodo**, **quali elementi sono raggiungibili da quel nodo** e **quali elementi non sono raggiungibili da quel nodo**; in particolare:

nodi	raggiungibili	non raggiungibili
1	1, 3	2
2	1, 2, 3	$\emptyset$
3	1, 3	2

Una volta compilate le entrate della tabella si procede scegliendo un indice tale per cui l'insieme dei **nodi non raggiungibili** non sia vuoto. Una volta fatto ciò si prende una matrice identità $I_{n \times n}$ e, supponendo $\bar{n}$ il nodo di riferimento, si permutano le righe di I secondo questa logica:

- Le righe di I relative a indici di nodi non raggiungibili da $\bar{n}$ vanno in **testa**.
- Le righe di I relative a indici di nodi **raggiungibili** da $\bar{n}$ vanno in coda.

Quindi, supponendo di prendere come nodo di riferimento 1, si ha che la riga 2 deve essere portata in testa, mentre la riga 1 deve essere portata in coda. Possiamo quindi fare uno scambio delle due righe ottenendo come matrice di permutazione

$$\Pi = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Abbiamo quindi ottenuto una matrice di permutazione che, moltiplicata opportunamente per la matrice A , la porta in forma triangolare superiore (o inferiore) a blocchi.